Assignment II (Set B) — Robust Reinforcement Learning under Sticky Actuators

Group 109 — Q_learning_SARSA

Group Contribution Declaration

#	Name	BITS ID	Contribution
1	Stuti Srivastava	2025AG05747	100 %
2	Chirag Bhatnagar	2025AG05066	100 %
3	Neelam Devi	2025AG05908	100 %
4	Aditya Saraswat	2025AG05058	100 %

Course: AIML ZG512 Deep Reinforcement Learning · **Group:** 109 (Set B, groups 101–200) · **Master random seed:** `109` (our group number)

The notebook was executed top-to-bottom in the BITS virtual lab; timestamped screenshots of that run are in the Appendix. Key cells print a `[lab-stamp]` line (date, time, host, user) so every output can be matched to a screenshot. Besides the required tasks, the notebook runs extra correctness checks (twelve bonus edge cases, a unit test of both TD targets and of episode-end handling) and saves every trained Q-table, the raw greedy-test results and the complete per-episode training logs to `outputs/`.

Section	Assignment task	Marks
0	Setup: installs, imports, constants, hyper-parameters	–
1	**(a)** Sticky-actuator wrapper + six verification checks	2
2	Shared machinery for (b) and (c): discretiser, validation set, ε schedule, agent, training loop	–
3	**(b)** Q-learning and **(c)** SARSA, each trained on the original and the modified environment	3 + 3
4	**(d)** Five required plots with interpretation + greedy test table	3
5	Extra experiments: cross-evaluation, 3-seed robustness, memory-augmented agent	–
6	**(e)** Discussion	2

0. Setup

0.1 Install the libraries (first code cell, safe to re-run)

In [3]:

```
# Install the exact library versions of the BITS virtual lab (no-op there; pins them
# elsewhere so the run is reproducible).
%pip install -q "gymnasium==1.4.0" "numpy==1.26.4" "matplotlib==3.10.1" "pandas==2.2.3"
```

```
Note: you may need to restart the kernel to use updated packages.
```

0.2 Imports and the run-stamp helper

In [5]:

```
import json  # checkpoint metadata and verification summaries
import os  # paths for the outputs/ folder (CSV logs, figures)
import math  # erfc() for the normal-approximation p-value in part (a)
import time  # wall-clock timing of each training run
import socket  # host name printed in the lab-stamp (identifies the virtual lab machine)
import getpass  # user name printed in the lab-stamp
import platform  # Python version printed in the lab-stamp
import datetime as dt  # timestamps on every progress line

# typed containers for config/results; replace() copies a config
from dataclasses import dataclass, replace

import numpy as np  # numerics, random generators and the Q-tables
import pandas as pd  # tables for verification and evaluation results
import matplotlib  # version string only
import matplotlib.pyplot as plt  # all figures

# 3-colour map for the greedy-policy heatmaps
from matplotlib.colors import ListedColormap

# legend swatches + hatched "unvisited" background
from matplotlib.patches import Patch, Rectangle
import gymnasium as gym  # MountainCar-v0 and the gym.Wrapper base class
from IPython.display import display  # pretty tables inside Jupyter

OUT_DIR = "outputs"  # every CSV log and saved figure goes here
os.makedirs(OUT_DIR, exist_ok=True)  # create the folder if it does not exist yet

pd.set_option("display.width", 220)  # wide tables are printed on one line
pd.set_option("display.max_colwidth", None)  # keep verification descriptions complete
pd.set_option("display.max_columns", 30)  # never truncate the evaluation tables


def now() -> str:
    """Return the current local time as 'YYYY-MM-DD HH:MM:SS' (used in every log
    line)."""
    return dt.datetime.now().strftime("%Y-%m-%d %H:%M:%S")  # formatted wall-clock time


def clock() -> str:
    """Return only the time of day 'HH:MM:SS' (short prefix for progress lines)."""
    return dt.datetime.now().strftime("%H:%M:%S")  # formatted time of day


def current_user() -> str:
    """Login name of the person running the notebook ('unknown' if the OS hides it)."""
    try:  # the login name may be hidden
        return getpass.getuser()  # works on Windows, Linux and most containers
    except Exception:  # some containers have no user entry
        return "unknown"  # fallback name


def lab_stamp(label: str) -> None:
    """Print one evidence line: timestamp, machine, user and a label naming the cell
    that produced it."""
    # one-line stamp
    print(
        f"[lab-stamp] {now()} | host={socket.gethostname()} | user={current_user()} "
```

```
        f"| {label}"
    )


lab_stamp("setup")  # stamp the start of the run
print(
    f"python {platform.python_version()} | gymnasium {gym.__version__} | numpy "
    f"{np.__version__} | "
    # library versions for reproducibility
    f"pandas {pd.__version__} | matplotlib {matplotlib.__version__}"
)
NOTEBOOK_T0 = time.perf_counter()  # used to report the total run time at the end
```

```
[lab-stamp] 2026-11-09 17:16:10 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | setup
python 3.12.7 | gymnasium 1.4.0 | numpy 1.26.4 | pandas 2.2.3 | matplotlib 3.10.1
```

0.3 Figure style

One visual grammar for all figures: **colour = algorithm** (Q-learning blue, SARSA orange) and **line style = environment** (dashed = original, solid = sticky/modified). Quantities with different units (e.g. original vs modified rewards) are never forced onto one shared y-axis; they get separate panels.

In [7]:

```
plt.rcParams.update(
    {  # light, recessive chart chrome so the data stands out
        "figure.dpi": 100,
        "savefig.dpi": 150,
        "savefig.bbox": "tight",
        "figure.facecolor": "#fcfcfb",
        "axes.facecolor": "#fcfcfb",
        "axes.edgecolor": "#c3c2b7",
        "axes.labelcolor": "#52514e",
        "axes.titlecolor": "#0b0b0b",
        "axes.titlesize": 11,
        "axes.titleweight": "bold",
        "axes.labelsize": 10,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.grid": True,
        "grid.color": "#e1e0d9",
        "grid.linewidth": 0.6,
        "xtick.color": "#898781",
        "ytick.color": "#898781",
        "text.color": "#0b0b0b",
        "font.family": "sans-serif",
        "font.size": 9.5,
        "legend.frameon": False,
        "legend.fontsize": 9,
        "lines.linewidth": 1.8,
    }
)
# colour encodes the algorithm
ALGO_COLOR = {"q_learning": "#2a78d6", "sarsa": "#eb6834"}
# display names of the algorithms
ALGO_LABEL = {"q_learning": "Q-learning", "sarsa": "SARSA"}
ENV_STYLE = {"original": "--", "modified": "-"}  # line style encodes the environment
# display names of the environments
ENV_LABEL = {"original": "original", "modified": "sticky"}
INK, INK_2, INK_MUTED = "#0b0b0b", "#52514e", "#898781"  # text / reference-line colours


def save_fig(fig, name: str) -> None:
    """Save a figure to outputs/<name>.png (for the report) and then show it inline."""
    fig.savefig(os.path.join(OUT_DIR, f"{name}.png"))  # high-resolution copy on disk
    plt.show()  # render inside the notebook
```

0.4 Constants from the question and our experimental configuration

Hyper-parameters (identical for all four agents, as required):

Setting	Value	Why
Discretisation	**36 position × 28 velocity** uniform bins	Nominal bin widths are 0.05 and 0.005, putting edges close to the flag and gentle-speed threshold (float32 observation bounds prevent exact decimal

		alignment). This improves resolution near the threshold but does not remove aliasing within bins. A 25 × 25 grid is coarser: it puts every velocity from 0.0252 to 0.0308 into one bin, mixing gentle and too-fast arrivals. 1 008 states × 3 actions stay small enough to be visited many times in 10 000 episodes (finer grids = "narrow features" that need more samples, Lecture 11).
α (learning rate)	0.10	Standard tabular step size; stable under the noisy sticky transitions.
γ (discount)	0.99	MountainCar needs 100–200 steps to reach the flag; γ = 0.99 keeps the +25 bonus visible from ~100 steps away ($0.99^{100} \approx 0.37$).
ε schedule	exponential decay 1.0 → 0.05, floor reached at episode 5 000, then held at 0.05	Broad exploration first; the 0.05 floor keeps SARSA's on-policy values genuinely different from Q-learning's greedy values.
Episodes	10 000 per agent (≥ 6 000 required)	Learning curves are still improving at 6 000 (pilot runs).
Q-table initialisation	zeros, shape (36, 28, 3)	As specified. Zero is optimistic for predominantly negative returns, encouraging exploration. A gentle terminal transition can instead have positive reward (23.7 or 24), so zero is not an upper bound on every action value.
Seed	109 for every run	Same initial base-environment RNG state and matching episode starts. Agent RNGs start identically but consume different numbers of draws as policies, ties and episode lengths diverge; trajectories are not identical.

In [9]:

```
ENV_ID = "MountainCar-v0"  # base environment from Gymnasium
PUSH_LEFT, NO_PUSH, PUSH_RIGHT = 0, 1, 2  # action ids of MountainCar-v0
# readable names
ACTION_NAMES = {PUSH_LEFT: "push left", NO_PUSH: "no push", PUSH_RIGHT: "push right"}
STICK_PROB = 0.30  # probability that an attempted action change is ignored (Step 2)
PUSH_PENALTY = 0.3  # energy penalty for every *selected* push (Step 4)
GENTLE_BONUS = 25.0  # gentle-arrival bonus B (Step 5)
GOAL_POSITION = 0.5  # flag position
GENTLE_SPEED = 0.03  # maximum arrival velocity that earns the bonus
MAX_EPISODE_STEPS = 200  # Gymnasium's TimeLimit for MountainCar-v0
SEED = 109  # master seed = our group number


@dataclass(frozen=True)
class ExperimentConfig:
    """All hyper-parameters shared by Q-learning and SARSA (frozen so no run can change
    them by accident)."""

    n_pos_bins: int = 36  # position bins over [-1.2, 0.6]
    n_vel_bins: int = 28  # velocity bins over [-0.07, 0.07]
    alpha: float = 0.10  # learning rate
    gamma: float = 0.99  # discount factor
    eps_start: float = 1.0  # ε at episode 0
    eps_end: float = 0.05  # ε floor
    # fraction of training after which ε reaches the floor
    eps_decay_fraction: float = 0.5
    n_episodes: int = 10_000  # training episodes per agent
    log_every: int = 100  # print one progress line every N episodes


CFG = ExperimentConfig()  # the single configuration used everywhere
TEST_SEEDS = list(range(50_000, 50_100))  # 100 fixed seeds for the greedy test episodes
# print the configuration as a table
display(pd.DataFrame([vars(CFG)]).T.rename(columns={0: "value"}))
```

	value
n_pos_bins	36.00
n_vel_bins	28.00
alpha	0.10
gamma	0.99

eps_start	1.00
eps_end	0.05
eps_decay_fraction	0.50
n_episodes	10000.00
log_every	100.00

1. Part (a) — Modified environment and verification

1.1 The wrapper

`StickyActuatorMountainCar` follows Steps 1–6 of the question in that order. Three design decisions:

1. **Separate random generator for the actuator.** The stuck/not-stuck draw does *not* use the base environment's `np_random`. Sharing it would shift the base env's random stream and change the initial states. With a separate generator, seed 109 gives the original and the modified environment exactly the same start positions, so the comparison is fair. The generator is re-seeded whenever `reset(seed=…)` is called.
2. **A random number is drawn only when `a ≠ a_prev`**, exactly as in the worked example (step 2 shows "–").
3. **Audit log for verification only.** With `audit=True` the wrapper appends one record per step to `self.audit_log` (selected action, random number, executed action, reward parts). The log is a Python attribute that no agent ever reads; the `info` dictionary returned to the agent stays untouched (Step 6). Training always runs with `audit=False`.

In [11]:

```
class StickyActuatorMountainCar(gym.Wrapper):
    """MountainCar-v0 with a sticky actuator, an energy penalty and a gentle-arrival
    bonus (question Steps 1-6).

    Everything else (observation space, action space, physics, termination and
    truncation) is inherited unchanged from the wrapped environment.
    """

    def __init__(
        self,
        env,
        stick_prob=STICK_PROB,
        push_penalty=PUSH_PENALTY,
        bonus=GENTLE_BONUS,
        goal_position=GOAL_POSITION,
        gentle_speed=GENTLE_SPEED,
        rng=None,
        audit=False,
    ):
        """Store the modification parameters; `rng` may be injected for unit tests,
        `audit` enables the log."""
        super().__init__(env)  # gym.Wrapper keeps spaces/spec of the base env
        self.stick_prob = float(stick_prob)  # P(replace an attempted change by a_prev)
        self.push_penalty = float(push_penalty)  # energy cost of a selected push
        self.bonus = float(bonus)  # gentle-arrival bonus B
        # flag position used in the bonus test
        self.goal_position = float(goal_position)
        # speed threshold used in the bonus test
        self.gentle_speed = float(gentle_speed)
        # True when a test supplies its own generator
        self._rng_injected = rng is not None
        # actuator random generator
        self._rng = rng if rng is not None else np.random.default_rng()
        self.a_prev = NO_PUSH  # action actually executed on the previous step
        self.audit = bool(audit)  # whether step() records an audit entry
        self.audit_log = []  # verification records (never shown to the agent)

    def reset(self, *, seed=None, options=None):
        """Reset the base env, set a_prev = 1 (do not accelerate) and re-seed the
        actuator generator if seeded."""
        # base reset (initial position is random)
        obs, info = self.env.reset(seed=seed, options=options)
        self.a_prev = NO_PUSH  # question: a_prev = 1 on every reset
        # a seeded reset makes the actuator reproducible
        if seed is not None and not self._rng_injected:
            # independent stream derived from the same seed
            self._rng = np.random.default_rng([seed, 2026])
        return obs, info  # observation and info are passed through

    def _gentle_arrival_bonus(self, obs, terminated, truncated) -> float:
        """Step 5: return B if ALL four gentle-arrival conditions hold, otherwise 0."""
```

```
        # car position at/after the flag
        reached_flag = bool(obs[0] >= self.goal_position)
        slow_enough = bool(obs[1] <= self.gentle_speed)  # arrival velocity at most 0.03
        # all conditions simultaneously
        if terminated and not truncated and reached_flag and slow_enough:
            return self.bonus  # gentle arrival
        return 0.0  # any failed condition -> no bonus

    def step(self, action):
        """Apply Steps 1-6 of the question in order and return (obs, R, terminated,
        truncated, info)."""
        a = int(action)  # Step 1: the agent's selected action (kept)
        a_prev_before = self.a_prev  # remembered only for the audit record
        r_draw = np.nan  # no random number unless the action changes
        if a == self.a_prev:  # Step 2: repeating the executed action...
            a_exec = a  # ...is never modified
        else:  # attempted change of action
            r_draw = float(self._rng.random())  # r ~ U[0, 1)
            # stuck -> keep previous executed action
            a_exec = self.a_prev if r_draw < self.stick_prob else a
        # Step 3: execute in the base env
        obs, r_base, terminated, truncated, info = self.env.step(a_exec)
        self.a_prev = a_exec  # update a_prev with the *executed* action
        # Step 4: penalty uses the *selected* action
        penalty = self.push_penalty if a != NO_PUSH else 0.0
        bonus = self._gentle_arrival_bonus(obs, terminated, truncated)  # Step 5
        reward = float(r_base) - penalty + bonus  # R = R_base - 0.3*1[a != 1] + B
        if self.audit:  # verification-only bookkeeping
            # one audit record per step
            self.audit_log.append(
                dict(
                    a_prev=a_prev_before,
                    a=a,
                    r=r_draw,
                    a_exec=a_exec,
                    r_base=float(r_base),
                    penalty=penalty,
                    bonus=bonus,
                    R=reward,
                    pos=float(obs[0]),
                    vel=float(obs[1]),
                    terminated=bool(terminated),
                    truncated=bool(truncated),
                )
            )
        # Step 6: info is the base env's, unchanged
        return obs, reward, terminated, truncated, info


def make_env(kind: str, audit: bool = False, stick_prob: float = STICK_PROB):
    """Create the 'original' MountainCar-v0 (with its 200-step TimeLimit) or the
    'modified' sticky version."""
    base = gym.make(ENV_ID)  # includes TimeLimit(200) and OrderEnforcing
    if kind == "original":  # unmodified environment
        return base  # unmodified environment
    if kind == "modified":  # our wrapper sits on top of the TimeLimit
        # sticky actuator + reward changes
        return StickyActuatorMountainCar(base, stick_prob=stick_prob, audit=audit)
    raise ValueError(f"unknown environment kind: {kind}")  # guard against typos


print(make_env("modified"))  # show the wrapper stack
```

```
<StickyActuatorMountainCar<TimeLimit<OrderEnforcing<PassiveEnvChecker<MountainCarEnv<MountainCar-v0>>>>>>
```

1.2 Verification V1 — the worked example from the question, reproduced exactly

A scripted random generator returns the question's random numbers (0.62, 0.27, 0.81, 0.18), and the agent selects the question's actions (0, 0, 2, 2, 1). The wrapper must produce the executed actions in the table and must *not* draw a random number at step 2.

In [13]:

```
class ScriptedRNG:
    """Stand-in random generator that returns a fixed list of numbers and counts how
```

```
    many were requested."""

    def __init__(self, values):
        """Store the numbers to hand out, in order."""
        self.values = list(values)  # the scripted random numbers
        self.calls = 0  # how many numbers the wrapper has asked for

    def random(self) -> float:
        """Return the next scripted number (same interface as numpy's
        Generator.random)."""
        value = self.values[self.calls]  # next number in the script
        self.calls += 1  # count the draw
        return value  # return the scripted number


scripted = ScriptedRNG([0.62, 0.27, 0.81, 0.18])  # r values from the question's table
# wrapper with scripted draws
env_ex = StickyActuatorMountainCar(gym.make(ENV_ID), rng=scripted, audit=True)
env_ex.reset(seed=SEED)  # fresh episode -> a_prev = 1
for a in [0, 0, 2, 2, 1]:  # the agent actions from the question
    env_ex.step(a)  # run one step
# recorded steps
ex = pd.DataFrame(env_ex.audit_log)[["a_prev", "a", "r", "a_exec", "penalty", "R"]]
ex.insert(0, "step", range(1, 6))  # step numbers 1..5
ex["expected a_exec"] = [0, 0, 0, 2, 2]  # executed actions from the question
ex["expected a_prev"] = [1, 0, 0, 0, 2]  # a_prev column from the question
# row check
ex["match"] = (ex["a_exec"] == ex["expected a_exec"]) & (
    ex["a_prev"] == ex["expected a_prev"]
)
# print "–" where no number was drawn
ex["r"] = ex["r"].map(lambda v: "–" if np.isnan(v) else f"{v:.2f}")
display(ex)  # show the reproduced example with checks
# all rows match and exactly 4 draws
V1_PASS = bool(ex["match"].all() and scripted.calls == 4)
# verdict of check V1
print(
    f"random numbers requested: {scripted.calls} (expected 4) -> V1 "
    f"{'PASS' if V1_PASS else 'FAIL'}"
)
```

	step	a_prev	a	r	a_exec	penalty	R	expected a_exec	expected a_prev	match
0	1	1	0	0.62	0	0.3	-1.3	0	1	True
1	2	0	0	–	0	0.3	-1.3	0	0	True
2	3	0	2	0.27	0	0.3	-1.3	0	0	True
3	4	0	2	0.81	2	0.3	-1.3	2	0	True
4	5	2	1	0.18	2	0.0	-1.0	2	2	True

```
random numbers requested: 4 (expected 4) -> V1 PASS
```

1.3 Verification V2 — the interface is unchanged

In [15]:

```
# one of each environment
base_env, mod_env = make_env("original"), make_env("modified")
obs_b, info_b = base_env.reset(seed=SEED)  # same seed for both resets
obs_m, info_m = mod_env.reset(seed=SEED)  # same seed for the modified env
info_keys_b, info_keys_m = set(info_b), set(info_m)  # info keys seen during a short run
rng_v2 = np.random.default_rng([SEED, 21])  # random actions for the short run
for _ in range(300):  # 300 random steps in both envs
    act = int(rng_v2.integers(3))  # same action sent to both
    # send the same action to both envs
    for env_, keys in ((base_env, info_keys_b), (mod_env, info_keys_m)):
        _, _, te, tr, inf = env_.step(act)  # step and collect the info keys
        keys |= set(inf)  # collect any info keys returned
        if te or tr:  # restart a finished episode
            env_.reset()  # start a new episode
v2 = pd.DataFrame(
    [
        (
            "observation space",
            str(base_env.observation_space),
            str(mod_env.observation_space),
        ),
```

```
        ("action space", str(base_env.action_space), str(mod_env.action_space)),
        (
            "max episode steps",
            base_env.spec.max_episode_steps,
            mod_env.spec.max_episode_steps,
        ),
        (
            "initial observation (seed 109)",
            np.round(obs_b, 6).tolist(),
            np.round(obs_m, 6).tolist(),
        ),
        ("info keys seen in 300 steps", sorted(info_keys_b), sorted(info_keys_m)),
    ],
    columns=["property", "original", "modified"],
)  # side-by-side comparison
# row-wise equality
v2["identical"] = [str(x) == str(y) for x, y in zip(v2["original"], v2["modified"])]
display(v2)  # show the side-by-side comparison
V2_PASS = bool(v2["identical"].all())  # everything must be identical
print(f"V2 {'PASS' if V2_PASS else 'FAIL'}")  # verdict of check V2
```

	property	original	modified	identical
0	observation space	Box([-1.2 -0.07], [0.6 0.07], (2,), float32)	Box([-1.2 -0.07], [0.6 0.07], (2,), float32)	True
1	action space	Discrete(3)	Discrete(3)	True
2	max episode steps	200	200	True
3	initial observation (seed 109)	[-0.49125200510025024, 0.0]	[-0.49125200510025024, 0.0]	True
4	info keys seen in 300 steps	[]	[]	True

```
V2 PASS
```

1.4 Verification V3 — about 30 % of attempted changes are replaced; repeats are never replaced

We run **300 episodes (60 000 steps) of a uniformly random policy** on the modified environment and read the audit log. Each episode uses its own seed (`1000 + episode`) so it can be replayed exactly in V5.

In [17]:

```
def run_policy_with_audit(policy, n_episodes: int, seed_base: int, rng_seed: int):
    """Run `policy(obs, rng)` on the audited modified env; return one DataFrame row per
    step (with episode ids)."""
    env = make_env("modified", audit=True)  # audited wrapper (verification only)
    # random generator for the policy itself
    rng = np.random.default_rng([SEED, rng_seed])
    rows = []  # collected audit records
    for ep in range(n_episodes):  # one episode per seed
        # per-episode seed -> exactly replayable
        obs, _ = env.reset(seed=seed_base + ep)
        env.audit_log.clear()  # start a fresh log for this episode
        while True:  # act until the episode ends
            # act with the given policy
            obs, _, terminated, truncated, _ = env.step(policy(obs, rng))
            if terminated or truncated:  # episode finished
                break  # leave the step loop
        # tag every record with episode, seed and time index
        for t, rec in enumerate(env.audit_log):
            # tag record with episode, seed and step index
            rec.update(episode=ep, seed=seed_base + ep, t=t)
        rows.extend(env.audit_log)  # keep this episode's records
    env.close()  # release the environment after the audit
    return pd.DataFrame(rows)  # one row per audited step


def random_policy(obs, rng) -> int:
    """Uniformly random action (ignores the observation)."""
    return int(rng.integers(3))  # 0, 1 or 2 with equal probability


def wilson_interval(k: int, n: int, z: float = 1.96):
    """95 % Wilson score interval for a binomial proportion k/n."""
```

```
    p = k / n  # observed proportion
    centre = (p + z * z / (2 * n)) / (1 + z * z / n)  # Wilson centre
    # half-width
    half = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / (1 + z * z / n)
    return centre - half, centre + half  # lower and upper bound


lab_stamp("part (a) - random-policy audit")  # stamp the start of the audit
# 60 000 audited steps
audit = run_policy_with_audit(
    random_policy, n_episodes=300, seed_base=1000, rng_seed=31
)
# steps where the agent tried to switch
audit["attempted_change"] = audit["a"] != audit["a_prev"]
# steps where the actuator ignored the agent
audit["replaced"] = audit["a_exec"] != audit["a"]
changes = audit[audit["attempted_change"]]  # subset: attempted changes
repeats = audit[~audit["attempted_change"]]  # subset: repeated actions
# replacements among attempted changes
k, n = int(changes["replaced"].sum()), len(changes)
p_hat = k / n  # observed replacement rate
lo, hi = wilson_interval(k, n)  # 95 % confidence interval
# z-statistic against H0: p = 0.30
z = (p_hat - STICK_PROB) / math.sqrt(STICK_PROB * (1 - STICK_PROB) / n)
p_value = math.erfc(abs(z) / math.sqrt(2))  # two-sided p-value
# size of the audit
print(f"steps audited: {len(audit):,} in {audit['episode'].nunique()} episodes")
# replacement statistics with CI and z-test
print(
    f"attempted changes (a != a_prev): {n:,} | replaced by a_prev: {k:,} | rate = "
    f"{p_hat:.4f} "
    f"| 95% CI [{lo:.4f}, {hi:.4f}] | z = {z:+.2f}, p = {p_value:.2f}"
)
# repeats must never be replaced
print(
    f"repeated actions (a == a_prev): {len(repeats):,} | replaced: "
    f"{int(repeats['replaced'].sum())} "
    f"| random numbers drawn on repeats: {int(repeats['r'].notna().sum())}"
)
# all three conditions of V3
V3_PASS = bool(
    lo <= STICK_PROB <= hi
    and repeats["replaced"].sum() == 0
    and repeats["r"].notna().sum() == 0
)
# verdict of check V3
print(
    f"V3 {'PASS' if V3_PASS else 'FAIL'} (0.30 inside the CI, zero replacements and "
    "zero draws on repeats)"
)

# replacement rate for every (a_prev -> a) transition; the diagonal holds the repeats
trans = audit.groupby(["a_prev", "a"])["replaced"].agg(["mean", "size"]).reset_index()
# "rate (n)"
trans["cell"] = trans.apply(
    lambda r: f"{r['mean']:.3f}  (n={int(r['size']):,})", axis=1
)
trans_table = trans.pivot(index="a_prev", columns="a", values="cell")  # 3 x 3 table
# readable row labels
trans_table.index = [f"a_prev={i} ({ACTION_NAMES[i]})" for i in trans_table.index]
# readable column labels
trans_table.columns = [f"a={j} ({ACTION_NAMES[j]})" for j in trans_table.columns]
# heading for the 3 x 3 table
print("\nReplacement rate by transition (diagonal = repeats, must be 0):")
display(trans_table)  # show the 3 x 3 table
```

```
[lab-stamp] 2026-11-09 17:16:11 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | part
(a) - random-policy audit
steps audited: 60,000 in 300 episodes
attempted changes (a != a_prev): 39,914 | replaced by a_prev: 12,037 | rate = 0.3016 | 95% CI [0.2971, 0.3061]
| z = +0.69, p = 0.49
repeated actions (a == a_prev): 20,086 | replaced: 0 | random numbers drawn on repeats: 0
V3 PASS (0.30 inside the CI, zero replacements and zero draws on repeats)

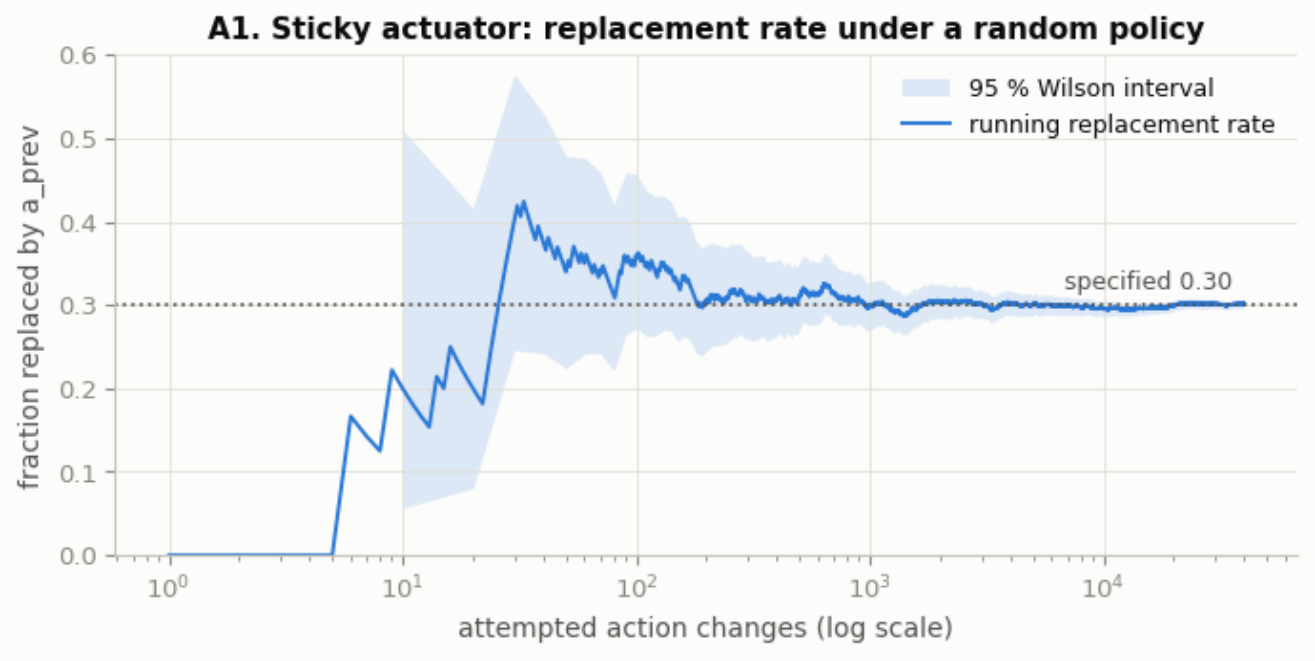
Replacement rate by transition (diagonal = repeats, must be 0):
```

	a=0 (push left)	a=1 (no push)	a=2 (push right)

a_prev=0 (push left)	0.000 (n=6,601)	0.307 (n=6,518)	0.300 (n=6,664)
a_prev=1 (no push)	0.300 (n=6,794)	0.000 (n=6,763)	0.310 (n=6,802)
a_prev=2 (push right)	0.299 (n=6,462)	0.292 (n=6,674)	0.000 (n=6,722)

In [18]:

```
# Figure A1: running replacement rate converging to 0.30 as attempted changes accumulate
cum_k = changes["replaced"].cumsum().to_numpy()  # cumulative replacements
cum_n = np.arange(1, n + 1)  # cumulative attempted changes
cum_p = cum_k / cum_n  # running rate
# CI every 10
band = np.array(
    [wilson_interval(int(kk), int(nn)) for kk, nn in zip(cum_k[9::10], cum_n[9::10])]
)
fig, ax = plt.subplots(figsize=(8, 3.4))  # one wide panel
# shaded confidence band
ax.fill_between(
    cum_n[9::10],
    band[:, 0],
    band[:, 1],
    color=ALGO_COLOR["q_learning"],
    alpha=0.15,
    lw=0,
    label="95 % Wilson interval",
)
# running replacement rate
ax.plot(
    cum_n,
    cum_p,
    color=ALGO_COLOR["q_learning"],
    lw=1.4,
    label="running replacement rate",
)
ax.axhline(STICK_PROB, color=INK_2, ls=":", lw=1.2)  # target probability
# label the reference line
ax.annotate(
    "specified 0.30",
    (cum_n[-1], STICK_PROB),
    xytext=(-4, 6),
    textcoords="offset points",
    ha="right",
    color=INK_2,
    fontsize=9,
)
# log x-axis shows early and late behaviour
ax.set(
    xscale="log",
    ylim=(0, 0.6),
    xlabel="attempted action changes (log scale)",
    ylabel="fraction replaced by a_prev",
    title="A1. Sticky actuator: replacement rate under a random policy",
)
ax.legend(loc="upper right")  # legend in the empty corner
save_fig(fig, "A1_replacement_rate")  # save and show
```



Reading A1 and the transition table. The running rate is noisy over the first few hundred switches, settles near 0.30 after ≈ 1 000 attempted changes, and ends at 0.302 with a 95 % interval [0.297, 0.306] that contains 0.30 (z = +0.69, p = 0.49: no evidence of any deviation). The 3 × 3 table shows ≈ 0.30 for *every* kind of switch (left → right, none → left, …) and exactly 0 on the diagonal, where the agent repeats its executed action.

1.5 Verification V4 — the energy penalty follows the *selected* action

Each step is placed in one of the four cases from the question's note. The penalty the agent actually received is recovered from the returned reward as `R_base + B − R`, where B is recomputed independently from the observation and flags.

In [21]:

```
def independent_bonus(df: pd.DataFrame) -> np.ndarray:
    """Recompute B from the logged observation and flags, without using the wrapper's
    own bonus value."""
    # all four bonus conditions
    ok = (
        df["terminated"]
        & ~df["truncated"]
        & (df["pos"] >= GOAL_POSITION)
        & (df["vel"] <= GENTLE_SPEED)
    )
    return np.where(ok, GENTLE_BONUS, 0.0)  # 25 where all conditions hold, else 0


audit["B_check"] = independent_bonus(audit)  # independently recomputed bonus
# what the agent was charged
audit["penalty_received"] = audit["r_base"] + audit["B_check"] - audit["R"]
audit["case"] = np.select(  # the four cases from the question's note
    [
        (audit["a"] != NO_PUSH) & ~audit["replaced"],
        (audit["a"] != NO_PUSH) & audit["replaced"],
        (audit["a"] == NO_PUSH) & audit["replaced"],
        (audit["a"] == NO_PUSH) & ~audit["replaced"],
    ],
    [
        "selected push, executed as selected",
        "selected push, actuator stuck on another action",
        "selected 'no push', stuck actuator keeps pushing",
        "selected 'no push', executed 'no push'",
    ],
    default="unclassified",
)  # never used: the four cases cover every step
# penalty statistics per case
v4 = audit.groupby("case", sort=False)["penalty_received"].agg(
    steps="size", min_penalty="min", max_penalty="max"
```

```
)
# rule from the question
v4["expected"] = [0.3 if c.startswith("selected push") else 0.0 for c in v4.index]
# observed penalty equals the rule in every step
v4["match"] = np.isclose(v4["min_penalty"], v4["expected"]) & np.isclose(
    v4["max_penalty"], v4["expected"]
)
display(v4.round(6))  # show the four cases
# largest deviation from the reward formula
max_err = float(
    np.abs(
        audit["R"]
        - (audit["r_base"] - PUSH_PENALTY * (audit["a"] != NO_PUSH) + audit["B_check"])
    ).max()
)
# all conditions of V4
V4_PASS = bool(
    v4["match"].all() and max_err < 1e-12 and (audit["r_base"] == -1.0).all()
)
# verdict of check V4
print(
    f"max |R - (R_base - 0.3*1[a!=1] + B)| over {len(audit):,} steps = {max_err:.1e} "
    f"-> V4 {'PASS' if V4_PASS else 'FAIL'}"
)
```

	steps	min_penalty	max_penalty	expected	match
case					
selected 'no push', executed 'no push'	16003	0.0	0.0	0.0	True
selected push, executed as selected	31960	0.3	0.3	0.3	True
selected 'no push', stuck actuator keeps pushing	3952	0.0	0.0	0.0	True
selected push, actuator stuck on another action	8085	0.3	0.3	0.3	True

```
max |R - (R_base - 0.3*1[a!=1] + B)| over 60,000 steps = 0.0e+00 -> V4 PASS
```

1.6 Verification V5 — physics, termination and truncation are unchanged

For 25 audited episodes we replay the *executed* actions in a fresh, unwrapped `MountainCar-v0` with the same reset seed. If the wrapper changed nothing except the action substitution, every observation and every terminated/truncated flag must match exactly.

In [23]:

```
mismatches, compared = 0, 0  # counters over all replayed steps
for ep, ep_log in list(audit.groupby("episode"))[:25]:  # first 25 audited episodes
    env_r = gym.make(ENV_ID)  # fresh unwrapped environment
    # same initial state as the audited run
    env_r.reset(seed=int(ep_log["seed"].iloc[0]))
    for rec in ep_log.itertuples():  # replay executed actions one by one
        o, _, te, tr, _ = env_r.step(int(rec.a_exec))  # replay the executed action
        # compare position, velocity and flags exactly
        same = (
            (float(o[0]) == rec.pos)
            and (float(o[1]) == rec.vel)
            and te == rec.terminated
            and tr == rec.truncated
        )
        mismatches += int(not same)  # count any difference
        compared += 1  # count compared steps
    env_r.close()  # release each replay environment
V5_PASS = mismatches == 0  # V5 passes only with zero mismatches
# verdict of check V5
print(
    f"replayed {compared:,} steps from 25 episodes: {mismatches} mismatches -> V5 "
    f"{'PASS' if V5_PASS else 'FAIL'}"
)
```

```
replayed 5,000 steps from 25 episodes: 0 mismatches -> V5 PASS
```

1.7 Verification V6 — the +25 bonus is paid only for a gentle arrival

A random policy almost never reaches the flag, so we use hand-coded policies. **Pure "push with the velocity" (the question's suggestion) is not enough here:** in the sampled original-environment runs it arrived at about 0.033–0.050. This observation does not prove that the policy can never earn the bonus. (In the table below it does earn it

in 10 of 60 episodes, but only because the sticky actuator sometimes delays its pushes and bleeds off speed.) We therefore add *pump-then-coast* variants: push with the velocity until position x_c, then stop pushing and let gravity slow the car. Varying x_c gives a mix of gentle arrivals, fast arrivals and time-outs.

Then **twelve boundary unit tests** use a scripted fake base environment to hit the exact edge cases: v = 0.03 (inclusive), v = 0.0301, and reaching the flag on step 200 (terminated *and* truncated).

In [25]:

```
def pump_policy(obs, rng) -> int:
    """Hand-coded policy from webinar 1: push in the direction of the velocity."""
    return PUSH_RIGHT if obs[1] >= 0 else PUSH_LEFT  # push in the direction of motion


def make_pump_and_coast(x_coast: float):
    """Return a policy that pumps energy, then coasts (no push) once moving right beyond
    position x_coast."""

    def policy(obs, rng) -> int:
        """Push with the velocity, except coast on the final rightward approach."""
        if obs[1] < 0:  # moving left: keep pumping left
            return PUSH_LEFT  # keep pumping left while moving left
        # moving right: coast after x_coast
        return NO_PUSH if obs[0] > x_coast else PUSH_RIGHT

    return policy  # the configured policy


lab_stamp("part (a) - bonus audit")  # stamp the start of the bonus audit
policies = {"pump (push with velocity)": pump_policy}  # baseline heuristic
# six coasting thresholds
policies.update(
    {
        f"pump, coast after x>{xc:+.1f}": make_pump_and_coast(xc)
        for xc in (-0.2, -0.1, 0.0, 0.1, 0.2, 0.3)
    }
)
bonus_rows = []  # one row per policy
terminal_steps = []  # last step of every episode (for the pooled check)
for i, (name, pol) in enumerate(policies.items()):  # run every hand-coded policy
    # 60 audited episodes per policy
    log = run_policy_with_audit(
        pol, n_episodes=60, seed_base=20_000 + 100 * i, rng_seed=40 + i
    )
    last = log.groupby("episode").tail(1).copy()  # final step of each episode
    last["policy"] = name  # remember which policy produced it
    terminal_steps.append(last)  # collect terminal steps
    # gentle arrivals (bonus expected)
    gentle = last["terminated"] & ~last["truncated"] & (last["vel"] <= GENTLE_SPEED)
    # fast arrivals (no bonus expected)
    fast = last["terminated"] & (last["vel"] > GENTLE_SPEED)
    # summary row for this policy
    bonus_rows.append(
        dict(
            policy=name,
            episodes=len(last),
            gentle_arrivals=int(gentle.sum()),
            fast_arrivals=int(fast.sum()),
            timeouts=int((~last["terminated"]).sum()),
            bonus_paid=int((last["bonus"] == GENTLE_BONUS).sum()),
            bonus_on_fast_or_timeout=int((last["bonus"][~gentle] != 0).sum()),
        )
    )
display(pd.DataFrame(bonus_rows))  # show the per-policy summary
term = pd.concat(terminal_steps)  # pooled terminal steps of all 420 episodes
# random-policy audit: on all 60 000 steps the bonus must equal the independent rule
random_bonus_ok = bool((audit["bonus"] == audit["B_check"]).all())
paid = term[term["bonus"] == GENTLE_BONUS]  # episodes that earned the bonus
# arrivals without the bonus
unpaid_arrivals = term[(term["bonus"] == 0) & term["terminated"]]
# boundary sharpness of the bonus rule
print(
    f"pooled: {len(term)} episodes | bonus paid {len(paid)} times | "
    f"max arrival speed among paid = {paid['vel'].max():.4f} (must be <= 0.03) | "
```

```
    f"min arrival speed among unpaid arrivals = {unpaid_arrivals['vel'].min():.4f} "
    "(must be > 0.03)"
)
# wrapper vs independent rule
pooled_ok = bool((term["bonus"].to_numpy() == independent_bonus(term)).all())


# --- boundary unit tests with a scripted fake base environment
# ----------------------------------------
class ScriptedBaseEnv(gym.Env):
    """Fake MountainCar whose next step returns a scripted (position, velocity,
    terminated, truncated)."""

    def __init__(self, pos, vel, terminated, truncated):
        """Store the outcome that step() will return; spaces copy MountainCar-v0."""
        # same bounds as MountainCar-v0
        self.observation_space = gym.spaces.Box(
            np.array([-1.2, -0.07], np.float32), np.array([0.6, 0.07], np.float32)
        )
        self.action_space = gym.spaces.Discrete(3)  # three actions
        # scripted step result
        self.outcome = (
            np.array([pos, vel], np.float32),
            -1.0,
            terminated,
            truncated,
            {},
        )

    def reset(self, *, seed=None, options=None):
        """Return an arbitrary valley-floor state."""
        super().reset(seed=seed)  # seed the gym.Env base class
        return np.array([-0.5, 0.0], np.float32), {}  # valley-floor start state

    def step(self, action):
        """Return the scripted outcome regardless of the action."""
        return self.outcome  # scripted (obs, reward, terminated, truncated, info)


cases = [  # (description, pos, vel, terminated, truncated, expected B)
    ("gentle arrival, v = 0.020", 0.52, 0.020, True, False, 25.0),
    ("boundary, v = 0.030 exactly (inclusive)", 0.52, 0.030, True, False, 25.0),
    ("just too fast, v = 0.0301", 0.52, 0.0301, True, False, 0.0),
    ("fast arrival, v = 0.045", 0.55, 0.045, True, False, 0.0),
    (
        "flag reached ON step 200 (terminated and truncated)",
        0.51,
        0.010,
        True,
        True,
        0.0,
    ),
    ("time-out below the flag", 0.40, 0.010, False, True, 0.0),
    ("ordinary mid-episode step", 0.30, 0.010, False, False, 0.0),
]
# five more edge cases: each bonus condition failing alone, the exact flag position,
# and the float32 neighbours just below / above the 0.03 threshold
cases.extend(
    [
        ("terminated below the flag", 0.49, 0.010, True, False, 0.0),
        ("at flag but not terminated", 0.51, 0.010, False, False, 0.0),
        ("exact flag position", 0.50, 0.020, True, False, 25.0),
        (
            "float32 speed just below threshold",
            0.52,
            float(np.nextafter(np.float32(0.03), np.float32(-np.inf))),
            True,
            False,
            25.0,
        ),
        (
            "float32 speed just above threshold",
            0.52,
            float(np.nextafter(np.float32(0.03), np.float32(np.inf))),
            True,
```

```
            False,
            0.0,
        ),
    ]
)  # exercise each bonus gate independently and catch threshold off-by-one errors
unit_rows = []  # results of the boundary tests
for desc, pos, vel, te, tr, expected in cases:  # one fake env per edge case
    # wrap the fake env
    w = StickyActuatorMountainCar(ScriptedBaseEnv(pos, vel, te, tr))
    w.reset(seed=0)  # fresh episode (a_prev = 1)
    _, R, _, _, _ = w.step(NO_PUSH)  # 'no push' -> no penalty, no random draw
    # R + 1 recovers B (R_base = -1, no penalty)
    unit_rows.append(
        dict(
            case=desc,
            terminated=te,
            truncated=tr,
            velocity=vel,
            B_expected=expected,
            B_returned=R + 1.0,
            match=np.isclose(R + 1.0, expected),
        )
    )
unit = pd.DataFrame(unit_rows)  # table of the 12 cases
display(unit)  # show the unit tests
# all bonus conditions verified
V6_PASS = bool(
    len(paid) > 0
    and paid["vel"].max() <= GENTLE_SPEED
    and unpaid_arrivals["vel"].min() > GENTLE_SPEED
    and random_bonus_ok
    and pooled_ok
    and unit["match"].all()
)
# verdict of check V6, with the two pooled consistency checks spelled out
print(
    f"bonus = independent rule on all {len(audit):,} random-policy steps: "
    f"{random_bonus_ok} | on all {len(term)} hand-coded episodes: {pooled_ok}"
)
print(f"V6 {'PASS' if V6_PASS else 'FAIL'}")  # verdict of check V6
```

```
[lab-stamp] 2026-11-09 17:16:16 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | part (a) - bonus audit
```

	policy	episodes	gentle_arrivals	fast_arrivals	timeouts	bonus_paid	bonus_on_fast_or_timeout
0	pump (push with velocity)	60	10	50	0	10	0
1	pump, coast after x>-0.2	60	20	33	7	20	0
2	pump, coast after x>-0.1	60	17	43	0	17	0
3	pump, coast after x>+0.0	60	12	48	0	12	0
4	pump, coast after x>+0.1	60	13	47	0	13	0
5	pump, coast after x>+0.2	60	11	49	0	11	0
6	pump, coast after x>+0.3	60	10	50	0	10	0

```
pooled: 420 episodes | bonus paid 93 times | max arrival speed among paid = 0.0298 (must be <= 0.03) | min arrival speed among unpaid arrivals = 0.0301 (must be > 0.03)
```

	case	terminated	truncated	velocity	B_expected	B_returned	match
0	gentle arrival, v = 0.020	True	False	0.0200	25.0	25.0	True
1	boundary, v = 0.030 exactly (inclusive)	True	False	0.0300	25.0	25.0	True
2	just too fast, v = 0.0301	True	False	0.0301	0.0	0.0	True
3	fast arrival, v = 0.045	True	False	0.0450	0.0	0.0	True
4	flag reached ON step 200 (terminated and truncated)	True	True	0.0100	0.0	0.0	True
5	time-out below the flag	False	True	0.0100	0.0	0.0	True
6	ordinary mid-episode step	False	False	0.0100	0.0	0.0	True
7	terminated below the flag	True	False	0.0100	0.0	0.0	True
8	at flag but not terminated	False	False	0.0100	0.0	0.0	True
9	exact flag position	True	False	0.0200	25.0	25.0	True
10	float32 speed just below threshold	True	False	0.0300	25.0	25.0	True
11	float32 speed just above threshold	True	False	0.0300	0.0	0.0	True

```
bonus = independent rule on all 60,000 random-policy steps: True | on all 420 hand-coded episodes: True
V6 PASS
```

1.8 Verification summary

In [27]:

```
summary_a = pd.DataFrame(
    [
        (
            "V1",
            "Question's worked example reproduced step-for-step (4 random draws, none "
            "on the repeat)",
            V1_PASS,
        ),
        (
            "V2",
            "Observation/action spaces, time limit, initial state and info keys "
            "unchanged",
            V2_PASS,
        ),
        (
            "V3",
            f"Replacement rate {p_hat:.3f} on {n:,} attempted changes (CI contains "
            "0.30); 0 replacements on repeats",
            V3_PASS,
        ),
        (
            "V4",
            "Penalty 0.3 for every selected push (stuck or not), 0 for every selected "
            "'no push'",
            V4_PASS,
        ),
        (
            "V5",
            "Executed actions replayed in the plain env give identical observations "
            "and flags",
            V5_PASS,
        ),
        (
            "V6",
            "Bonus paid only for gentle arrivals (hand-coded runs + 12 boundary unit "
            "tests)",
            V6_PASS,
        ),
    ],
    columns=["check", "what is verified", "pass"],
)
display(summary_a)  # show the six checks
# stop the notebook if any check failed
assert summary_a[
    "pass"
].all(), "a verification check failed - fix the wrapper before training"
lab_stamp("part (a) complete")  # stamp the end of part (a)
```

	check	what is verified	pass
0	V1	Question's worked example reproduced step-for-step (4 random draws, none on the repeat)	True
1	V2	Observation/action spaces, time limit, initial state and info keys unchanged	True
2	V3	Replacement rate 0.302 on 39,914 attempted changes (CI contains 0.30); 0 replacements on repeats	True
3	V4	Penalty 0.3 for every selected push (stuck or not), 0 for every selected 'no push'	True
4	V5	Executed actions replayed in the plain env give identical observations and flags	True
5	V6	Bonus paid only for gentle arrivals (hand-coded runs + 12 boundary unit tests)	True

```
[lab-stamp] 2026-11-09 17:16:19 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | part
(a) complete
```

2. Shared machinery for parts (b) and (c)

2.1 Uniform discretiser

In [29]:

```
class UniformDiscretizer:
    """Map a continuous (position, velocity) observation to integer bin indices on a
    uniform grid."""

    def __init__(self, low, high, bins):
        """Pre-compute bin widths and edges for each observation dimension."""
        self.low = np.asarray(low, dtype=float)  # lower bounds of the observation box
        self.high = np.asarray(high, dtype=float)  # upper bounds
        self.bins = tuple(int(b) for b in bins)  # bins per dimension
        # bin width per dimension
        self.width = (self.high - self.low) / np.array(self.bins)
        # bin edges
        self.edges = [
            np.linspace(l, h, b + 1) for l, h, b in zip(self.low, self.high, self.bins)
        ]
        # cached scalars for speed
        self._lo0, self._lo1 = float(self.low[0]), float(self.low[1])
        # cached bin widths
        self._w0, self._w1 = float(self.width[0]), float(self.width[1])
        self._n0, self._n1 = self.bins[0] - 1, self.bins[1] - 1  # largest valid index

    def index(self, obs) -> tuple:
        """Return (position_bin, velocity_bin) for one observation, clipped to the
        grid."""
        i = int((float(obs[0]) - self._lo0) / self._w0)  # position bin
        j = int((float(obs[1]) - self._lo1) / self._w1)  # velocity bin
        # clip the top edge into the last bin
        return (min(max(i, 0), self._n0), min(max(j, 0), self._n1))

    def index_batch(self, states: np.ndarray):
        """Vectorised version of index() for an (N, 2) array; returns two integer
        arrays."""
        idx = ((states - self.low) / self.width).astype(int)  # raw bin indices
        idx = np.clip(idx, 0, np.array(self.bins) - 1)  # clip into the grid
        return idx[:, 0], idx[:, 1]  # position and velocity indices

    def centers(self, dim: int) -> np.ndarray:
        """Bin centres along one dimension (used for axis ticks)."""
        e = self.edges[dim]  # edges of this dimension
        return (e[:-1] + e[1:]) / 2  # midpoints between edges


_space = gym.make(ENV_ID).observation_space  # official observation bounds
# 36 x 28 grid
DISC = UniformDiscretizer(_space.low, _space.high, (CFG.n_pos_bins, CFG.n_vel_bins))
# grid size and bin widths
print(
    f"grid {DISC.bins[0]} x {DISC.bins[1]} = {DISC.bins[0] * DISC.bins[1]} states | "
    f"bin widths: position {DISC.width[0]:.4f}, velocity {DISC.width[1]:.5f}"
)
# check approximate alignment; float32 space bounds make it inexact
print(
    "flag x = 0.5 is a position edge: "
    f"{np.isclose(DISC.edges[0], GOAL_POSITION).any()} | "
    f"v = 0.03 is a velocity edge: {np.isclose(DISC.edges[1], GENTLE_SPEED).any()}"
)
e25 = np.linspace(-0.07, 0.07, 26)  # comparison: the minimum 25 x 25 grid
# show why the finer grid was chosen (25 x 25 still meets the assignment)
print(
    f"(for comparison, on a 25 x 25 grid the velocity edges around 0.03 are "
    f"{e25[e25 < 0.03][-1]:.4f} and {e25[e25 >= 0.03][0]:.4f}, so 'gentle' and 'too "
    "fast' share a bin)"
)


def state_shape(state_mode: str) -> tuple:
```

```
    """Shape of the discrete state space: (pos, vel) for the standard agent, (pos, vel,
    last action) for 'memory'."""
    # memory adds a last-action dimension
    return DISC.bins if state_mode == "obs" else DISC.bins + (3,)


def make_state_encoder(state_mode: str):
    """Return encode(obs, last_selected_action) -> state tuple for the chosen state
    representation."""
    if state_mode == "obs":  # required representation: observation only
        return lambda obs, last_action: DISC.index(obs)  # ignore the last action
    if state_mode == "memory":  # extra experiment: agent remembers its own last choice
        # append the last selected action
        return lambda obs, last_action: DISC.index(obs) + (int(last_action),)
    raise ValueError(state_mode)  # unknown representation
```

```
grid 36 x 28 = 1008 states | bin widths: position 0.0500, velocity 0.00500
flag x = 0.5 is a position edge: True | v = 0.03 is a velocity edge: True
(for comparison, on a 25 x 25 grid the velocity edges around 0.03 are 0.0252 and 0.0308, so 'gentle' and 'too
fast' share a bin)
```

2.2 Fixed validation set of states (for the average-max-Q plot)

Following the DQN paper (Mnih et al., Session 12), the validation states are collected **once, before any training**, by running policies in the *original* environment: half the episodes use a random policy (valley states), half use the pump heuristic with 20 % random actions (high-energy states near the flag). We keep 1 000 non-terminal states. The same array is used for all four agents and never changes.

In [31]:

```
def collect_validation_states(n_states: int = 1000, seed: int = SEED) -> np.ndarray:
    """Sample a fixed set of reachable states from random and heuristic roll-outs in the
    original environment."""
    env = gym.make(ENV_ID)  # original environment
    rng = np.random.default_rng([seed, 7])  # dedicated generator for this sampling
    pool = []  # every visited non-terminal state
    for ep in range(60):  # 60 roll-outs
        obs, _ = env.reset(seed=seed * 1000 + ep)  # seeded start state
        while True:  # roll out one episode
            pool.append(obs.copy())  # record the state before acting
            # random episodes / 20 % noise in heuristic ones
            if ep % 2 == 0 or rng.random() < 0.2:
                action = int(rng.integers(3))  # random action
            else:
                action = pump_policy(obs, rng)  # energy-pumping heuristic
            # advance the environment
            obs, _, terminated, truncated, _ = env.step(action)
            if terminated or truncated:  # episode finished
                break  # leave the loop
    pool = np.array(pool)  # (N, 2) array of states
    # subsample without replacement
    pick = rng.choice(len(pool), size=n_states, replace=False)
    env.close()  # release the sampling environment
    return pool[pick]  # the fixed validation states


VAL_STATES = collect_validation_states()  # sampled ONCE, before training
VAL_IDX = DISC.index_batch(VAL_STATES)  # their grid cells (pre-computed)
# saved for reproducibility
np.save(os.path.join(OUT_DIR, "validation_states.npy"), VAL_STATES)
# size and coverage of the validation set
print(
    f"validation set: {len(VAL_STATES)} states covering {len(set(zip(*VAL_IDX)))} "
    "distinct grid cells"
)
```

```
validation set: 1000 states covering 308 distinct grid cells
```

2.3 ε-greedy exploration schedule

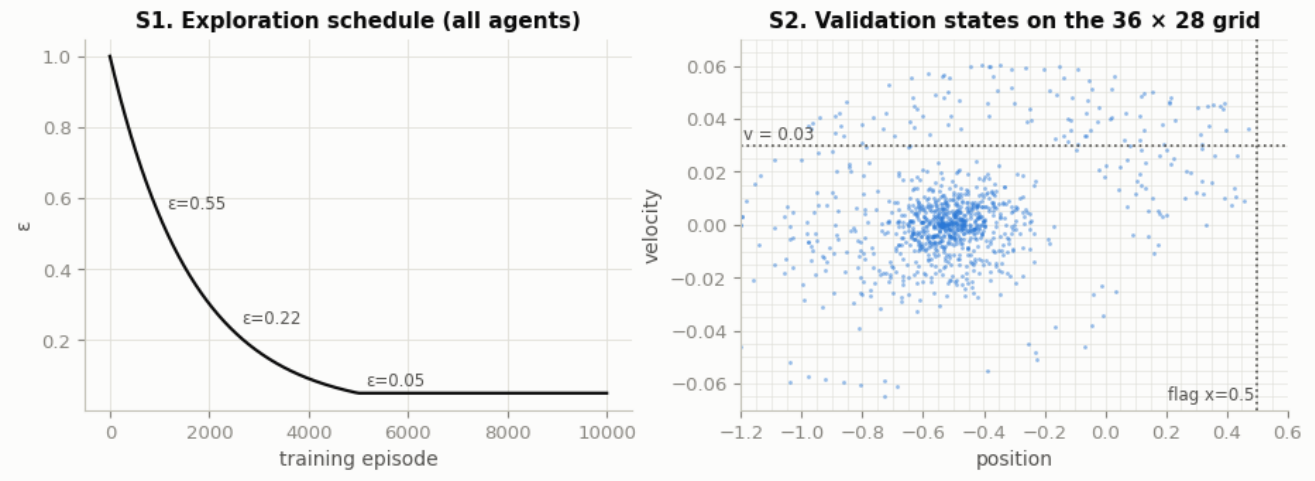
ε_k = max(0.05, 1.0 · ρ^k) with ρ = (0.05 / 1.0)^(1 / 5000). ε reaches 0.05 at episode 5 000 and stays there.

In [33]:

```
def epsilon_schedule(cfg: ExperimentConfig) -> np.ndarray:
    """Exploration rate for every training episode: exponential decay to eps_end, then
    constant."""
```

```
    decay_episodes = cfg.eps_decay_fraction * cfg.n_episodes  # episodes until the floor
    # per-episode decay factor
    rho = (cfg.eps_end / cfg.eps_start) ** (1.0 / decay_episodes)
    # clipped at the floor
    return np.maximum(cfg.eps_end, cfg.eps_start * rho ** np.arange(cfg.n_episodes))


EPS = epsilon_schedule(CFG)  # schedule shared by all agents
# two panels: schedule and validation set
fig, axes = plt.subplots(1, 2, figsize=(11, 3.4))
axes[0].plot(EPS, color=INK, lw=1.6)  # ε versus episode
# label the schedule panel
axes[0].set(
    xlabel="training episode", ylabel="ε", title="S1. Exploration schedule (all agents)"
)
# annotate reference points
for ep_mark in (CFG.n_episodes // 10, CFG.n_episodes // 4, CFG.n_episodes // 2):
    # write the epsilon value at the mark
    axes[0].annotate(
        f"ε={EPS[ep_mark]:.2f}",
        (ep_mark, EPS[ep_mark]),
        xytext=(4, 4),
        textcoords="offset points",
        fontsize=8.5,
        color=INK_2,
    )
ax = axes[1]  # right panel
# states
ax.scatter(
    VAL_STATES[:, 0],
    VAL_STATES[:, 1],
    s=4,
    color=ALGO_COLOR["q_learning"],
    alpha=0.5,
    lw=0,
)
for x in DISC.edges[0]:  # vertical bin edges
    ax.axvline(x, color="#e1e0d9", lw=0.4, zorder=0)  # position grid lines
for v in DISC.edges[1]:  # horizontal bin edges
    ax.axhline(v, color="#e1e0d9", lw=0.4, zorder=0)  # velocity grid lines
ax.axvline(GOAL_POSITION, color=INK_2, ls=":", lw=1.2)  # flag
ax.axhline(GENTLE_SPEED, color=INK_2, ls=":", lw=1.2)  # gentle threshold
# label the flag line
ax.text(
    GOAL_POSITION - 0.01, -0.066, "flag x=0.5", ha="right", fontsize=8.5, color=INK_2
)
# label the threshold line
ax.text(-1.19, GENTLE_SPEED + 0.002, "v = 0.03", fontsize=8.5, color=INK_2)
# axes limits = observation box
ax.set(
    xlim=(-1.2, 0.6),
    ylim=(-0.07, 0.07),
    xlabel="position",
    ylabel="velocity",
    axisbelow=True,
    title="S2. Validation states on the 36 × 28 grid",
)
ax.grid(False)  # bin edges replace the default grid
save_fig(fig, "S_setup")  # save and show
```



2.4 The agent — Q-learning and SARSA differ only in `td_target`

Both algorithms use one class and one training loop. The loop always picks the next action `a'` with ε-greedy **before** the update (SARSA needs it for its target; Q-learning just executes it on the next step), so the action-selection code path is shared. Random-number consumption can still diverge because tie counts and episode lengths differ. The only branch on the algorithm is the bootstrap term:

- **Q-learning (off-policy):** target = R + γ · max_a' Q(s', a')
- **SARSA (on-policy):** target = R + γ · Q(s', a') with a' the action actually chosen by ε-greedy

If the episode **terminated** (flag reached) the target is just R. **Truncation is not terminal:** at step 200 both algorithms still bootstrap from s' (SARSA uses the a' it selected at s', even though it is never executed). `a` in the update is the action *selected* by the agent, because the executed action is hidden from it.

In [35]:

```
class TabularTDAgent:
    """Tabular one-step TD control agent; algorithm='q_learning' (off-policy) or 'sarsa'
    (on-policy)."""

    def __init__(
        self,
        algorithm: str,
        state_shape: tuple,
        n_actions: int,
        alpha: float,
        gamma: float,
        rng,
    ):
        """Create a zero-initialised Q-table of shape state_shape + (n_actions,)."""
        if algorithm not in ("q_learning", "sarsa"):  # only the two required algorithms
            raise ValueError(algorithm)  # reject unknown algorithms
        self.algorithm = algorithm  # selects the TD target
        self.Q = np.zeros(tuple(state_shape) + (n_actions,))  # Q-table, all zeros
        self.n_actions = n_actions  # 3 for MountainCar
        self.alpha = alpha  # learning rate
        self.gamma = gamma  # discount factor
        self.rng = rng  # agent's own random generator

    def greedy_action(self, s: tuple, rng=None) -> int:
        """argmax_a Q(s, a), breaking ties uniformly at random (all-zero rows are common
        early on)."""
        q = self.Q[s]  # the 3 action values of state s
        best = np.flatnonzero(q == q.max())  # all maximising actions
        if len(best) == 1:  # usual case: a unique maximum
            return int(best[0])  # the unique best action
        return int((rng or self.rng).choice(best))  # random tie-break

    def select_action(self, s: tuple, epsilon: float) -> int:
        """ε-greedy: a random action with probability ε, otherwise the greedy action."""
        if self.rng.random() < epsilon:  # explore
            return int(self.rng.integers(self.n_actions))  # uniformly random action
        return self.greedy_action(s)  # exploit
```

```
    def td_target(self, r: float, s_next: tuple, a_next, terminated: bool) -> float:
        """THE ONLY ALGORITHM-SPECIFIC LINE: the bootstrap term of the one-step TD
        target."""
        if terminated:  # flag reached: no bootstrap
            return r  # terminal: target is the reward only
        if self.algorithm == "q_learning":  # off-policy: value of the greedy action
            bootstrap = self.Q[s_next].max()  # max over next actions (greedy)
        else:  # on-policy: value of the action chosen
            bootstrap = self.Q[s_next + (a_next,)]  # value of the chosen next action
        return r + self.gamma * bootstrap  # one-step TD target

    def update(
        self, s: tuple, a: int, r: float, s_next: tuple, a_next, terminated: bool
    ) -> float:
        """Q(s,a) <- Q(s,a) + α [target - Q(s,a)]; returns the TD error."""
        idx = s + (a,)  # index of the updated entry
        # target minus current estimate
        td_error = self.td_target(r, s_next, a_next, terminated) - self.Q[idx]
        self.Q[idx] += self.alpha * td_error  # move the estimate towards the target
        return td_error  # returned for logging

    def avg_max_q(self, vi: np.ndarray, vj: np.ndarray) -> float:
        """Average over the validation states of max_a Q(s, a) (memory agents use last
        action = 'no push')."""
        # (N, 3) action values
        q = self.Q[vi, vj] if self.Q.ndim == 3 else self.Q[vi, vj, NO_PUSH]
        return float(q.max(axis=1).mean())  # average over validation states
```

Unit test of the TD targets, episode endings and discretiser bounds

A tiny 2 × 2 Q-table with known values checks, for both algorithms: the non-terminal target (Q-learning uses the greedy value −2, SARSA the selected next action's value −4), the terminal target (reward only, no bootstrap), that a truncated step still bootstraps (it is a non-terminal step), that only the *selected* action is updated, and that the discretiser maps the observation-box corners to the first and last bins.

In [37]:

```
# Verify the only algorithm-specific target and correct handling of episode endings.
td_checks = []  # one summary row per algorithm
for algorithm in ("q_learning", "sarsa"):  # test both algorithms with the same table
    test_agent = TabularTDAgent(
        algorithm, (2, 2), 3, 0.1, 0.99, np.random.default_rng(109)
    )  # tiny 2 x 2 test agent
    test_agent.Q[1, 1] = [-4.0, -2.0, -3.0]  # greedy value -2, selected-next value -4
    # expected non-terminal target: greedy (-2) for Q-learning, chosen a'=0 (-4) for
    # SARSA
    expected = -1.3 + 0.99 * (-2.0 if algorithm == "q_learning" else -4.0)
    # non-terminal (also truncated) step bootstraps from s'
    assert np.isclose(test_agent.td_target(-1.3, (1, 1), 0, False), expected)
    # terminated step: the target is the reward only
    assert test_agent.td_target(23.7, (1, 1), None, True) == 23.7
    test_agent.update((0, 0), 2, -1.3, (1, 1), 0, False)  # one update of action 2
    assert np.isclose(test_agent.Q[0, 0, 2], 0.1 * expected)  # alpha * TD error
    assert np.all(test_agent.Q[0, 0, :2] == 0)  # update selected action 2 only
    td_checks.append(
        dict(
            algorithm=algorithm,
            nonterminal_target=expected,
            terminated_target=23.7,
            truncated_bootstrap=True,
            selected_action_update=True,
        )
    )  # record the passed checks
assert DISC.index(DISC.low) == (0, 0)  # lower corner -> first bins
# upper corner -> last bins
assert DISC.index(DISC.high) == (CFG.n_pos_bins - 1, CFG.n_vel_bins - 1)
# the assignment's minimum grid size and episode count
assert CFG.n_pos_bins >= 25 and CFG.n_vel_bins >= 25 and CFG.n_episodes >= 6000
print(  # verdict of the TD-target unit test
    "TD targets, truncation bootstrap, selected-action update and discretizer bounds "
    "PASS"
)
display(pd.DataFrame(td_checks))  # show the per-algorithm results
```

TD targets, truncation bootstrap, selected-action update and discretizer bounds PASS

	algorithm	nonterminal_target	terminated_target	truncated_bootstrap	selected_action_update
0	q_learning	-3.28	23.7	True	True
1	sarsa	-5.26	23.7	True	True

2.5 Training loop (shared by all agents)

In [39]:

```
@dataclass
class RunResult:
    """Everything produced by one training run."""

    label: str  # e.g. "Q-learning | sticky"
    algorithm: str  # 'q_learning' or 'sarsa'
    env_kind: str  # 'original' or 'modified'
    seed: int  # random seed of the run
    state_mode: str  # 'obs' (required) or 'memory' (extra experiment)
    agent: TabularTDAgent  # trained agent (holds the Q-table)
    visits: np.ndarray  # number of updates per discrete state
    log: pd.DataFrame  # one row per training episode
    seconds: float  # wall-clock training time


def trailing_mean(x, w: int = 100) -> np.ndarray:
    """Mean over the previous w episodes (fewer at the start); NaNs are ignored."""
    # pandas rolling mean
    return pd.Series(x).rolling(w, min_periods=1).mean().to_numpy()


def train_agent(
    algorithm: str,
    env_kind: str,
    cfg: ExperimentConfig = CFG,
    seed: int = SEED,
    state_mode: str = "obs",
    verbose: bool = True,
    log_every=None,
) -> RunResult:
    """Train one tabular agent for cfg.n_episodes episodes and return its Q-table plus
    per-episode logs."""
    # human-readable run name
    label = f"{ALGO_LABEL[algorithm]} | {ENV_LABEL[env_kind]}" + (
        " | memory" if state_mode == "memory" else ""
    )
    log_every = log_every or cfg.log_every  # progress-line frequency
    env = make_env(env_kind)  # fresh environment for this run
    encode = make_state_encoder(state_mode)  # observation -> discrete state
    agent = TabularTDAgent(
        algorithm,
        state_shape(state_mode),
        3,
        cfg.alpha,
        cfg.gamma,
        # agent generator derived from the seed
        np.random.default_rng([seed, 1]),
    )
    eps_values = epsilon_schedule(cfg)  # identical schedule for every agent
    n = cfg.n_episodes  # number of training episodes
    # logs
    rec = {
        k: np.zeros(n)
        for k in ("reward", "length", "pushes", "epsilon", "avg_max_q", "mean_abs_td")
    }
    success = np.zeros(n, dtype=bool)  # flag reached (terminated)
    gentle = np.zeros(n, dtype=bool)  # gentle-arrival criterion met
    arrival_v = np.full(n, np.nan)  # velocity when the flag was reached
    visits = np.zeros(agent.Q.shape[:-1], dtype=np.int64)  # update counts per state
    vi, vj = VAL_IDX  # fixed validation states
    t0 = time.perf_counter()  # start the stopwatch
    if verbose:  # optional start stamp + column legend
        lab_stamp(f"train {label} | seed {seed} | {n} ep")  # start stamp of this run
        # legend for the progress columns (two short lines so they never wrap)
```

```
        print("  R100 / succ / gentle / push = means over the previous 100 episodes")
        # legend for maxQ
        print("  maxQ = average of max_a Q(s,a) over the fixed validation states")
    for ep in range(n):  # training episodes
        eps = eps_values[ep]  # ε for this episode
        # seed only the first reset (Gym convention)
        obs, _ = env.reset(seed=seed if ep == 0 else None)
        s = encode(obs, NO_PUSH)  # initial discrete state
        a = agent.select_action(s, eps)  # first action
        ep_reward, steps, pushes, abs_td = 0.0, 0, 0, 0.0  # per-episode accumulators
        while True:  # steps within the episode
            # act (the env may execute a different action)
            obs, r, terminated, truncated, _ = env.step(a)
            ep_reward += r  # undiscounted episode reward
            steps += 1  # count steps
            pushes += int(a != NO_PUSH)  # pushes *selected* by the agent
            s_next = encode(obs, a)  # next discrete state
            # a' chosen BEFORE the update
            a_next = None if terminated else agent.select_action(s_next, eps)
            # TD update (only target differs)
            abs_td += abs(agent.update(s, a, r, s_next, a_next, terminated))
            visits[s] += 1  # count the update of state s
            if terminated or truncated:  # episode over
                break  # leave the step loop
            s, a = s_next, a_next  # SARSA-style hand-over for both algorithms
        # store per-episode results
        rec["reward"][ep], rec["length"][ep], rec["pushes"][ep] = (
            ep_reward,
            steps,
            pushes,
        )
        # epsilon used and mean |TD error| per step
        rec["epsilon"][ep], rec["mean_abs_td"][ep] = eps, abs_td / steps
        # value estimate on the fixed validation set
        rec["avg_max_q"][ep] = agent.avg_max_q(vi, vj)
        success[ep] = terminated  # reached the flag
        # bonus criterion, also tracked in the original env
        gentle[ep] = (
            terminated
            and not truncated
            and obs[0] >= GOAL_POSITION
            and obs[1] <= GENTLE_SPEED
        )
        # arrival speed (only if arrived)
        arrival_v[ep] = obs[1] if terminated else np.nan
        if verbose and (ep + 1) % log_every == 0:  # periodic progress line
            w = slice(max(0, ep - 99), ep + 1)  # previous 100 episodes
            # one progress line
            print(
                f"{clock()}  ep {ep + 1:>5}/{n}  eps {eps:.3f}  "
                f"R100 {rec['reward'][w].mean():7.1f}  succ {success[w].mean():.2f}  "
                f"gentle {gentle[w].mean():.2f}  push {rec['pushes'][w].mean():5.1f}  "
                f"maxQ {rec['avg_max_q'][ep]:6.1f}"
            )
    seconds = time.perf_counter() - t0  # training time
    log = pd.DataFrame(rec)  # per-episode log table
    # add the boolean columns
    log["success"], log["gentle"], log["arrival_velocity"] = success, gentle, arrival_v
    log.index.name = "episode"  # name the index
    # unique file name per run
    tag = f"{algorithm}_{env_kind}_{state_mode}_seed{seed}_ep{n}"
    log.to_csv(os.path.join(OUT_DIR, f"log_{tag}.csv"))  # full per-episode log on disk
    # Persist the trained policy, coverage and validation sample for later evaluation.
    np.savez_compressed(
        os.path.join(OUT_DIR, f"checkpoint_{tag}.npz"),
        Q=agent.Q,
        visits=visits,
        validation_states=VAL_STATES,
        metadata=json.dumps(
            dict(
                algorithm=algorithm,
                env_kind=env_kind,
                seed=seed,
                state_mode=state_mode,
                config=vars(cfg),
```

```
                gymnasium=gym.__version__,
                numpy=np.__version__,
            )
        ),
    )
    env.close()  # close only after the full training run
    if verbose:  # optional end stamp
        # end-of-run line with the log path
        print(f"{clock()}  done in {seconds:.1f} s -> {OUT_DIR}/log_{tag}.csv")
    # everything the analysis needs
    return RunResult(
        label, algorithm, env_kind, seed, state_mode, agent, visits, log, seconds
    )
```

3. Training

3.1 Part (b) — Q-learning on the original and the modified environment

One progress line is printed every 100 episodes: time, episode, ε, then R100 / succ / gentle / push (means over the previous 100 episodes) and maxQ (average max_a Q on the fixed validation states). The complete per-episode log of every run is written to `outputs/log_*.csv`.

In [41]:

```
RUNS = {}  # (algorithm, env_kind) -> RunResult
for env_kind in ("original", "modified"):  # same seed, config and episode count
    # Q-learning run
    RUNS[("q_learning", env_kind)] = train_agent("q_learning", env_kind)
```

```
[lab-stamp] 2026-11-09 17:16:20 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | train
Q-learning | original | seed 109 | 10000 ep
  R100 / succ / gentle / push = means over the previous 100 episodes
  maxQ = average of max_a Q(s,a) over the fixed validation states
17:16:21  ep   100/10000  eps 0.942  R100  -200.0  succ 0.00  gentle 0.00  push 133.0  maxQ   -5.7
17:16:23  ep   200/10000  eps 0.888  R100  -200.0  succ 0.00  gentle 0.00  push 135.5  maxQ   -8.0
17:16:24  ep   300/10000  eps 0.836  R100  -200.0  succ 0.00  gentle 0.00  push 137.3  maxQ   -9.8
17:16:25  ep   400/10000  eps 0.787  R100  -200.0  succ 0.00  gentle 0.00  push 140.3  maxQ  -11.1
17:16:26  ep   500/10000  eps 0.742  R100  -200.0  succ 0.00  gentle 0.00  push 140.3  maxQ  -12.7
17:16:27  ep   600/10000  eps 0.698  R100  -200.0  succ 0.00  gentle 0.00  push 140.9  maxQ  -13.7
17:16:28  ep   700/10000  eps 0.658  R100  -200.0  succ 0.00  gentle 0.00  push 142.1  maxQ  -14.9
17:16:29  ep   800/10000  eps 0.620  R100  -200.0  succ 0.00  gentle 0.00  push 142.3  maxQ  -16.3
17:16:31  ep   900/10000  eps 0.584  R100  -200.0  succ 0.00  gentle 0.00  push 143.5  maxQ  -17.0
17:16:32  ep  1000/10000  eps 0.550  R100  -200.0  succ 0.00  gentle 0.00  push 143.4  maxQ  -17.9
17:16:33  ep  1100/10000  eps 0.518  R100  -200.0  succ 0.00  gentle 0.00  push 141.8  maxQ  -19.1
17:16:34  ep  1200/10000  eps 0.488  R100  -200.0  succ 0.00  gentle 0.00  push 143.7  maxQ  -20.0
17:16:36  ep  1300/10000  eps 0.459  R100  -200.0  succ 0.00  gentle 0.00  push 143.5  maxQ  -21.0
17:16:37  ep  1400/10000  eps 0.432  R100  -200.0  succ 0.00  gentle 0.00  push 145.1  maxQ  -21.5
17:16:38  ep  1500/10000  eps 0.407  R100  -200.0  succ 0.00  gentle 0.00  push 144.1  maxQ  -22.2
17:16:39  ep  1600/10000  eps 0.384  R100  -200.0  succ 0.00  gentle 0.00  push 144.7  maxQ  -22.9
17:16:41  ep  1700/10000  eps 0.361  R100  -199.8  succ 0.02  gentle 0.02  push 145.8  maxQ  -23.6
17:16:42  ep  1800/10000  eps 0.340  R100  -200.0  succ 0.00  gentle 0.00  push 146.2  maxQ  -24.3
17:16:43  ep  1900/10000  eps 0.321  R100  -200.0  succ 0.00  gentle 0.00  push 145.9  maxQ  -25.0
17:16:44  ep  2000/10000  eps 0.302  R100  -199.6  succ 0.04  gentle 0.04  push 146.3  maxQ  -25.4
17:16:46  ep  2100/10000  eps 0.284  R100  -198.6  succ 0.07  gentle 0.06  push 146.0  maxQ  -26.2
17:16:47  ep  2200/10000  eps 0.268  R100  -196.8  succ 0.15  gentle 0.15  push 145.2  maxQ  -26.9
17:16:48  ep  2300/10000  eps 0.252  R100  -197.3  succ 0.11  gentle 0.11  push 144.0  maxQ  -27.4
17:16:49  ep  2400/10000  eps 0.238  R100  -199.6  succ 0.02  gentle 0.02  push 146.5  maxQ  -28.1
17:16:51  ep  2500/10000  eps 0.224  R100  -197.4  succ 0.08  gentle 0.08  push 146.1  maxQ  -28.5
17:16:52  ep  2600/10000  eps 0.211  R100  -198.2  succ 0.10  gentle 0.09  push 148.2  maxQ  -29.1
17:16:53  ep  2700/10000  eps 0.198  R100  -197.2  succ 0.10  gentle 0.09  push 147.0  maxQ  -29.6
17:16:54  ep  2800/10000  eps 0.187  R100  -198.8  succ 0.13  gentle 0.11  push 146.1  maxQ  -30.2
17:16:56  ep  2900/10000  eps 0.176  R100  -196.0  succ 0.22  gentle 0.19  push 143.4  maxQ  -30.6
17:16:57  ep  3000/10000  eps 0.166  R100  -190.8  succ 0.41  gentle 0.39  push 143.3  maxQ  -31.1
17:16:58  ep  3100/10000  eps 0.156  R100  -193.2  succ 0.24  gentle 0.21  push 144.2  maxQ  -31.5
17:17:00  ep  3200/10000  eps 0.147  R100  -198.8  succ 0.12  gentle 0.11  push 148.0  maxQ  -32.1
17:17:01  ep  3300/10000  eps 0.139  R100  -193.8  succ 0.41  gentle 0.26  push 146.4  maxQ  -32.4
17:17:02  ep  3400/10000  eps 0.130  R100  -179.8  succ 0.76  gentle 0.46  push 137.3  maxQ  -32.8
17:17:03  ep  3500/10000  eps 0.123  R100  -188.8  succ 0.40  gentle 0.32  push 142.4  maxQ  -33.5
17:17:04  ep  3600/10000  eps 0.116  R100  -186.9  succ 0.54  gentle 0.37  push 141.8  maxQ  -33.8
17:17:06  ep  3700/10000  eps 0.109  R100  -192.1  succ 0.36  gentle 0.24  push 146.2  maxQ  -34.2
17:17:07  ep  3800/10000  eps 0.103  R100  -195.7  succ 0.19  gentle 0.16  push 148.2  maxQ  -34.6
17:17:08  ep  3900/10000  eps 0.097  R100  -197.4  succ 0.09  gentle 0.08  push 151.2  maxQ  -35.0
17:17:09  ep  4000/10000  eps 0.091  R100  -198.2  succ 0.06  gentle 0.06  push 147.3  maxQ  -35.5
17:17:11  ep  4100/10000  eps 0.086  R100  -194.8  succ 0.27  gentle 0.23  push 144.8  maxQ  -36.1
17:17:12  ep  4200/10000  eps 0.081  R100  -183.3  succ 0.62  gentle 0.57  push 138.5  maxQ  -36.4
17:17:13  ep  4300/10000  eps 0.076  R100  -187.9  succ 0.48  gentle 0.38  push 142.6  maxQ  -36.8
17:17:14  ep  4400/10000  eps 0.072  R100  -193.9  succ 0.24  gentle 0.14  push 145.8  maxQ  -37.2
17:17:16  ep  4500/10000  eps 0.068  R100  -196.5  succ 0.38  gentle 0.09  push 148.6  maxQ  -37.6
17:17:17  ep  4600/10000  eps 0.064  R100  -182.2  succ 0.74  gentle 0.01  push 142.2  maxQ  -37.9
17:17:18  ep  4700/10000  eps 0.060  R100  -181.1  succ 0.55  gentle 0.08  push 139.6  maxQ  -38.4
17:17:19  ep  4800/10000  eps 0.056  R100  -188.4  succ 0.38  gentle 0.22  push 143.4  maxQ  -39.0
17:17:20  ep  4900/10000  eps 0.053  R100  -181.6  succ 0.84  gentle 0.31  push 140.9  maxQ  -39.3
17:17:21  ep  5000/10000  eps 0.050  R100  -173.1  succ 0.82  gentle 0.37  push 135.6  maxQ  -39.6
17:17:22  ep  5100/10000  eps 0.050  R100  -178.0  succ 0.72  gentle 0.39  push 137.2  maxQ  -39.9
17:17:24  ep  5200/10000  eps 0.050  R100  -171.8  succ 0.86  gentle 0.61  push 134.7  maxQ  -40.3
17:17:25  ep  5300/10000  eps 0.050  R100  -170.7  succ 0.77  gentle 0.64  push 131.8  maxQ  -40.6
17:17:26  ep  5400/10000  eps 0.050  R100  -167.7  succ 0.72  gentle 0.62  push 130.9  maxQ  -41.0
17:17:27  ep  5500/10000  eps 0.050  R100  -193.5  succ 0.25  gentle 0.12  push 148.5  maxQ  -41.5
17:17:28  ep  5600/10000  eps 0.050  R100  -181.2  succ 0.52  gentle 0.40  push 142.4  maxQ  -41.8
17:17:29  ep  5700/10000  eps 0.050  R100  -176.8  succ 0.77  gentle 0.15  push 139.2  maxQ  -42.2
```

```
17:17:30  ep  5800/10000  eps 0.050  R100  -175.7  succ 0.76  gentle 0.08  push 136.0  maxQ  -42.4
17:17:32  ep  5900/10000  eps 0.050  R100  -176.4  succ 0.89  gentle 0.38  push 137.9  maxQ  -42.5
17:17:33  ep  6000/10000  eps 0.050  R100  -175.1  succ 0.97  gentle 0.06  push 140.9  maxQ  -42.7
17:17:34  ep  6100/10000  eps 0.050  R100  -161.1  succ 0.90  gentle 0.35  push 128.1  maxQ  -43.0
17:17:35  ep  6200/10000  eps 0.050  R100  -164.0  succ 0.80  gentle 0.52  push 128.9  maxQ  -43.4
17:17:36  ep  6300/10000  eps 0.050  R100  -163.6  succ 0.79  gentle 0.68  push 130.3  maxQ  -43.7
17:17:37  ep  6400/10000  eps 0.050  R100  -167.4  succ 0.97  gentle 0.26  push 133.3  maxQ  -43.8
17:17:38  ep  6500/10000  eps 0.050  R100  -180.0  succ 0.66  gentle 0.11  push 142.9  maxQ  -44.1
17:17:39  ep  6600/10000  eps 0.050  R100  -175.6  succ 0.82  gentle 0.05  push 139.4  maxQ  -44.4
17:17:40  ep  6700/10000  eps 0.050  R100  -168.6  succ 0.83  gentle 0.04  push 137.8  maxQ  -44.7
17:17:41  ep  6800/10000  eps 0.050  R100  -174.2  succ 0.86  gentle 0.02  push 140.8  maxQ  -45.1
17:17:42  ep  6900/10000  eps 0.050  R100  -174.8  succ 0.80  gentle 0.05  push 143.1  maxQ  -45.1
17:17:44  ep  7000/10000  eps 0.050  R100  -174.0  succ 0.87  gentle 0.18  push 138.4  maxQ  -45.3
17:17:45  ep  7100/10000  eps 0.050  R100  -182.2  succ 0.62  gentle 0.19  push 141.3  maxQ  -45.7
17:17:46  ep  7200/10000  eps 0.050  R100  -174.2  succ 0.86  gentle 0.37  push 137.6  maxQ  -45.9
17:17:47  ep  7300/10000  eps 0.050  R100  -170.5  succ 0.88  gentle 0.41  push 135.2  maxQ  -46.2
17:17:48  ep  7400/10000  eps 0.050  R100  -164.2  succ 0.95  gentle 0.12  push 134.1  maxQ  -46.4
17:17:49  ep  7500/10000  eps 0.050  R100  -153.4  succ 0.98  gentle 0.46  push 124.7  maxQ  -46.6
17:17:50  ep  7600/10000  eps 0.050  R100  -151.9  succ 0.96  gentle 0.48  push 122.2  maxQ  -46.8
17:17:51  ep  7700/10000  eps 0.050  R100  -143.3  succ 1.00  gentle 0.80  push 116.4  maxQ  -46.8
17:17:52  ep  7800/10000  eps 0.050  R100  -156.6  succ 0.95  gentle 0.65  push 122.8  maxQ  -47.0
17:17:53  ep  7900/10000  eps 0.050  R100  -148.8  succ 0.99  gentle 0.66  push 119.0  maxQ  -47.3
17:17:54  ep  8000/10000  eps 0.050  R100  -147.8  succ 0.99  gentle 0.49  push 119.1  maxQ  -47.5
17:17:55  ep  8100/10000  eps 0.050  R100  -150.1  succ 1.00  gentle 0.32  push 120.3  maxQ  -47.7
17:17:56  ep  8200/10000  eps 0.050  R100  -153.0  succ 0.98  gentle 0.39  push 123.5  maxQ  -47.9
17:17:57  ep  8300/10000  eps 0.050  R100  -151.7  succ 0.95  gentle 0.51  push 123.9  maxQ  -48.2
17:17:58  ep  8400/10000  eps 0.050  R100  -151.0  succ 0.93  gentle 0.38  push 124.2  maxQ  -48.2
17:17:59  ep  8500/10000  eps 0.050  R100  -157.3  succ 0.97  gentle 0.25  push 129.1  maxQ  -48.3
17:18:00  ep  8600/10000  eps 0.050  R100  -165.2  succ 0.90  gentle 0.09  push 134.6  maxQ  -48.4
17:18:01  ep  8700/10000  eps 0.050  R100  -156.1  succ 0.91  gentle 0.16  push 129.3  maxQ  -48.7
17:18:02  ep  8800/10000  eps 0.050  R100  -167.9  succ 0.95  gentle 0.02  push 140.4  maxQ  -48.8
17:18:03  ep  8900/10000  eps 0.050  R100  -171.1  succ 0.93  gentle 0.00  push 141.5  maxQ  -49.0
17:18:04  ep  9000/10000  eps 0.050  R100  -160.9  succ 0.89  gentle 0.04  push 131.6  maxQ  -49.2
17:18:05  ep  9100/10000  eps 0.050  R100  -158.2  succ 0.94  gentle 0.12  push 126.9  maxQ  -49.3
17:18:06  ep  9200/10000  eps 0.050  R100  -160.2  succ 0.88  gentle 0.45  push 128.0  maxQ  -49.5
17:18:07  ep  9300/10000  eps 0.050  R100  -162.3  succ 0.94  gentle 0.23  push 130.1  maxQ  -49.6
17:18:08  ep  9400/10000  eps 0.050  R100  -158.1  succ 0.87  gentle 0.24  push 128.8  maxQ  -49.6
17:18:09  ep  9500/10000  eps 0.050  R100  -156.2  succ 0.91  gentle 0.24  push 130.7  maxQ  -49.9
17:18:10  ep  9600/10000  eps 0.050  R100  -153.8  succ 0.94  gentle 0.23  push 127.8  maxQ  -49.9
17:18:11  ep  9700/10000  eps 0.050  R100  -158.5  succ 0.90  gentle 0.22  push 130.4  maxQ  -50.1
17:18:12  ep  9800/10000  eps 0.050  R100  -154.3  succ 0.91  gentle 0.37  push 125.4  maxQ  -50.2
17:18:13  ep  9900/10000  eps 0.050  R100  -150.1  succ 1.00  gentle 0.77  push 121.4  maxQ  -50.4
17:18:14  ep 10000/10000  eps 0.050  R100  -140.2  succ 0.99  gentle 0.76  push 118.9  maxQ  -50.5
17:18:14  done in 113.4 s -> outputs/log_q_learning_original_obs_seed109_ep10000.csv
[lab-stamp] 2026-11-09 17:18:14 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | train
Q-learning | sticky | seed 109 | 10000 ep
  R100 / succ / gentle / push = means over the previous 100 episodes
  maxQ = average of max_a Q(s,a) over the fixed validation states
17:18:15  ep   100/10000  eps 0.942  R100  -239.3  succ 0.00  gentle 0.00  push 131.0  maxQ   -7.7
17:18:16  ep   200/10000  eps 0.888  R100  -239.4  succ 0.00  gentle 0.00  push 131.3  maxQ  -11.5
17:18:18  ep   300/10000  eps 0.836  R100  -239.2  succ 0.00  gentle 0.00  push 130.8  maxQ  -14.0
17:18:19  ep   400/10000  eps 0.787  R100  -239.8  succ 0.00  gentle 0.00  push 132.6  maxQ  -15.7
17:18:20  ep   500/10000  eps 0.742  R100  -239.3  succ 0.00  gentle 0.00  push 131.1  maxQ  -17.7
17:18:22  ep   600/10000  eps 0.698  R100  -239.7  succ 0.00  gentle 0.00  push 132.4  maxQ  -19.0
17:18:23  ep   700/10000  eps 0.658  R100  -239.0  succ 0.00  gentle 0.00  push 130.0  maxQ  -20.7
17:18:25  ep   800/10000  eps 0.620  R100  -239.6  succ 0.00  gentle 0.00  push 131.9  maxQ  -22.0
17:18:26  ep   900/10000  eps 0.584  R100  -239.1  succ 0.00  gentle 0.00  push 130.4  maxQ  -23.2
17:18:27  ep  1000/10000  eps 0.550  R100  -238.9  succ 0.00  gentle 0.00  push 129.6  maxQ  -24.1
17:18:29  ep  1100/10000  eps 0.518  R100  -238.9  succ 0.00  gentle 0.00  push 129.7  maxQ  -25.3
17:18:30  ep  1200/10000  eps 0.488  R100  -239.7  succ 0.00  gentle 0.00  push 132.2  maxQ  -26.5
17:18:32  ep  1300/10000  eps 0.459  R100  -239.5  succ 0.00  gentle 0.00  push 131.7  maxQ  -27.4
17:18:33  ep  1400/10000  eps 0.432  R100  -238.8  succ 0.00  gentle 0.00  push 129.3  maxQ  -28.4
17:18:35  ep  1500/10000  eps 0.407  R100  -239.1  succ 0.00  gentle 0.00  push 130.2  maxQ  -29.3
17:18:36  ep  1600/10000  eps 0.384  R100  -238.5  succ 0.01  gentle 0.01  push 129.2  maxQ  -30.4
17:18:37  ep  1700/10000  eps 0.361  R100  -238.9  succ 0.00  gentle 0.00  push 129.5  maxQ  -31.1
17:18:39  ep  1800/10000  eps 0.340  R100  -238.4  succ 0.01  gentle 0.01  push 129.6  maxQ  -31.9
17:18:40  ep  1900/10000  eps 0.321  R100  -238.4  succ 0.01  gentle 0.01  push 129.5  maxQ  -32.5
17:18:42  ep  2000/10000  eps 0.302  R100  -238.8  succ 0.00  gentle 0.00  push 129.3  maxQ  -33.2
17:18:43  ep  2100/10000  eps 0.284  R100  -238.5  succ 0.01  gentle 0.01  push 129.3  maxQ  -34.1
17:18:45  ep  2200/10000  eps 0.268  R100  -238.0  succ 0.02  gentle 0.02  push 130.2  maxQ  -34.9
17:18:46  ep  2300/10000  eps 0.252  R100  -238.6  succ 0.00  gentle 0.00  push 128.6  maxQ  -35.3
17:18:48  ep  2400/10000  eps 0.238  R100  -238.2  succ 0.02  gentle 0.02  push 130.0  maxQ  -36.1
17:18:49  ep  2500/10000  eps 0.224  R100  -237.7  succ 0.04  gentle 0.03  push 129.5  maxQ  -36.9
17:18:51  ep  2600/10000  eps 0.211  R100  -239.1  succ 0.00  gentle 0.00  push 130.3  maxQ  -37.7
17:18:52  ep  2700/10000  eps 0.198  R100  -238.3  succ 0.01  gentle 0.01  push 129.5  maxQ  -38.2
17:18:53  ep  2800/10000  eps 0.187  R100  -235.5  succ 0.06  gentle 0.06  push 128.3  maxQ  -38.7
17:18:55  ep  2900/10000  eps 0.176  R100  -236.9  succ 0.06  gentle 0.05  push 130.8  maxQ  -39.2
17:18:56  ep  3000/10000  eps 0.166  R100  -234.6  succ 0.09  gentle 0.09  push 128.7  maxQ  -39.7
17:18:58  ep  3100/10000  eps 0.156  R100  -237.2  succ 0.03  gentle 0.03  push 128.2  maxQ  -40.2
```

```
17:18:59  ep  3200/10000  eps 0.147  R100  -238.9  succ 0.01  gentle 0.01  push 131.0  maxQ  -40.8
17:19:01  ep  3300/10000  eps 0.139  R100  -237.6  succ 0.04  gentle 0.03  push 129.4  maxQ  -41.4
17:19:03  ep  3400/10000  eps 0.130  R100  -238.7  succ 0.02  gentle 0.02  push 131.5  maxQ  -42.0
17:19:04  ep  3500/10000  eps 0.123  R100  -235.9  succ 0.07  gentle 0.06  push 128.1  maxQ  -42.6
17:19:06  ep  3600/10000  eps 0.116  R100  -232.1  succ 0.13  gentle 0.12  push 127.0  maxQ  -43.1
17:19:08  ep  3700/10000  eps 0.109  R100  -235.7  succ 0.05  gentle 0.05  push 127.9  maxQ  -43.7
17:19:09  ep  3800/10000  eps 0.103  R100  -232.6  succ 0.12  gentle 0.12  push 127.2  maxQ  -44.5
17:19:11  ep  3900/10000  eps 0.097  R100  -223.3  succ 0.34  gentle 0.29  push 125.2  maxQ  -44.8
17:19:12  ep  4000/10000  eps 0.091  R100  -226.2  succ 0.24  gentle 0.24  push 127.1  maxQ  -45.2
17:19:14  ep  4100/10000  eps 0.086  R100  -217.8  succ 0.41  gentle 0.39  push 124.0  maxQ  -45.7
17:19:16  ep  4200/10000  eps 0.081  R100  -228.6  succ 0.23  gentle 0.22  push 129.6  maxQ  -46.2
17:19:17  ep  4300/10000  eps 0.076  R100  -236.1  succ 0.06  gentle 0.05  push 127.3  maxQ  -46.8
17:19:19  ep  4400/10000  eps 0.072  R100  -226.8  succ 0.32  gentle 0.28  push 128.8  maxQ  -47.3
17:19:21  ep  4500/10000  eps 0.068  R100  -218.0  succ 0.40  gentle 0.39  push 125.7  maxQ  -47.4
17:19:22  ep  4600/10000  eps 0.064  R100  -218.4  succ 0.36  gentle 0.35  push 125.0  maxQ  -47.9
17:19:24  ep  4700/10000  eps 0.060  R100  -232.0  succ 0.14  gentle 0.13  push 126.0  maxQ  -48.4
17:19:25  ep  4800/10000  eps 0.056  R100  -227.3  succ 0.22  gentle 0.20  push 124.9  maxQ  -48.9
17:19:27  ep  4900/10000  eps 0.053  R100  -215.0  succ 0.46  gentle 0.45  push 124.2  maxQ  -49.3
17:19:29  ep  5000/10000  eps 0.050  R100  -220.9  succ 0.31  gentle 0.29  push 123.7  maxQ  -49.6
17:19:30  ep  5100/10000  eps 0.050  R100  -213.3  succ 0.52  gentle 0.48  push 122.0  maxQ  -49.8
17:19:32  ep  5200/10000  eps 0.050  R100  -219.3  succ 0.33  gentle 0.30  push 122.3  maxQ  -50.3
17:19:33  ep  5300/10000  eps 0.050  R100  -215.7  succ 0.44  gentle 0.41  push 122.4  maxQ  -50.7
17:19:35  ep  5400/10000  eps 0.050  R100  -199.7  succ 0.69  gentle 0.68  push 119.2  maxQ  -51.2
17:19:36  ep  5500/10000  eps 0.050  R100  -189.1  succ 0.76  gentle 0.75  push 114.5  maxQ  -51.4
17:19:38  ep  5600/10000  eps 0.050  R100  -190.9  succ 0.66  gentle 0.66  push 115.8  maxQ  -51.6
17:19:39  ep  5700/10000  eps 0.050  R100  -206.7  succ 0.49  gentle 0.48  push 119.0  maxQ  -52.1
17:19:40  ep  5800/10000  eps 0.050  R100  -206.0  succ 0.67  gentle 0.62  push 124.3  maxQ  -52.4
17:19:42  ep  5900/10000  eps 0.050  R100  -192.6  succ 0.80  gentle 0.74  push 118.4  maxQ  -52.6
17:19:43  ep  6000/10000  eps 0.050  R100  -189.7  succ 0.73  gentle 0.70  push 117.1  maxQ  -52.9
17:19:45  ep  6100/10000  eps 0.050  R100  -204.7  succ 0.56  gentle 0.55  push 119.0  maxQ  -53.1
17:19:47  ep  6200/10000  eps 0.050  R100  -207.2  succ 0.52  gentle 0.50  push 121.0  maxQ  -53.4
17:19:48  ep  6300/10000  eps 0.050  R100  -200.6  succ 0.65  gentle 0.64  push 118.3  maxQ  -53.9
17:19:50  ep  6400/10000  eps 0.050  R100  -195.0  succ 0.74  gentle 0.72  push 116.1  maxQ  -54.1
17:19:51  ep  6500/10000  eps 0.050  R100  -200.8  succ 0.69  gentle 0.68  push 117.7  maxQ  -54.4
17:19:53  ep  6600/10000  eps 0.050  R100  -208.4  succ 0.46  gentle 0.45  push 116.9  maxQ  -54.9
17:19:54  ep  6700/10000  eps 0.050  R100  -200.3  succ 0.68  gentle 0.67  push 115.1  maxQ  -55.2
17:19:56  ep  6800/10000  eps 0.050  R100  -190.0  succ 0.80  gentle 0.77  push 111.7  maxQ  -55.2
17:19:57  ep  6900/10000  eps 0.050  R100  -186.7  succ 0.80  gentle 0.79  push 109.6  maxQ  -55.2
17:19:59  ep  7000/10000  eps 0.050  R100  -193.5  succ 0.72  gentle 0.71  push 111.1  maxQ  -55.4
17:20:00  ep  7100/10000  eps 0.050  R100  -200.2  succ 0.57  gentle 0.57  push 115.2  maxQ  -55.8
17:20:02  ep  7200/10000  eps 0.050  R100  -201.9  succ 0.54  gentle 0.54  push 111.7  maxQ  -55.9
17:20:03  ep  7300/10000  eps 0.050  R100  -190.5  succ 0.67  gentle 0.67  push 109.7  maxQ  -55.9
17:20:05  ep  7400/10000  eps 0.050  R100  -202.0  succ 0.58  gentle 0.58  push 115.3  maxQ  -56.5
17:20:07  ep  7500/10000  eps 0.050  R100  -212.6  succ 0.43  gentle 0.42  push 114.6  maxQ  -56.8
17:20:08  ep  7600/10000  eps 0.050  R100  -213.2  succ 0.42  gentle 0.40  push 112.0  maxQ  -57.2
17:20:10  ep  7700/10000  eps 0.050  R100  -212.3  succ 0.45  gentle 0.44  push 114.1  maxQ  -57.2
17:20:11  ep  7800/10000  eps 0.050  R100  -206.4  succ 0.60  gentle 0.58  push 115.6  maxQ  -57.3
17:20:13  ep  7900/10000  eps 0.050  R100  -214.3  succ 0.45  gentle 0.44  push 117.6  maxQ  -57.5
17:20:15  ep  8000/10000  eps 0.050  R100  -201.5  succ 0.61  gentle 0.61  push 114.3  maxQ  -57.7
17:20:16  ep  8100/10000  eps 0.050  R100  -190.2  succ 0.75  gentle 0.75  push 115.1  maxQ  -57.9
17:20:18  ep  8200/10000  eps 0.050  R100  -198.4  succ 0.59  gentle 0.59  push 116.1  maxQ  -58.2
17:20:19  ep  8300/10000  eps 0.050  R100  -215.0  succ 0.34  gentle 0.34  push 121.4  maxQ  -58.6
17:20:21  ep  8400/10000  eps 0.050  R100  -230.0  succ 0.15  gentle 0.11  push 118.2  maxQ  -59.0
17:20:23  ep  8500/10000  eps 0.050  R100  -215.7  succ 0.43  gentle 0.40  push 116.2  maxQ  -59.2
17:20:24  ep  8600/10000  eps 0.050  R100  -204.8  succ 0.57  gentle 0.54  push 114.6  maxQ  -59.3
17:20:26  ep  8700/10000  eps 0.050  R100  -193.4  succ 0.80  gentle 0.77  push 113.8  maxQ  -59.3
17:20:27  ep  8800/10000  eps 0.050  R100  -186.7  succ 0.82  gentle 0.77  push 110.4  maxQ  -59.4
17:20:29  ep  8900/10000  eps 0.050  R100  -194.5  succ 0.64  gentle 0.62  push 107.0  maxQ  -59.8
17:20:30  ep  9000/10000  eps 0.050  R100  -202.1  succ 0.54  gentle 0.53  push 110.4  maxQ  -60.0
17:20:31  ep  9100/10000  eps 0.050  R100  -190.2  succ 0.71  gentle 0.71  push 110.4  maxQ  -60.2
17:20:33  ep  9200/10000  eps 0.050  R100  -192.3  succ 0.73  gentle 0.70  push 111.7  maxQ  -60.6
17:20:34  ep  9300/10000  eps 0.050  R100  -194.7  succ 0.78  gentle 0.75  push 115.0  maxQ  -60.7
17:20:36  ep  9400/10000  eps 0.050  R100  -191.5  succ 0.77  gentle 0.75  push 110.0  maxQ  -60.9
17:20:37  ep  9500/10000  eps 0.050  R100  -171.6  succ 0.84  gentle 0.83  push 104.0  maxQ  -60.7
17:20:38  ep  9600/10000  eps 0.050  R100  -167.6  succ 0.91  gentle 0.90  push 101.7  maxQ  -60.6
17:20:40  ep  9700/10000  eps 0.050  R100  -173.8  succ 0.86  gentle 0.85  push 103.7  maxQ  -60.5
17:20:41  ep  9800/10000  eps 0.050  R100  -174.6  succ 0.88  gentle 0.88  push 104.0  maxQ  -60.7
17:20:42  ep  9900/10000  eps 0.050  R100  -180.8  succ 0.87  gentle 0.85  push 106.9  maxQ  -60.9
17:20:44  ep 10000/10000  eps 0.050  R100  -177.6  succ 0.91  gentle 0.90  push 102.0  maxQ  -60.9
17:20:44  done in 150.0 s -> outputs/log_q_learning_modified_obs_seed109_ep10000.csv
```

3.2 Part (c) — SARSA on the original and the modified environment

In [43]:

```
for env_kind in ("original", "modified"):  # identical conditions to part (b)
    RUNS[("sarsa", env_kind)] = train_agent("sarsa", env_kind)  # SARSA run
```

```
[lab-stamp] 2026-11-09 17:20:44 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | train
SARSA | original | seed 109 | 10000 ep
```

```
  R100 / succ / gentle / push = means over the previous 100 episodes
  maxQ = average of max_a Q(s,a) over the fixed validation states
17:20:45  ep   100/10000  eps 0.942  R100  -200.0  succ 0.00  gentle 0.00  push 132.9  maxQ   -9.1
17:20:46  ep   200/10000  eps 0.888  R100  -200.0  succ 0.00  gentle 0.00  push 134.6  maxQ  -13.9
17:20:47  ep   300/10000  eps 0.836  R100  -200.0  succ 0.00  gentle 0.00  push 136.9  maxQ  -17.0
17:20:48  ep   400/10000  eps 0.787  R100  -200.0  succ 0.00  gentle 0.00  push 140.0  maxQ  -18.8
17:20:49  ep   500/10000  eps 0.742  R100  -200.0  succ 0.00  gentle 0.00  push 139.8  maxQ  -20.7
17:20:50  ep   600/10000  eps 0.698  R100  -200.0  succ 0.00  gentle 0.00  push 141.2  maxQ  -22.2
17:20:51  ep   700/10000  eps 0.658  R100  -200.0  succ 0.00  gentle 0.00  push 143.0  maxQ  -23.2
17:20:52  ep   800/10000  eps 0.620  R100  -200.0  succ 0.00  gentle 0.00  push 142.5  maxQ  -24.4
17:20:53  ep   900/10000  eps 0.584  R100  -200.0  succ 0.00  gentle 0.00  push 144.3  maxQ  -25.0
17:20:54  ep  1000/10000  eps 0.550  R100  -200.0  succ 0.00  gentle 0.00  push 142.4  maxQ  -25.8
17:20:55  ep  1100/10000  eps 0.518  R100  -200.0  succ 0.00  gentle 0.00  push 143.9  maxQ  -26.4
17:20:56  ep  1200/10000  eps 0.488  R100  -200.0  succ 0.00  gentle 0.00  push 145.4  maxQ  -27.1
17:20:57  ep  1300/10000  eps 0.459  R100  -200.0  succ 0.00  gentle 0.00  push 143.9  maxQ  -27.7
17:20:58  ep  1400/10000  eps 0.432  R100  -200.0  succ 0.00  gentle 0.00  push 145.6  maxQ  -28.0
17:20:59  ep  1500/10000  eps 0.407  R100  -200.0  succ 0.00  gentle 0.00  push 146.0  maxQ  -28.5
17:21:00  ep  1600/10000  eps 0.384  R100  -199.6  succ 0.01  gentle 0.01  push 145.4  maxQ  -29.0
17:21:02  ep  1700/10000  eps 0.361  R100  -199.3  succ 0.04  gentle 0.04  push 145.8  maxQ  -29.4
17:21:03  ep  1800/10000  eps 0.340  R100  -199.7  succ 0.02  gentle 0.02  push 145.9  maxQ  -29.9
17:21:04  ep  1900/10000  eps 0.321  R100  -199.7  succ 0.02  gentle 0.02  push 146.6  maxQ  -30.1
17:21:05  ep  2000/10000  eps 0.302  R100  -198.1  succ 0.07  gentle 0.07  push 146.6  maxQ  -30.4
17:21:06  ep  2100/10000  eps 0.284  R100  -196.7  succ 0.12  gentle 0.12  push 146.5  maxQ  -30.9
17:21:07  ep  2200/10000  eps 0.268  R100  -194.6  succ 0.24  gentle 0.23  push 145.5  maxQ  -31.3
17:21:08  ep  2300/10000  eps 0.252  R100  -197.6  succ 0.10  gentle 0.10  push 144.2  maxQ  -31.5
17:21:09  ep  2400/10000  eps 0.238  R100  -197.4  succ 0.15  gentle 0.13  push 147.6  maxQ  -31.9
17:21:10  ep  2500/10000  eps 0.224  R100  -192.0  succ 0.30  gentle 0.28  push 143.3  maxQ  -32.3
17:21:11  ep  2600/10000  eps 0.211  R100  -196.8  succ 0.11  gentle 0.11  push 145.8  maxQ  -32.7
17:21:13  ep  2700/10000  eps 0.198  R100  -197.2  succ 0.11  gentle 0.10  push 146.8  maxQ  -33.1
17:21:14  ep  2800/10000  eps 0.187  R100  -196.4  succ 0.20  gentle 0.19  push 147.5  maxQ  -33.5
17:21:15  ep  2900/10000  eps 0.176  R100  -192.2  succ 0.38  gentle 0.36  push 145.9  maxQ  -33.7
17:21:16  ep  3000/10000  eps 0.166  R100  -192.9  succ 0.35  gentle 0.25  push 144.6  maxQ  -34.2
17:21:17  ep  3100/10000  eps 0.156  R100  -192.4  succ 0.26  gentle 0.22  push 144.1  maxQ  -34.6
17:21:18  ep  3200/10000  eps 0.147  R100  -187.8  succ 0.43  gentle 0.29  push 143.9  maxQ  -35.0
17:21:19  ep  3300/10000  eps 0.139  R100  -197.6  succ 0.11  gentle 0.06  push 150.2  maxQ  -35.3
17:21:20  ep  3400/10000  eps 0.130  R100  -197.1  succ 0.24  gentle 0.08  push 150.1  maxQ  -35.7
17:21:21  ep  3500/10000  eps 0.123  R100  -191.8  succ 0.41  gentle 0.23  push 143.0  maxQ  -36.0
17:21:23  ep  3600/10000  eps 0.116  R100  -185.0  succ 0.53  gentle 0.42  push 140.1  maxQ  -36.4
17:21:24  ep  3700/10000  eps 0.109  R100  -187.5  succ 0.42  gentle 0.38  push 142.7  maxQ  -37.0
17:21:25  ep  3800/10000  eps 0.103  R100  -178.9  succ 0.83  gentle 0.76  push 138.1  maxQ  -37.2
17:21:26  ep  3900/10000  eps 0.097  R100  -175.8  succ 0.88  gentle 0.51  push 136.4  maxQ  -37.5
17:21:27  ep  4000/10000  eps 0.091  R100  -191.5  succ 0.29  gentle 0.17  push 144.8  maxQ  -37.9
17:21:28  ep  4100/10000  eps 0.086  R100  -199.0  succ 0.05  gentle 0.00  push 152.7  maxQ  -38.2
17:21:29  ep  4200/10000  eps 0.081  R100  -194.6  succ 0.22  gentle 0.03  push 149.0  maxQ  -38.7
17:21:30  ep  4300/10000  eps 0.076  R100  -181.8  succ 0.76  gentle 0.23  push 142.2  maxQ  -39.1
17:21:31  ep  4400/10000  eps 0.072  R100  -183.5  succ 0.86  gentle 0.54  push 141.8  maxQ  -39.3
17:21:32  ep  4500/10000  eps 0.068  R100  -180.5  succ 0.70  gentle 0.44  push 141.5  maxQ  -39.6
17:21:33  ep  4600/10000  eps 0.064  R100  -180.9  succ 0.52  gentle 0.48  push 137.8  maxQ  -39.9
17:21:34  ep  4700/10000  eps 0.060  R100  -170.5  succ 0.64  gentle 0.59  push 133.7  maxQ  -40.3
17:21:35  ep  4800/10000  eps 0.056  R100  -187.5  succ 0.51  gentle 0.22  push 146.7  maxQ  -40.7
17:21:37  ep  4900/10000  eps 0.053  R100  -180.6  succ 0.78  gentle 0.71  push 140.2  maxQ  -40.9
17:21:38  ep  5000/10000  eps 0.050  R100  -169.3  succ 0.84  gentle 0.49  push 133.2  maxQ  -41.2
17:21:38  ep  5100/10000  eps 0.050  R100  -151.4  succ 0.97  gentle 0.76  push 122.8  maxQ  -41.5
17:21:39  ep  5200/10000  eps 0.050  R100  -176.5  succ 0.60  gentle 0.55  push 137.4  maxQ  -41.8
17:21:41  ep  5300/10000  eps 0.050  R100  -191.2  succ 0.38  gentle 0.16  push 145.5  maxQ  -42.1
17:21:42  ep  5400/10000  eps 0.050  R100  -174.1  succ 0.93  gentle 0.15  push 136.2  maxQ  -42.4
17:21:43  ep  5500/10000  eps 0.050  R100  -174.9  succ 0.85  gentle 0.14  push 137.7  maxQ  -42.6
17:21:44  ep  5600/10000  eps 0.050  R100  -174.4  succ 0.83  gentle 0.07  push 137.5  maxQ  -42.9
17:21:45  ep  5700/10000  eps 0.050  R100  -173.3  succ 0.86  gentle 0.05  push 139.8  maxQ  -43.1
17:21:46  ep  5800/10000  eps 0.050  R100  -175.6  succ 0.84  gentle 0.15  push 138.4  maxQ  -43.4
17:21:47  ep  5900/10000  eps 0.050  R100  -181.2  succ 0.44  gentle 0.28  push 144.1  maxQ  -43.7
17:21:48  ep  6000/10000  eps 0.050  R100  -181.1  succ 0.65  gentle 0.15  push 143.7  maxQ  -44.2
17:21:49  ep  6100/10000  eps 0.050  R100  -178.8  succ 0.91  gentle 0.24  push 140.8  maxQ  -44.6
17:21:50  ep  6200/10000  eps 0.050  R100  -171.1  succ 0.92  gentle 0.25  push 134.8  maxQ  -44.8
17:21:51  ep  6300/10000  eps 0.050  R100  -169.1  succ 0.78  gentle 0.36  push 132.7  maxQ  -45.0
17:21:52  ep  6400/10000  eps 0.050  R100  -155.5  succ 0.94  gentle 0.75  push 121.7  maxQ  -45.2
17:21:53  ep  6500/10000  eps 0.050  R100  -157.7  succ 0.88  gentle 0.84  push 127.5  maxQ  -45.5
17:21:54  ep  6600/10000  eps 0.050  R100  -174.9  succ 0.63  gentle 0.31  push 137.6  maxQ  -45.9
17:21:55  ep  6700/10000  eps 0.050  R100  -171.9  succ 0.80  gentle 0.07  push 139.8  maxQ  -46.3
17:21:56  ep  6800/10000  eps 0.050  R100  -167.0  succ 0.91  gentle 0.00  push 136.7  maxQ  -46.6
17:21:57  ep  6900/10000  eps 0.050  R100  -169.4  succ 0.97  gentle 0.00  push 136.5  maxQ  -46.6
17:21:58  ep  7000/10000  eps 0.050  R100  -168.4  succ 1.00  gentle 0.06  push 134.4  maxQ  -46.7
17:21:59  ep  7100/10000  eps 0.050  R100  -166.3  succ 0.92  gentle 0.16  push 133.1  maxQ  -47.0
17:22:00  ep  7200/10000  eps 0.050  R100  -168.0  succ 0.83  gentle 0.25  push 133.7  maxQ  -47.2
17:22:01  ep  7300/10000  eps 0.050  R100  -166.4  succ 0.72  gentle 0.47  push 131.1  maxQ  -47.3
17:22:02  ep  7400/10000  eps 0.050  R100  -168.3  succ 0.75  gentle 0.57  push 132.3  maxQ  -47.6
17:22:03  ep  7500/10000  eps 0.050  R100  -156.9  succ 0.97  gentle 0.65  push 126.8  maxQ  -47.8
17:22:04  ep  7600/10000  eps 0.050  R100  -164.4  succ 0.91  gentle 0.18  push 134.9  maxQ  -48.0
17:22:05  ep  7700/10000  eps 0.050  R100  -173.6  succ 0.90  gentle 0.09  push 143.7  maxQ  -48.2
```

```
17:22:06  ep  7800/10000  eps 0.050  R100  -162.0  succ 0.93  gentle 0.09  push 134.0  maxQ  -48.3
17:22:07  ep  7900/10000  eps 0.050  R100  -151.1  succ 0.92  gentle 0.26  push 124.0  maxQ  -48.5
17:22:08  ep  8000/10000  eps 0.050  R100  -150.7  succ 0.95  gentle 0.43  push 124.6  maxQ  -48.8
17:22:08  ep  8100/10000  eps 0.050  R100  -155.8  succ 0.91  gentle 0.21  push 126.8  maxQ  -48.9
17:22:09  ep  8200/10000  eps 0.050  R100  -163.9  succ 0.96  gentle 0.02  push 129.2  maxQ  -49.1
17:22:10  ep  8300/10000  eps 0.050  R100  -162.5  succ 0.95  gentle 0.12  push 130.8  maxQ  -49.4
17:22:11  ep  8400/10000  eps 0.050  R100  -170.2  succ 0.97  gentle 0.00  push 137.2  maxQ  -49.6
17:22:12  ep  8500/10000  eps 0.050  R100  -167.7  succ 0.96  gentle 0.02  push 137.2  maxQ  -49.9
17:22:13  ep  8600/10000  eps 0.050  R100  -163.1  succ 0.92  gentle 0.00  push 131.9  maxQ  -50.0
17:22:14  ep  8700/10000  eps 0.050  R100  -158.5  succ 0.92  gentle 0.18  push 125.2  maxQ  -50.2
17:22:15  ep  8800/10000  eps 0.050  R100  -148.8  succ 0.98  gentle 0.51  push 117.9  maxQ  -50.3
17:22:16  ep  8900/10000  eps 0.050  R100  -158.7  succ 0.87  gentle 0.36  push 124.7  maxQ  -50.5
17:22:17  ep  9000/10000  eps 0.050  R100  -151.5  succ 0.99  gentle 0.41  push 122.2  maxQ  -50.6
17:22:18  ep  9100/10000  eps 0.050  R100  -160.1  succ 0.85  gentle 0.13  push 129.0  maxQ  -51.0
17:22:19  ep  9200/10000  eps 0.050  R100  -167.8  succ 0.82  gentle 0.05  push 135.1  maxQ  -51.3
17:22:20  ep  9300/10000  eps 0.050  R100  -173.6  succ 0.69  gentle 0.01  push 140.7  maxQ  -51.4
17:22:21  ep  9400/10000  eps 0.050  R100  -163.7  succ 0.86  gentle 0.00  push 133.9  maxQ  -51.5
17:22:22  ep  9500/10000  eps 0.050  R100  -156.3  succ 0.97  gentle 0.04  push 129.3  maxQ  -51.7
17:22:23  ep  9600/10000  eps 0.050  R100  -153.3  succ 0.99  gentle 0.04  push 127.9  maxQ  -51.7
17:22:24  ep  9700/10000  eps 0.050  R100  -154.7  succ 0.97  gentle 0.13  push 128.0  maxQ  -51.9
17:22:24  ep  9800/10000  eps 0.050  R100  -162.1  succ 0.87  gentle 0.20  push 131.7  maxQ  -52.2
17:22:25  ep  9900/10000  eps 0.050  R100  -165.5  succ 0.91  gentle 0.05  push 138.3  maxQ  -52.3
17:22:26  ep 10000/10000  eps 0.050  R100  -169.3  succ 0.97  gentle 0.00  push 140.5  maxQ  -52.4
17:22:27  done in 102.6 s -> outputs/log_sarsa_original_obs_seed109_ep10000.csv
[lab-stamp] 2026-11-09 17:22:27 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | train
SARSA | sticky | seed 109 | 10000 ep
  R100 / succ / gentle / push = means over the previous 100 episodes
  maxQ = average of max_a Q(s,a) over the fixed validation states
17:22:28  ep   100/10000  eps 0.942  R100  -239.4  succ 0.00  gentle 0.00  push 131.3  maxQ   -9.7
17:22:29  ep   200/10000  eps 0.888  R100  -239.4  succ 0.00  gentle 0.00  push 131.4  maxQ  -16.2
17:22:30  ep   300/10000  eps 0.836  R100  -239.8  succ 0.00  gentle 0.00  push 132.8  maxQ  -20.1
17:22:31  ep   400/10000  eps 0.787  R100  -240.1  succ 0.00  gentle 0.00  push 133.5  maxQ  -23.3
17:22:32  ep   500/10000  eps 0.742  R100  -240.0  succ 0.00  gentle 0.00  push 133.4  maxQ  -25.4
17:22:34  ep   600/10000  eps 0.698  R100  -239.8  succ 0.00  gentle 0.00  push 132.8  maxQ  -27.5
17:22:35  ep   700/10000  eps 0.658  R100  -239.9  succ 0.00  gentle 0.00  push 133.0  maxQ  -29.2
17:22:36  ep   800/10000  eps 0.620  R100  -239.9  succ 0.00  gentle 0.00  push 133.1  maxQ  -30.6
17:22:37  ep   900/10000  eps 0.584  R100  -240.0  succ 0.00  gentle 0.00  push 133.5  maxQ  -31.9
17:22:39  ep  1000/10000  eps 0.550  R100  -239.8  succ 0.00  gentle 0.00  push 132.5  maxQ  -33.0
17:22:40  ep  1100/10000  eps 0.518  R100  -239.8  succ 0.00  gentle 0.00  push 132.6  maxQ  -33.7
17:22:41  ep  1200/10000  eps 0.488  R100  -240.1  succ 0.00  gentle 0.00  push 133.8  maxQ  -34.6
17:22:42  ep  1300/10000  eps 0.459  R100  -239.5  succ 0.00  gentle 0.00  push 131.7  maxQ  -35.4
17:22:44  ep  1400/10000  eps 0.432  R100  -239.0  succ 0.00  gentle 0.00  push 130.1  maxQ  -36.2
17:22:45  ep  1500/10000  eps 0.407  R100  -239.5  succ 0.00  gentle 0.00  push 131.8  maxQ  -37.2
17:22:46  ep  1600/10000  eps 0.384  R100  -239.4  succ 0.00  gentle 0.00  push 131.3  maxQ  -37.9
17:22:48  ep  1700/10000  eps 0.361  R100  -240.0  succ 0.00  gentle 0.00  push 133.4  maxQ  -38.2
17:22:49  ep  1800/10000  eps 0.340  R100  -239.3  succ 0.01  gentle 0.01  push 132.8  maxQ  -38.7
17:22:50  ep  1900/10000  eps 0.321  R100  -239.5  succ 0.03  gentle 0.02  push 134.2  maxQ  -39.1
17:22:52  ep  2000/10000  eps 0.302  R100  -239.7  succ 0.00  gentle 0.00  push 132.4  maxQ  -39.7
17:22:53  ep  2100/10000  eps 0.284  R100  -239.2  succ 0.01  gentle 0.01  push 132.0  maxQ  -40.3
17:22:54  ep  2200/10000  eps 0.268  R100  -238.3  succ 0.02  gentle 0.02  push 130.8  maxQ  -40.6
17:22:56  ep  2300/10000  eps 0.252  R100  -238.0  succ 0.03  gentle 0.03  push 131.1  maxQ  -41.0
17:22:57  ep  2400/10000  eps 0.238  R100  -238.6  succ 0.02  gentle 0.02  push 132.3  maxQ  -41.5
17:22:58  ep  2500/10000  eps 0.224  R100  -237.5  succ 0.05  gentle 0.05  push 134.3  maxQ  -41.9
17:23:00  ep  2600/10000  eps 0.211  R100  -238.4  succ 0.04  gentle 0.04  push 133.2  maxQ  -42.3
17:23:01  ep  2700/10000  eps 0.198  R100  -237.2  succ 0.04  gentle 0.04  push 130.6  maxQ  -43.0
17:23:02  ep  2800/10000  eps 0.187  R100  -235.4  succ 0.07  gentle 0.07  push 129.9  maxQ  -43.3
17:23:04  ep  2900/10000  eps 0.176  R100  -235.6  succ 0.08  gentle 0.07  push 130.1  maxQ  -43.7
17:23:05  ep  3000/10000  eps 0.166  R100  -235.3  succ 0.08  gentle 0.08  push 129.8  maxQ  -44.2
17:23:07  ep  3100/10000  eps 0.156  R100  -235.3  succ 0.07  gentle 0.07  push 131.0  maxQ  -44.7
17:23:08  ep  3200/10000  eps 0.147  R100  -233.9  succ 0.11  gentle 0.11  push 128.8  maxQ  -45.3
17:23:09  ep  3300/10000  eps 0.139  R100  -238.1  succ 0.03  gentle 0.03  push 132.4  maxQ  -45.5
17:23:11  ep  3400/10000  eps 0.130  R100  -238.2  succ 0.03  gentle 0.03  push 131.9  maxQ  -46.1
17:23:12  ep  3500/10000  eps 0.123  R100  -235.2  succ 0.07  gentle 0.06  push 128.6  maxQ  -46.7
17:23:13  ep  3600/10000  eps 0.116  R100  -223.8  succ 0.38  gentle 0.32  push 130.2  maxQ  -46.9
17:23:15  ep  3700/10000  eps 0.109  R100  -224.9  succ 0.33  gentle 0.30  push 130.4  maxQ  -47.1
17:23:16  ep  3800/10000  eps 0.103  R100  -227.7  succ 0.19  gentle 0.19  push 124.2  maxQ  -47.5
17:23:17  ep  3900/10000  eps 0.097  R100  -232.2  succ 0.13  gentle 0.13  push 129.2  maxQ  -47.9
17:23:19  ep  4000/10000  eps 0.091  R100  -229.8  succ 0.18  gentle 0.17  push 130.1  maxQ  -48.3
17:23:20  ep  4100/10000  eps 0.086  R100  -230.4  succ 0.16  gentle 0.15  push 127.7  maxQ  -48.9
17:23:22  ep  4200/10000  eps 0.081  R100  -220.5  succ 0.32  gentle 0.30  push 121.8  maxQ  -49.5
17:23:23  ep  4300/10000  eps 0.076  R100  -210.7  succ 0.55  gentle 0.54  push 123.8  maxQ  -49.6
17:23:24  ep  4400/10000  eps 0.072  R100  -222.8  succ 0.30  gentle 0.29  push 123.6  maxQ  -49.8
17:23:26  ep  4500/10000  eps 0.068  R100  -224.8  succ 0.26  gentle 0.24  push 126.6  maxQ  -50.3
17:23:27  ep  4600/10000  eps 0.064  R100  -226.5  succ 0.20  gentle 0.20  push 126.9  maxQ  -50.8
17:23:28  ep  4700/10000  eps 0.060  R100  -223.4  succ 0.30  gentle 0.27  push 126.3  maxQ  -51.0
17:23:30  ep  4800/10000  eps 0.056  R100  -225.5  succ 0.23  gentle 0.23  push 123.9  maxQ  -51.6
17:23:31  ep  4900/10000  eps 0.053  R100  -224.9  succ 0.23  gentle 0.22  push 125.4  maxQ  -51.9
17:23:32  ep  5000/10000  eps 0.050  R100  -232.3  succ 0.18  gentle 0.14  push 129.5  maxQ  -52.4
17:23:34  ep  5100/10000  eps 0.050  R100  -227.3  succ 0.24  gentle 0.24  push 128.0  maxQ  -52.9
```

```
17:23:35  ep  5200/10000  eps 0.050  R100  -213.3  succ 0.57  gentle 0.55  push 121.3  maxQ  -53.2
17:23:36  ep  5300/10000  eps 0.050  R100  -207.0  succ 0.59  gentle 0.58  push 119.3  maxQ  -53.7
17:23:38  ep  5400/10000  eps 0.050  R100  -203.2  succ 0.68  gentle 0.63  push 121.3  maxQ  -53.7
17:23:39  ep  5500/10000  eps 0.050  R100  -197.8  succ 0.68  gentle 0.65  push 118.1  maxQ  -53.8
17:23:40  ep  5600/10000  eps 0.050  R100  -196.2  succ 0.69  gentle 0.69  push 118.6  maxQ  -54.1
17:23:41  ep  5700/10000  eps 0.050  R100  -201.6  succ 0.60  gentle 0.59  push 121.8  maxQ  -54.5
17:23:43  ep  5800/10000  eps 0.050  R100  -213.8  succ 0.44  gentle 0.42  push 122.8  maxQ  -55.2
17:23:44  ep  5900/10000  eps 0.050  R100  -209.1  succ 0.52  gentle 0.50  push 121.6  maxQ  -55.5
17:23:45  ep  6000/10000  eps 0.050  R100  -208.1  succ 0.55  gentle 0.55  push 122.0  maxQ  -55.4
17:23:47  ep  6100/10000  eps 0.050  R100  -223.6  succ 0.32  gentle 0.32  push 121.1  maxQ  -55.8
17:23:48  ep  6200/10000  eps 0.050  R100  -201.7  succ 0.64  gentle 0.59  push 117.1  maxQ  -55.9
17:23:49  ep  6300/10000  eps 0.050  R100  -202.9  succ 0.67  gentle 0.55  push 120.4  maxQ  -56.0
17:23:50  ep  6400/10000  eps 0.050  R100  -197.5  succ 0.71  gentle 0.63  push 120.0  maxQ  -56.2
17:23:52  ep  6500/10000  eps 0.050  R100  -195.4  succ 0.70  gentle 0.65  push 113.8  maxQ  -56.4
17:23:53  ep  6600/10000  eps 0.050  R100  -202.6  succ 0.54  gentle 0.54  push 111.3  maxQ  -56.6
17:23:54  ep  6700/10000  eps 0.050  R100  -221.3  succ 0.27  gentle 0.26  push 119.5  maxQ  -57.0
17:23:56  ep  6800/10000  eps 0.050  R100  -221.4  succ 0.31  gentle 0.30  push 118.4  maxQ  -57.6
17:23:57  ep  6900/10000  eps 0.050  R100  -211.8  succ 0.52  gentle 0.46  push 120.7  maxQ  -57.8
17:23:58  ep  7000/10000  eps 0.050  R100  -202.1  succ 0.69  gentle 0.62  push 119.7  maxQ  -58.0
17:23:59  ep  7100/10000  eps 0.050  R100  -190.4  succ 0.84  gentle 0.82  push 112.7  maxQ  -58.2
17:24:01  ep  7200/10000  eps 0.050  R100  -195.7  succ 0.65  gentle 0.63  push 111.8  maxQ  -58.5
17:24:02  ep  7300/10000  eps 0.050  R100  -199.0  succ 0.60  gentle 0.56  push 112.4  maxQ  -58.6
17:24:03  ep  7400/10000  eps 0.050  R100  -183.6  succ 0.79  gentle 0.79  push 111.9  maxQ  -58.8
17:24:04  ep  7500/10000  eps 0.050  R100  -200.9  succ 0.63  gentle 0.60  push 117.6  maxQ  -59.1
17:24:06  ep  7600/10000  eps 0.050  R100  -196.8  succ 0.78  gentle 0.76  push 113.8  maxQ  -59.4
17:24:07  ep  7700/10000  eps 0.050  R100  -190.6  succ 0.79  gentle 0.77  push 107.6  maxQ  -59.4
17:24:08  ep  7800/10000  eps 0.050  R100  -188.6  succ 0.81  gentle 0.80  push 108.9  maxQ  -59.5
17:24:09  ep  7900/10000  eps 0.050  R100  -186.6  succ 0.75  gentle 0.74  push 105.5  maxQ  -59.7
17:24:10  ep  8000/10000  eps 0.050  R100  -186.8  succ 0.75  gentle 0.74  push 111.2  maxQ  -60.0
17:24:12  ep  8100/10000  eps 0.050  R100  -201.4  succ 0.61  gentle 0.61  push 112.9  maxQ  -60.3
17:24:13  ep  8200/10000  eps 0.050  R100  -193.7  succ 0.71  gentle 0.68  push 110.5  maxQ  -60.4
17:24:14  ep  8300/10000  eps 0.050  R100  -198.1  succ 0.62  gentle 0.62  push 113.4  maxQ  -60.9
17:24:15  ep  8400/10000  eps 0.050  R100  -194.2  succ 0.63  gentle 0.62  push 115.9  maxQ  -61.3
17:24:17  ep  8500/10000  eps 0.050  R100  -204.1  succ 0.61  gentle 0.58  push 115.1  maxQ  -61.6
17:24:18  ep  8600/10000  eps 0.050  R100  -189.4  succ 0.90  gentle 0.86  push 113.6  maxQ  -61.4
17:24:20  ep  8700/10000  eps 0.050  R100  -190.4  succ 0.83  gentle 0.78  push 107.8  maxQ  -61.5
17:24:22  ep  8800/10000  eps 0.050  R100  -186.7  succ 0.81  gentle 0.76  push 102.4  maxQ  -61.6
17:24:23  ep  8900/10000  eps 0.050  R100  -182.7  succ 0.80  gentle 0.76  push 105.8  maxQ  -61.7
17:24:24  ep  9000/10000  eps 0.050  R100  -199.2  succ 0.59  gentle 0.56  push 108.9  maxQ  -61.9
17:24:25  ep  9100/10000  eps 0.050  R100  -204.1  succ 0.55  gentle 0.52  push 111.6  maxQ  -62.2
17:24:27  ep  9200/10000  eps 0.050  R100  -199.5  succ 0.70  gentle 0.65  push 108.9  maxQ  -62.2
17:24:28  ep  9300/10000  eps 0.050  R100  -190.7  succ 0.73  gentle 0.69  push 107.2  maxQ  -62.4
17:24:29  ep  9400/10000  eps 0.050  R100  -194.6  succ 0.67  gentle 0.63  push 104.1  maxQ  -62.6
17:24:30  ep  9500/10000  eps 0.050  R100  -208.4  succ 0.50  gentle 0.50  push 113.2  maxQ  -63.0
17:24:32  ep  9600/10000  eps 0.050  R100  -185.3  succ 0.78  gentle 0.75  push 108.3  maxQ  -63.1
17:24:33  ep  9700/10000  eps 0.050  R100  -170.5  succ 0.93  gentle 0.92  push 104.4  maxQ  -63.1
17:24:34  ep  9800/10000  eps 0.050  R100  -174.7  succ 0.90  gentle 0.87  push 107.3  maxQ  -63.2
17:24:35  ep  9900/10000  eps 0.050  R100  -171.9  succ 0.91  gentle 0.91  push 105.1  maxQ  -63.3
17:24:36  ep 10000/10000  eps 0.050  R100  -169.6  succ 0.94  gentle 0.90  push 102.3  maxQ  -63.4
17:24:36  done in 129.6 s -> outputs/log_sarsa_modified_obs_seed109_ep10000.csv
```

3.3 Proof of identical experimental conditions

In [45]:

```
conditions = pd.DataFrame(
    [
        {
            "agent": r.label,
            "seed": r.seed,
            "grid": f"{DISC.bins[0]}x{DISC.bins[1]}",
            "Q-table shape": r.agent.Q.shape,
            "initial Q": 0.0,
            "alpha": r.agent.alpha,
            "gamma": r.agent.gamma,
            "episodes": len(r.log),
            "eps first/last": f"{r.log['epsilon'].iloc[0]:.2f} / "
            f"{r.log['epsilon'].iloc[-1]:.2f}",
            "train time (s)": round(r.seconds, 1),
        }
        for r in RUNS.values()
    ]
)
display(conditions)  # show the conditions table
```

	agent	seed	grid	Q-table shape	initial Q	alpha	gamma	episodes	eps first/last	train time (s)
0	Q-learning |	109	36x28	(36, 28, 3)	0.0	0.1	0.99	10000	1.00 / 0.05	113.4

	original									
1	Q-learning | sticky	109	36x28	(36, 28, 3)	0.0	0.1	0.99	10000	1.00 / 0.05	150.0
2	SARSA | original	109	36x28	(36, 28, 3)	0.0	0.1	0.99	10000	1.00 / 0.05	102.6
3	SARSA | sticky	109	36x28	(36, 28, 3)	0.0	0.1	0.99	10000	1.00 / 0.05	129.6

4. Part (d) — Performance evaluation

Four agents: Q-learning and SARSA, each on the original and the sticky (modified) environment. All curves are trailing means over the previous 100 episodes; the faint lines in D1 are raw per-episode values.

In [47]:

```
AGENTS = [
    ("q_learning", "original"),
    ("sarsa", "original"),
    ("q_learning", "modified"),
    ("sarsa", "modified"),
]


def curve(
    ax, run: RunResult, column: str, raw: bool = False, label: str = None
) -> None:
    """Plot the 100-episode trailing mean of one log column (optionally with the raw
    values faintly behind)."""
    y = run.log[column].to_numpy(dtype=float)  # raw per-episode values
    # encoding: colour=algo, style=env
    color, style = ALGO_COLOR[run.algorithm], ENV_STYLE[run.env_kind]
    if raw:  # raw curve only when requested
        ax.plot(y, color=color, lw=0.5, alpha=0.12)  # raw values, very faint
    # trailing 100-episode mean
    ax.plot(trailing_mean(y), color=color, ls=style, lw=1.8, label=label or run.label)

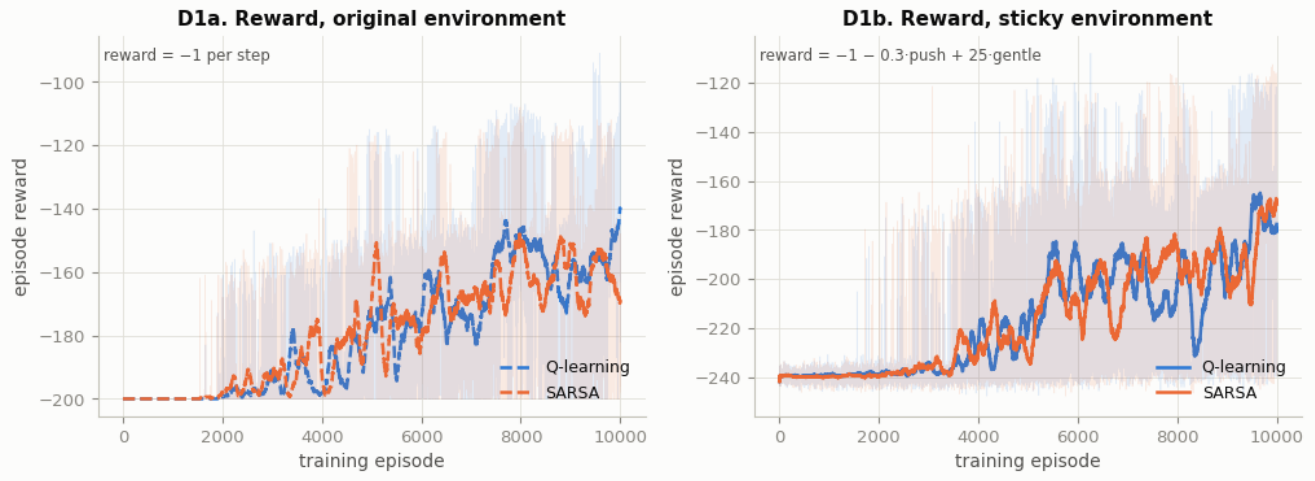

lab_stamp("part (d) - figures")  # stamp the start of part (d)
```

```
[lab-stamp] 2026-11-09 17:24:36 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | part
(d) - figures
```

4.1 Episode reward versus training episode

In [49]:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 3.8))
# separate panels: different reward scales
for ax, env_kind in zip(axes, ("original", "modified")):
    for alg in ("q_learning", "sarsa"):  # both algorithms in this panel
        # raw + moving average
        curve(ax, RUNS[(alg, env_kind)], "reward", raw=True, label=ALGO_LABEL[alg])
    # axis labels and panel title
    ax.set(
        xlabel="training episode",
        ylabel="episode reward",
        title=f"D1{'a' if env_kind == 'original' else 'b'}. Reward, "
        f"{ENV_LABEL[env_kind]} environment",
    )
    ax.legend(loc="lower right")  # legend in the empty corner
# reward definition, original env
axes[0].text(
    0.01,
    0.97,
    "reward = −1 per step",
    transform=axes[0].transAxes,
    va="top",
    fontsize=8.5,
    color=INK_2,
)
# reward definition, sticky env
axes[1].text(
    0.01,
    0.97,
    "reward = −1 − 0.3·push + 25·gentle",
    transform=axes[1].transAxes,
    va="top",
    fontsize=8.5,
    color=INK_2,
)
save_fig(fig, "D1_reward")  # save and show
```



Interpretation (D1).

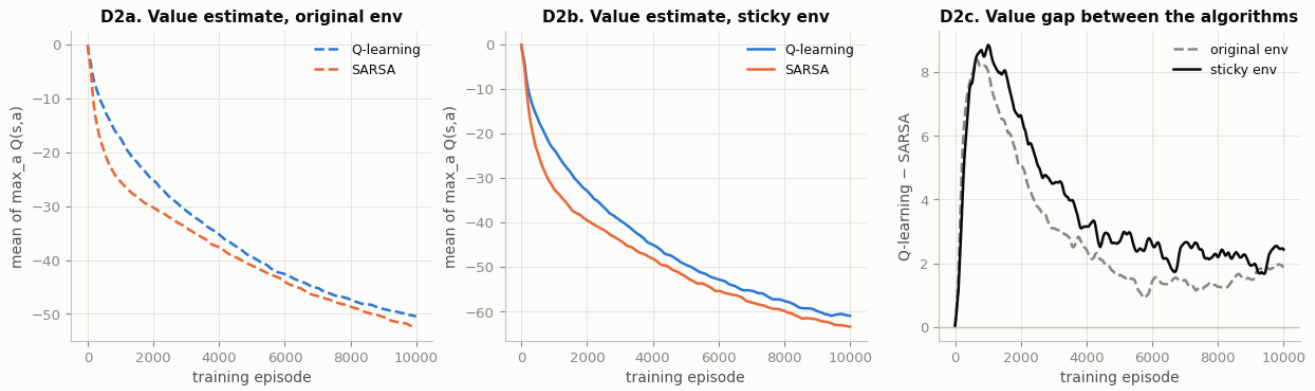
- **Plateau first.** In both environments no agent reaches the flag for roughly the first 2 000 episodes, so every episode scores the floor: −200 in the original env and about −240 in the sticky env (−200 plus 0.3 for each of the ~130 pushes an exploring agent selects). This is the classic MountainCar exploration problem.
- **Original env.** Learning starts once ε has dropped to ≈ 0.3–0.4 (episodes 1 600–2 000) and the optimistic zero-initialised values along the valley floor have been explored away. Both algorithms climb to about −140 … −170 by episode 10 000.
- **Sticky env.** The rise starts later (≈ episode 3 500) and is noisier. A wrong action can persist for several steps, and every selected push costs 0.3. The final level (≈ −170 … −180) already includes the +25 earned by gentle arrivals; without it these episodes would score ≈ −185 … −200.
- **Q-learning vs SARSA.** The curves overlap for most of training. The two algorithms differ more clearly in their final greedy policies (table 4.6) than in their ε-greedy training reward.

4.2 Average predicted value, max_a Q(s, a), on the fixed validation set

In [52]:

```
fig, axes = plt.subplots(1, 3, figsize=(15, 3.8))
# one panel per environment
for ax, env_kind in zip(axes[:2], ("original", "modified")):
    for alg in ("q_learning", "sarsa"):  # both algorithms
        # value-estimate curve
        curve(ax, RUNS[(alg, env_kind)], "avg_max_q", label=ALGO_LABEL[alg])
    ax.set(
        xlabel="training episode",
        ylabel="mean of max_a Q(s,a)",  # labels and title
        title=f"D2{'a' if env_kind == 'original' else 'b'}. Value estimate, "
        f"{ENV_LABEL[env_kind]} env",
    )
    ax.legend(loc="upper right")  # legend
ax = axes[2]  # gap Q-learning minus SARSA
for env_kind in ("original", "modified"):  # both environments
    # gap per episode
    gap = (
        RUNS[("q_learning", env_kind)].log["avg_max_q"]
        - RUNS[("sarsa", env_kind)].log["avg_max_q"]
    )
    # gap curve
    ax.plot(
        trailing_mean(gap),
        color=INK if env_kind == "modified" else INK_MUTED,
        ls=ENV_STYLE[env_kind],
        label=f"{ENV_LABEL[env_kind]} env",
    )
ax.axhline(0, color="#c3c2b7", lw=1)  # zero-gap reference
# labels and title
ax.set(
    xlabel="training episode",
    ylabel="Q-learning − SARSA",
    title="D2c. Value gap between the algorithms",
```

```
)
ax.legend(loc="upper right")  # legend
save_fig(fig, "D2_avg_max_q")  # save and show
gap_table = pd.DataFrame(
    {
        env_kind: {  # final values per environment
            "Q-learning avg max Q (last 500 ep)": RUNS[("q_learning", env_kind)]
            .log["avg_max_q"]
            .iloc[-500:]
            .mean(),
            "SARSA avg max Q (last 500 ep)": RUNS[("sarsa", env_kind)]
            .log["avg_max_q"]
            .iloc[-500:]
            .mean(),
        }
        for env_kind in ("original", "modified")
    }
).T
# absolute gap
gap_table["gap (Q - SARSA)"] = gap_table.iloc[:, 0] - gap_table.iloc[:, 1]
# gap relative to SARSA
gap_table["relative gap"] = gap_table["gap (Q - SARSA)"] / gap_table.iloc[:, 1].abs()
display(gap_table.round(3))  # show the gap table
```



	Q-learning avg max Q (last 500 ep)	SARSA avg max Q (last 500 ep)	gap (Q - SARSA)	relative gap
original	-50.178	-52.061	1.883	0.036
modified	-60.712	-63.163	2.451	0.039

Interpretation D2. The average predicted values decrease as initially zero entries learn mostly negative returns. They continue to drift at episode 10 000, so convergence is not established. The positive gentle terminal reward is an exception to negative step rewards.

Q-learning is generally above SARSA, whose sampled next action incorporates epsilon exploration. Maximisation bias is another possible contributor; this experiment does not measure it separately.

Over the last 500 episodes, Q-learning minus SARSA is 2.451 in the modified environment and 1.883 in the original environment (relative to absolute SARSA values, 3.9% and 3.6%). The three-seed means are approximately 3.0 and 1.4. These differences compare the complete modified environment with the original. Stickiness, the push penalty and the bonus change together, so an actuator-only causal effect cannot be identified. An ablation with identical rewards and only stick probability varied would resolve this.

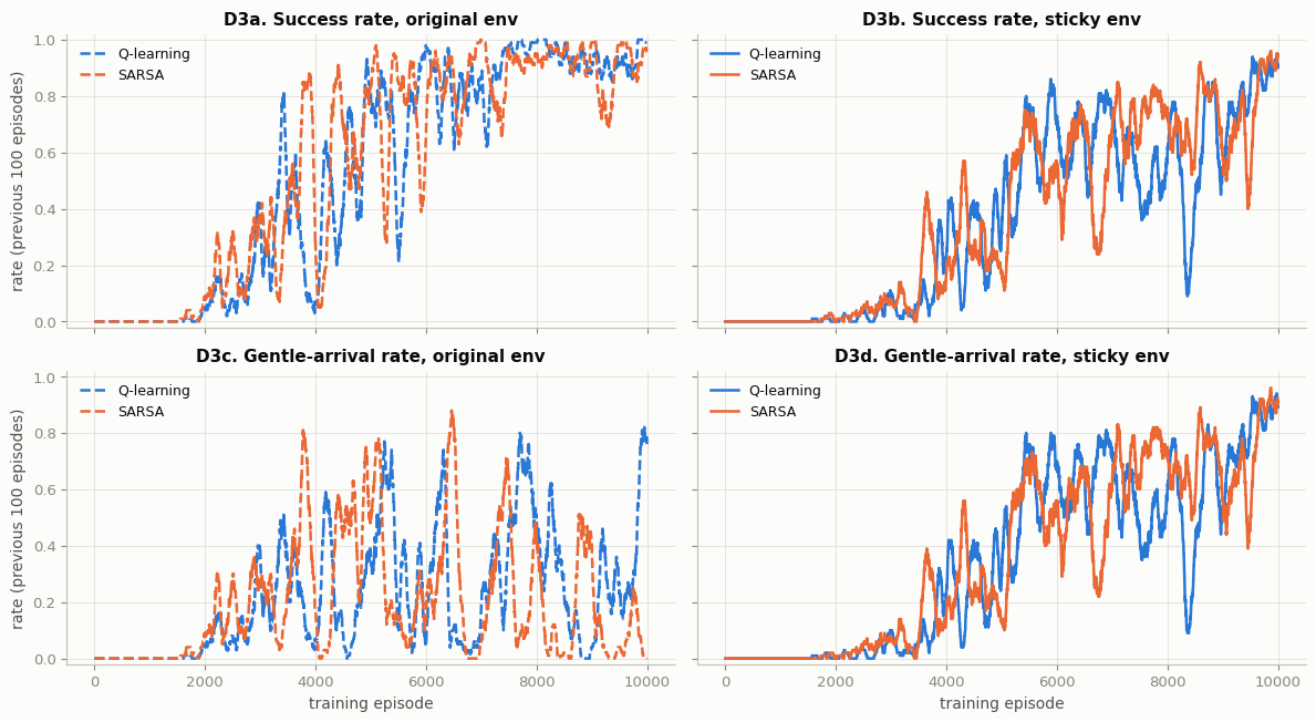
4.3 Success rate and gentle-arrival rate (trailing 100 episodes)

Success = the flag was reached (terminated). Gentle arrival = the bonus criterion (terminated, not truncated, v ≤ 0.03). The criterion is also evaluated for the original-environment agents, even though they were never rewarded for it.

In [55]:

```
# small multiples: metric x env
fig, axes = plt.subplots(2, 2, figsize=(12, 6.6), sharex=True, sharey=True)
panel = iter("abcd")  # panel letters D3a..D3d
# rows: success / gentle
for row, (column, name) in enumerate(
    (("success", "Success rate"), ("gentle", "Gentle-arrival rate"))
```

```
):
    # columns: original / sticky
    for col, env_kind in enumerate(("original", "modified")):
        ax = axes[row, col]  # current panel
        for alg in ("q_learning", "sarsa"):  # two lines per panel
            # Q-learning and SARSA
            curve(ax, RUNS[(alg, env_kind)], column, label=ALGO_LABEL[alg])
        # fixed 0-1 range and title
        ax.set(
            ylim=(-0.02, 1.02),
            title=f"D3{next(panel)}. {name}, {ENV_LABEL[env_kind]} env",
        )
        ax.legend(loc="upper left")  # legend
for ax in axes[1]:  # bottom row
    ax.set_xlabel("training episode")  # x label once
for ax in axes[:, 0]:  # left column
    ax.set_ylabel("rate (previous 100 episodes)")  # y label once
fig.tight_layout()  # avoid overlapping titles
save_fig(fig, "D3_success_gentle")  # save and show
```



Interpretation (D3).

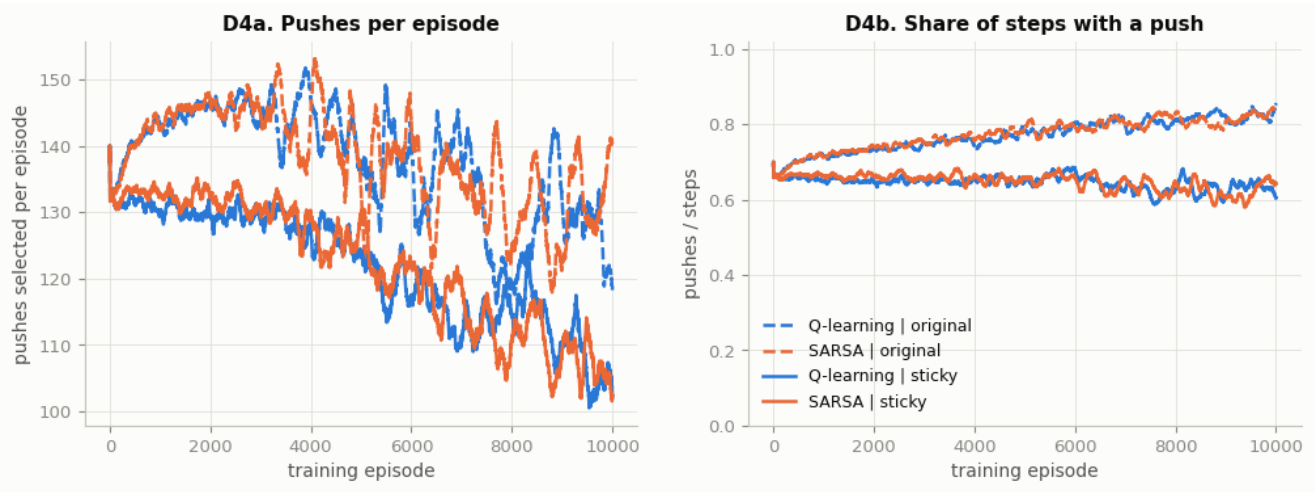
- **Original env (left).** Both algorithms go from 0 to a sustained ≈ 90 % success between episodes ≈ 2 000 and 7 000. Their gentle-arrival rate swings between 0 and 0.9 and ends near 0 for SARSA: nothing rewards a slow arrival, so arrival speed is just a side effect of whichever pumping cycle the agent locks into.
- **Sticky env (right).** Success rises later (≈ 20–30 % at episode 5 000 vs > 80 % in the original env) and more erratically; Q-learning even drops to ≈ 0.1 around episode 8 300 before recovering. Hidden actuator delays make the same greedy policy succeed in one episode and miss in the next.
- **The bonus shapes the arrival.** In the sticky env the gentle-arrival curve sits almost on top of the success curve: nearly every successful episode ends with a gentle arrival. Both algorithms finish at ≈ 0.9 or above on both metrics.

4.4 Push actions selected per episode

In [58]:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 3.8))
for alg, env_kind in AGENTS:  # all four agents
    run = RUNS[(alg, env_kind)]  # its run result
    curve(axes[0], run, "pushes")  # absolute pushes per episode
    # colour = algorithm, style = environment
    color, style = ALGO_COLOR[alg], ENV_STYLE[env_kind]
    # share of steps with a push
    axes[1].plot(
```

```
        trailing_mean(run.log["pushes"] / run.log["length"]),
        color=color,
        ls=style,
        label=run.label,
    )
# left panel labels
axes[0].set(
    xlabel="training episode",
    ylabel="pushes selected per episode",
    title="D4a. Pushes per episode",
)
# right panel labels
axes[1].set(
    xlabel="training episode",
    ylabel="pushes / steps",
    title="D4b. Share of steps with a push",
    ylim=(0, 1.02),
)
axes[1].legend(loc="lower left")  # legend
save_fig(fig, "D4_pushes")  # save and show
```



Interpretation D4. The selected-push fraction ends around 0.62 in the modified environment and 0.84 in the original environment. The modified greedy policies also use fewer pushes. These observations are consistent with the energy penalty encouraging coasting. They do not isolate its effect because actuator noise and the bonus also change. Shorter episodes can reduce counts, which is why the push fraction is reported alongside the absolute count. Both algorithms show the same broad pattern; their precise counts still differ.

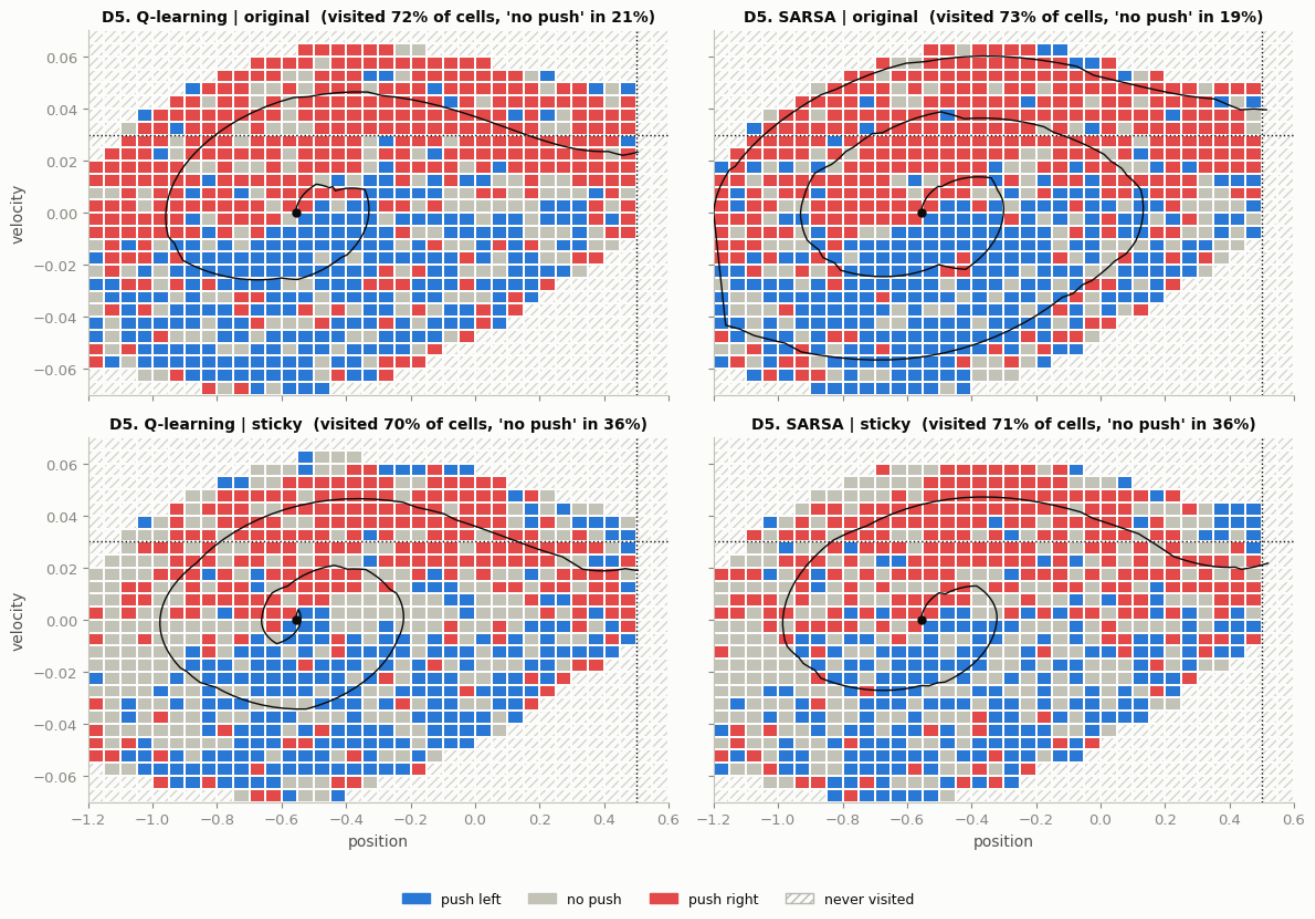
4.5 Greedy policy maps

Each cell is one discretised state, coloured by argmax_a Q(s, a). Hatched cells were never updated (all-zero rows), so the map omits their arbitrary argmax. Visited ties use NumPy first-index argmax in this visualisation; evaluation breaks ties randomly. The map shows a representative greedy action, not a unique action in tied cells. The black line is one greedy test episode (seed 50 000) in that agent's own environment.

In [61]:

```
def greedy_trajectory(run: RunResult, env_kind: str, seed: int) -> np.ndarray:
    """Positions/velocities visited by the greedy policy (ε = 0) in one seeded
    episode."""
    env = make_env(env_kind)  # evaluation environment
    encode = make_state_encoder(run.state_mode)  # state representation of the agent
    tie_rng = np.random.default_rng([seed, 3])  # deterministic tie-breaking
    obs, _ = env.reset(seed=seed)  # seeded start
    path, last = [obs.copy()], NO_PUSH  # path starts at the initial state
    while True:  # until the episode ends
        a = run.agent.greedy_action(encode(obs, last), tie_rng)  # greedy action
        obs, _, te, tr, _ = env.step(a)  # step the environment
        path.append(obs.copy())  # record the new state
        last = a  # remember the selected action
        if te or tr:  # end of episode
            env.close()  # release the trajectory environment
            return np.array(path)  # array of visited states
```

```
POLICY_COLORS = ["#2a78d6", "#c3c2b7", "#e34948"]  # push left / no push / push right
policy_cmap = ListedColormap(POLICY_COLORS)  # 3-colour map
policy_cmap.set_bad((0, 0, 0, 0))  # unvisited cells transparent (hatch shows)
# 2 x 2 small multiples
fig, axes = plt.subplots(2, 2, figsize=(12, 8.2), sharex=True, sharey=True)
# one panel per agent
for ax, (alg, env_kind) in zip(
    axes.flat,
    [
        ("q_learning", "original"),
        ("sarsa", "original"),
        ("q_learning", "modified"),
        ("sarsa", "modified"),
    ],
):
    run = RUNS[(alg, env_kind)]  # its run result
    greedy = run.agent.Q.argmax(axis=-1).astype(float)  # greedy action per state
    greedy[run.visits == 0] = np.nan  # mask states never updated
    ax.add_patch(
        Rectangle(
            (-1.2, -0.07),
            1.8,
            0.14,
            facecolor="#fcfcfb",
            edgecolor="#d6d5cf",
            # hatched background = unvisited
            hatch="////",
            lw=0,
            zorder=0,
        )
    )
    ax.pcolormesh(
        DISC.edges[0],
        DISC.edges[1],
        np.ma.masked_invalid(greedy.T),
        cmap=policy_cmap,
        vmin=-0.5,
        vmax=2.5,
        edgecolors="#fcfcfb",
        linewidth=0.3,
        zorder=1,
    )
    path = greedy_trajectory(run, env_kind, TEST_SEEDS[0])  # one greedy test episode
    ax.plot(path[:, 0], path[:, 1], color=INK, lw=1.0, zorder=3)  # trajectory line
    ax.plot(path[0, 0], path[0, 1], "o", ms=5, color=INK, zorder=3)  # start point
    ax.axvline(GOAL_POSITION, color=INK, ls=":", lw=1, zorder=2)  # flag
    ax.axhline(GENTLE_SPEED, color=INK, ls=":", lw=1, zorder=2)  # gentle threshold
    # title with coverage statistics
    ax.set_title(
        f"D5. {run.label}  (visited {np.isfinite(greedy).mean():.0%} of cells, "
        f"'no push' in {np.mean(greedy[np.isfinite(greedy)] == 1):.0%})",
        fontsize=10,
    )
    ax.set(xlim=(-1.2, 0.6), ylim=(-0.07, 0.07))  # observation box
    ax.grid(False)  # cells replace the grid
for ax in axes[1]:  # bottom row
    ax.set_xlabel("position")  # x label
for ax in axes[:, 0]:  # left column
    ax.set_ylabel("velocity")  # y label
# legend swatches
handles = [Patch(color=c, label=ACTION_NAMES[i]) for i, c in enumerate(POLICY_COLORS)]
handles.append(
    Patch(facecolor="#fcfcfb", edgecolor="#b8b7b0", hatch="////", label="never visited")
)
# shared legend under the panels
fig.legend(handles=handles, loc="lower center", ncol=4, bbox_to_anchor=(0.5, -0.02))
fig.tight_layout(rect=(0, 0.04, 1, 1))  # leave space for the legend
save_fig(fig, "D5_policy_maps")  # save and show
```



Interpretation (D5).

- **Original env (top): bang-bang.** Red (push right) fills most of the upper half (v > 0) and blue (push left) most of the lower half (v < 0). The agents have discovered "push in the direction of the velocity" for themselves; only ≈ 20 % of visited cells choose 'no push'.
- **Sticky env (bottom): coasting.** 'No push' grows to ≈ 36 % of cells, mostly where the car turns around (|v| small) and on the far-left slope, where gravity alone reverses the car.
- **Sticky env: braking.** Just left of the flag at high speed (x ≈ 0.4–0.5, v ≈ 0.03–0.045) there are **blue "brake" cells**: push-left selections are consistent with braking. A static action map cannot establish that these cells cause a gentle arrival, especially when actuator execution is hidden.
- **Greedy trajectories (black).** SARSA-original crosses the flag still accelerating (v ≈ 0.04), while both sticky agents level off at v ≈ 0.02 on the final approach. Q-learning-original also arrives slowly here, but that is chance (see 4.7).
- **The two algorithms.** In each environment Q-learning and SARSA produce similar structure; most differences are in rarely visited cells at the edge of the visited region. About 30 % of cells are never visited (hatched): some may be physically unreachable, while others may simply be undersampled. Non-visitation alone does not distinguish these cases.

4.6 Greedy test: 100 episodes with fixed seeds, ε = 0

Seeds 50 000 – 50 099 fix both the initial state and (in the modified env) the actuator's random stream, so all agents use the same reset seeds and initial states. Actuator draws are conditional on attempted changes, so different policies do not necessarily receive the same disturbances at the same time step. Ties in argmax are broken by a generator seeded with the episode seed.

In [64]:

```
def evaluate_greedy(run: RunResult, env_kind: str, seeds=TEST_SEEDS) -> pd.DataFrame:
    """Run the greedy policy (ε = 0) for one episode per seed; return one row per
    episode."""
    env = make_env(env_kind)  # evaluation environment
    encode = make_state_encoder(run.state_mode)  # agent's state representation
```

```
    rows = []  # one row per test episode
    for sd in seeds:  # 100 fixed seeds
        obs, _ = env.reset(seed=sd)  # fixed initial state (and actuator stream)
        tie_rng = np.random.default_rng([sd, 3])  # reproducible tie-breaking
        last, total, steps, pushes = NO_PUSH, 0.0, 0, 0  # per-episode accumulators
        while True:  # until the episode ends
            a = run.agent.greedy_action(encode(obs, last), tie_rng)  # ε = 0
            obs, r, te, tr, _ = env.step(a)  # step the environment
            # update the totals
            total, steps, pushes, last = (
                total + r,
                steps + 1,
                pushes + int(a != NO_PUSH),
                a,
            )
            if te or tr:  # end of episode
                break  # leave the step loop
        # record the episode
        rows.append(
            dict(
                seed=sd,
                reward=total,
                length=steps,
                pushes=pushes,
                success=te,
                gentle=bool(
                    te and not tr and obs[0] >= GOAL_POSITION and obs[1] <= GENTLE_SPEED
                ),
                arrival_velocity=float(obs[1]) if te else np.nan,
            )
        )
    result = pd.DataFrame(rows)  # keep the raw fixed-seed test evidence
    # unique file name: trained agent + the environment it was tested in
    tag = (
        f"{run.algorithm}_{run.env_kind}_{run.state_mode}_seed{run.seed}"
        f"_ep{len(run.log)}_tested_{env_kind}"
    )
    # raw per-episode test results on disk
    result.to_csv(os.path.join(OUT_DIR, f"test_{tag}.csv"), index=False)
    env.close()  # release the test environment
    return result  # one row per episode


def summarise(df: pd.DataFrame) -> dict:
    """Collapse 100 test episodes into the metrics reported in the evaluation table."""
    # metrics of the evaluation table
    return {
        "mean reward": df["reward"].mean(),
        "std reward": df["reward"].std(),
        "success rate": df["success"].mean(),
        "gentle-arrival rate": df["gentle"].mean(),
        "mean episode length": df["length"].mean(),
        "pushes / episode": df["pushes"].mean(),
        "mean arrival speed": df["arrival_velocity"].mean(),
    }


lab_stamp("part (d) - greedy test episodes")  # stamp the greedy tests
# each agent in its own environment
EVAL = {key: evaluate_greedy(RUNS[key], key[1]) for key in AGENTS}
# one row per agent
eval_table = pd.DataFrame(
    {RUNS[key].label: summarise(df) for key, df in EVAL.items()}
).T
eval_table.to_csv(os.path.join(OUT_DIR, "eval_table.csv"))  # save to CSV
display(eval_table.round(3))  # show the table
```

```
[lab-stamp] 2026-11-09 17:24:43 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | part (d) - greedy test episodes
```

	mean reward	std reward	success rate	gentle-arrival rate	mean episode length	pushes / episode	mean arrival speed
Q-learning | original	-139.040	25.186	0.99	0.63	139.04	119.66	0.030
SARSA | original	-171.710	21.016	1.00	0.00	171.71	151.47	0.040
Q-learning | sticky	-158.996	21.472	0.98	0.98	154.99	95.02	0.022

SARSA | sticky	-180.032	33.387	0.93	0.87	168.29	111.64	0.024

Interpretation (test table).

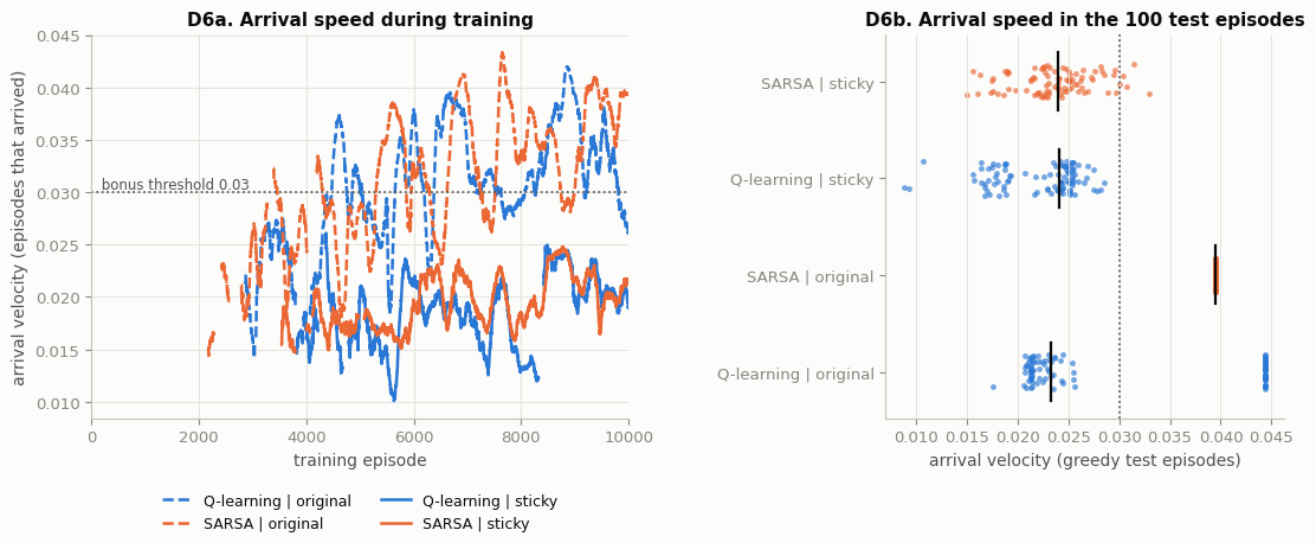
- **All four reach the flag.** Every greedy policy succeeds in ≥ 93 % of the 100 test episodes.
- **Original env.** Q-learning is clearly faster (139 vs 172 steps). SARSA's policy is a longer, more cautious pumping cycle that arrives at ≈ 0.04 every time, so it never earns the gentle criterion.
- **Sticky env.** Q-learning (seed 109) has better reported reward, success, gentle rate, length and push count: success 0.98 vs 0.93, gentle arrival 0.98 vs 0.87, reward −159 vs −180, and 95 vs 112 pushes per episode. Section 5.2 shows this ranking is partly seed-dependent.
- **Gentle arrival costs time.** The sticky agents need ≈ 155–170 steps against 139 for the fastest original agent; this is consistent with a time versus control tradeoff. A matched ablation is needed to establish the bonus as its cause.
- **Comparing across environments.** Mean reward cannot be compared between the two environments (different reward functions); success, gentle rate, length and pushes can.

4.7 Arrival-speed statistics (needed for part (e))

In [67]:

```
fig, axes = plt.subplots(
    1, 2, figsize=(13, 4.2), gridspec_kw={"width_ratios": [1.35, 1], "wspace": 0.55}
)
for alg, env_kind in AGENTS:  # training: mean arrival speed of arrivals
    run = RUNS[(alg, env_kind)]  # its run result
    v = run.log["arrival_velocity"]  # arrival speeds (NaN = no arrival)
    # rolling mean over arrivals
    axes[0].plot(
        v.rolling(100, min_periods=20).mean(),
        color=ALGO_COLOR[alg],
        ls=ENV_STYLE[env_kind],
        label=run.label,
    )
axes[0].axhline(GENTLE_SPEED, color=INK_2, ls=":", lw=1.2)  # bonus threshold
axes[0].text(
    0.01,
    GENTLE_SPEED,
    " bonus threshold 0.03",
    transform=axes[0].get_yaxis_transform(),
    va="bottom",
    fontsize=8.5,
    color=INK_2,
)  # x in axes units, y in data units
# labels and fixed x-range
axes[0].set(
    xlim=(0, CFG.n_episodes),
    xlabel="training episode",
    ylabel="arrival velocity (episodes that arrived)",
    title="D6a. Arrival speed during training",
)
# below the panel, clear of the data
axes[0].legend(loc="upper center", bbox_to_anchor=(0.5, -0.16), ncol=2)
ax = axes[1]  # test: strip plot of arrival speeds
jitter = np.random.default_rng([SEED, 5])  # vertical jitter so dots do not overlap
for row, (alg, env_kind) in enumerate(AGENTS):  # one row per agent
    # speeds of the test arrivals
    v = EVAL[(alg, env_kind)]["arrival_velocity"].dropna().to_numpy()
    # jittered dots
    ax.scatter(
        v,
        row + jitter.uniform(-0.18, 0.18, len(v)),
        s=12,
        color=ALGO_COLOR[alg],
        alpha=0.65,
        lw=0,
    )
    if len(v):  # only if the agent arrived
        # median tick
        ax.plot([np.median(v)] * 2, [row - 0.3, row + 0.3], color=INK, lw=1.6)
```

```
ax.axvline(GENTLE_SPEED, color=INK_2, ls=":", lw=1.2)  # bonus threshold
ax.set_yticks(range(4))  # one row per agent
ax.set_yticklabels([RUNS[k].label for k in AGENTS])  # agent names as row labels
# labels and title
ax.set(
    xlabel="arrival velocity (greedy test episodes)",
    title="D6b. Arrival speed in the 100 test episodes",
)
ax.grid(axis="y", visible=False)  # no horizontal grid lines
save_fig(fig, "D6_arrival_speed")  # save and show
speed_table = pd.DataFrame(
    {
        RUNS[k].label: {  # summary of arrival speeds
            "arrivals": int(EVAL[k]["success"].sum()),
            # gentle episodes over all 100 tests (the table-4.6 rate)
            "gentle episodes / all tests": EVAL[k]["gentle"].mean(),
            # full bonus criterion over arrivals only (excludes an arrival on step 200)
            "gentle criterion / arrivals": (
                EVAL[k]["gentle"].sum() / EVAL[k]["success"].sum()
                if EVAL[k]["success"].sum()
                else np.nan
            ),
            "median arrival v": EVAL[k]["arrival_velocity"].median(),
            "share of arrivals with v <= 0.03": (
                EVAL[k]["arrival_velocity"].dropna() <= GENTLE_SPEED
            ).mean(),
        }
        for k in AGENTS
    }
).T
display(speed_table.round(4))  # show the speed table
```



	arrivals	gentle episodes / all tests	gentle criterion / arrivals	median arrival v	share of arrivals with v <= 0.03
Q-learning | original	99.0	0.63	0.6364	0.0232	0.6364
SARSA | original	100.0	0.00	0.0000	0.0395	0.0000
Q-learning | sticky	98.0	0.98	1.0000	0.0241	1.0000
SARSA | sticky	93.0	0.87	0.9355	0.0240	0.9462

Interpretation (D6).

- **During training (D6a)** the sticky agents (solid) arrive at ≈ 0.010–0.025 almost from their first successes, below the 0.03 threshold. The original agents (dashed) drift *upwards* to ≈ 0.03–0.043 as they learn to pump harder, because nothing in their reward discourages a fast arrival.
- **In the greedy tests (D6b)** the sticky agents' arrivals cluster at 0.015–0.03 (median ≈ 0.024; 100 % of Q-learning's and 94.62% of SARSA's arrivals satisfy the speed threshold; its full gentle criterion rate among arrivals is 87/93 = 93.55%, because one arrival also truncates at step 200). SARSA-original arrives at 0.0395 in every episode (one fixed cycle), and Q-learning-original is bimodal (≈ 0.021–0.026 or ≈ 0.044), giving 63 gentle episodes out of all 100 tests and 63/99 = 63.64% among arrivals. These are different denominators.

5. Extra experiments (beyond the required tasks)

5.1 Cross-evaluation: robustness to the sticky actuator

The assignment title is about *robust* RL, so we test every greedy policy in **both** environments. An original-trained policy evaluated in the sticky env shows how fragile a policy is when it was never trained with actuator delays.

In [70]:

```
lab_stamp("extra - cross-evaluation")
cross_rows = []  # one row per agent and test env
for key in AGENTS:  # the four agents
    for test_env in ("original", "modified"):  # every agent in both environments
        s = summarise(evaluate_greedy(RUNS[key], test_env))  # 100 greedy test episodes
        # record the metrics
        cross_rows.append(
            {
                "agent (trained on)": RUNS[key].label,
                "tested on": ENV_LABEL[test_env],
                **{
                    k: s[k]
                    for k in (
                        "success rate",
                        "gentle-arrival rate",
                        "mean episode length",
                        "pushes / episode",
                    )
                },
            }
        )
cross = pd.DataFrame(cross_rows)  # cross-evaluation table
cross.to_csv(os.path.join(OUT_DIR, "cross_evaluation.csv"), index=False)  # save to CSV
display(cross.round(3))  # show the table
```

```
[lab-stamp] 2026-11-09 17:24:48 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | extra - cross-evaluation
```

	agent (trained on)	tested on	success rate	gentle-arrival rate	mean episode length	pushes / episode
0	Q-learning | original	original	0.99	0.63	139.04	119.66
1	Q-learning | original	sticky	1.00	0.70	135.19	116.02
2	SARSA | original	original	1.00	0.00	171.71	151.47
3	SARSA | original	sticky	0.88	0.00	173.56	150.45
4	Q-learning | sticky	original	1.00	1.00	152.89	93.26
5	Q-learning | sticky	sticky	0.98	0.98	154.99	95.02
6	SARSA | sticky	original	0.96	0.96	172.05	115.92
7	SARSA | sticky	sticky	0.93	0.87	168.29	111.64

Interpretation (cross-evaluation).

- **Sticky-trained agents transfer well.** In the clean environment their success and gentle rates are at least as high as in their own: Q-learning 1.00 / 1.00 (success / gentle) and SARSA 0.96 / 0.96, and they keep arriving gently even where it is no longer rewarded.
- **Clean-trained agents are less robust.** SARSA-original drops from 1.00 to 0.88 success once its actions start sticking: its precise pumping rhythm is broken by the delays. Q-learning-original happens to cope (0.99 → 1.00).
- **Gentle arrival varies by training condition.** Original-trained policies have gentle rates from 0.00 to 0.70 in these tests; they were not explicitly rewarded for slow arrival.
- **Conclusion.** These results support evaluating transfer explicitly. They do not establish a general robustness guarantee from one training seed.

5.2 Robustness across random seeds

One seed can be lucky. We repeat all four trainings with two more seeds (1109, 2109): same config, only the seed changes. The table reports mean ± standard deviation of the greedy-test metrics over the 3 seeds; the figure shows the mean success-rate curve with a min–max band.

In [73]:

```
EXTRA_SEEDS = (1109, 2109)  # two additional seeds
```

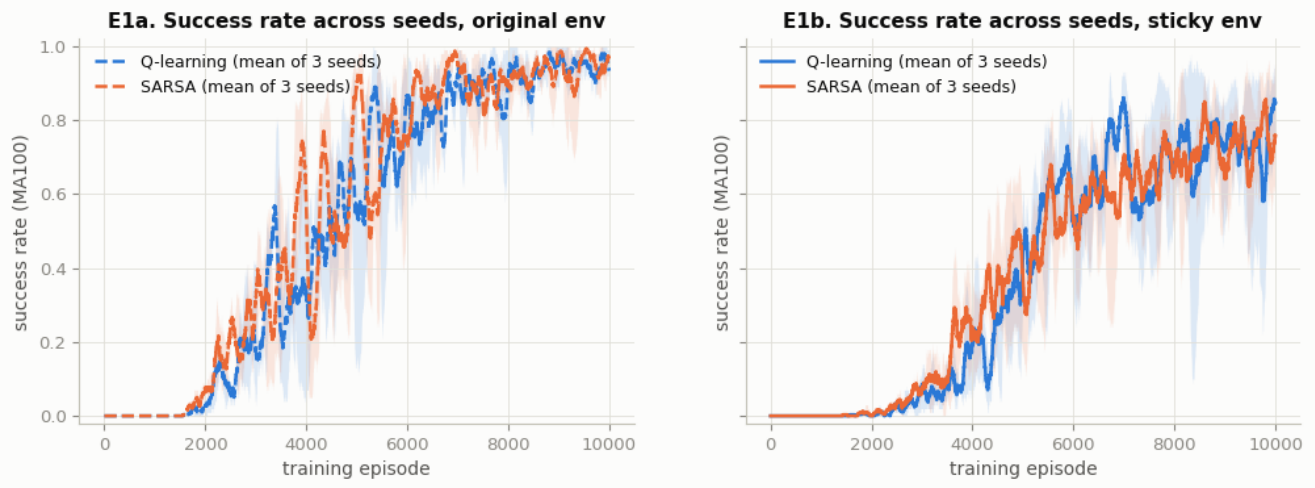
```
# reuse the seed-109 runs
SEED_RUNS = {(alg, env_kind, SEED): RUNS[(alg, env_kind)] for alg, env_kind in AGENTS}
lab_stamp("extra - multi-seed training")  # stamp the multi-seed runs
for sd in EXTRA_SEEDS:  # each extra seed
    for alg, env_kind in AGENTS:  # each agent
        # quiet run (log still saved to CSV)
        r = train_agent(alg, env_kind, seed=sd, verbose=False)
        SEED_RUNS[(alg, env_kind, sd)] = r  # store the run
        # one line per run
        print(
            f"{clock()}  {r.label:<21} seed {sd}  {r.seconds:5.1f} s  "
            f"last-500 success {r.log['success'].iloc[-500:].mean():.2f}"
        )
ALL_SEEDS = (SEED,) + EXTRA_SEEDS  # all three seeds
seed_rows = []  # evaluation rows
for (alg, env_kind, sd), r in SEED_RUNS.items():  # every (agent, seed) pair
    s = summarise(evaluate_greedy(r, env_kind))  # greedy test
    # metrics and final value estimate
    seed_rows.append(
        {
            "agent": r.label,
            "seed": sd,
            **s,
            "final avg max Q": r.log["avg_max_q"].iloc[-500:].mean(),
        }
    )
seed_df = pd.DataFrame(seed_rows)  # table of all runs
seed_df.to_csv(os.path.join(OUT_DIR, "multi_seed_eval.csv"), index=False)  # save to CSV
# metrics to aggregate
metrics = [
    "success rate",
    "gentle-arrival rate",
    "mean reward",
    "mean episode length",
    "pushes / episode",
    "final avg max Q",
]
# mean and std over seeds
agg = seed_df.groupby("agent", sort=False)[metrics].agg(["mean", "std"])
# format as mean +/- std
agg_fmt = pd.DataFrame(
    {
        m: agg[m]["mean"].map("{:.3f}".format)
        + " ± "
        + agg[m]["std"].map("{:.3f}".format)
        for m in metrics
    }
)
display(agg_fmt)  # show the summary

fig, axes = plt.subplots(1, 2, figsize=(12, 3.8), sharey=True)  # two panels
for ax, env_kind in zip(axes, ("original", "modified")):  # one per environment
    for alg in ("q_learning", "sarsa"):  # both algorithms
        curves = np.array(
            [
                trailing_mean(
                    SEED_RUNS[(alg, env_kind, sd)].log["success"].astype(float)
                )
                for sd in ALL_SEEDS
            ]
        )  # (3 seeds, episodes)
        x = np.arange(curves.shape[1])  # episode axis
        # min-max band over seeds
        ax.fill_between(
            x, curves.min(0), curves.max(0), color=ALGO_COLOR[alg], alpha=0.15, lw=0
        )
        # mean curve
        ax.plot(
            x,
            curves.mean(0),
            color=ALGO_COLOR[alg],
            ls=ENV_STYLE[env_kind],
            label=f"{ALGO_LABEL[alg]} (mean of 3 seeds)",
        )
    # labels and title
```

```
    ax.set(
        xlabel="training episode",
        ylabel="success rate (MA100)",
        ylim=(-0.02, 1.02),
        title=f"E1{'a' if env_kind == 'original' else 'b'}. Success rate across seeds, "
        f"{ENV_LABEL[env_kind]} env",
    )
    ax.legend(loc="upper left")  # legend
save_fig(fig, "E1_multi_seed")  # save and show
```

```
[lab-stamp] 2026-11-09 17:24:56 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | extra - multi-seed training
17:26:47  Q-learning | original seed 1109  111.2 s  last-500 success 0.94
17:28:27  SARSA | original      seed 1109  100.2 s  last-500 success 0.99
17:30:43  Q-learning | sticky   seed 1109  136.0 s  last-500 success 0.82
17:32:51  SARSA | sticky        seed 1109  127.8 s  last-500 success 0.73
17:34:41  Q-learning | original seed 2109  110.2 s  last-500 success 0.93
17:36:21  SARSA | original      seed 2109   99.7 s  last-500 success 0.93
17:38:46  Q-learning | sticky   seed 2109  144.4 s  last-500 success 0.52
17:40:54  SARSA | sticky        seed 2109  127.9 s  last-500 success 0.67
```

	success rate	gentle-arrival rate	mean reward	mean episode length	pushes / episode	final avg max Q
agent						
Q-learning | original	0.993 ± 0.006	0.223 ± 0.353	-146.067 ± 7.782	146.067 ± 7.782	121.740 ± 5.305	-50.467 ± 0.262
SARSA | original	0.970 ± 0.052	0.043 ± 0.075	-168.063 ± 3.733	168.063 ± 3.733	137.203 ± 19.195	-51.874 ± 0.201
Q-learning | sticky	0.930 ± 0.056	0.917 ± 0.057	-167.410 ± 10.455	161.337 ± 8.682	96.633 ± 1.936	-60.240 ± 0.457
SARSA | sticky	0.960 ± 0.030	0.933 ± 0.055	-178.520 ± 4.817	168.733 ± 5.010	110.400 ± 1.327	-63.237 ± 0.064



Interpretation (3 seeds).

- **Original env.** Greedy success is 0.91–1.00 on every seed (mean 0.99 for Q-learning, 0.97 for SARSA). Gentle arrival varies enormously between seeds (Q-learning 0.22 ± 0.35), which confirms that without the bonus arrival speed is an accident of the learned cycle.
- **Sticky env.** On average Q-learning has the higher return (−167 vs −179) and shorter episodes (161 vs 169 steps), on 2 of the 3 seeds, and selects fewer pushes on every seed (97 vs 110 on average). SARSA has slightly higher and more consistent success (0.96 ± 0.03 vs 0.93 ± 0.06). Seed 109 alone suggested Q-learning wins on every metric, which shows why several seeds are needed.
- **The value gap is larger with sticky actuators on every seed.** Q-learning − SARSA in final average max Q is 2.45 / 3.06 / 3.48 (sticky) against 1.88 / 1.12 / 1.21 (original) for seeds 109 / 1109 / 2109, i.e. ≈ 3.0 vs ≈ 1.4 on average. This is the strongest evidence for question (e)-1.
- **Learning curves (E1).** They overlap within the min–max bands; the sticky env is clearly slower to learn and has wider bands.

5.3 Implemented improvement: an agent that remembers its own last selected action

The modified environment is non-Markovian because `a_prev` is hidden (discussed in part (e)). The agent cannot see `a_prev`, but it *does* know which action it selected on the previous step. That action equals `a_prev` whenever the previous switch was not blocked: ≈ 80 % of steps even under a uniformly random policy (V3 audit: 1 − 12 037 / 60 000 = 0.799), and more under a learned policy, which switches less often. We append it to the state, giving a 36 × 28 × 3 × 3 Q-table. The environment is unchanged and exposes nothing new; the extra information lives in the agent's own memory.

The table is 3× larger, so we compare two budgets (seed 109, same config otherwise):

- **equal episodes:** 10 000 episodes for both representations;
- **longer budget:** 30 000 episodes, bringing the memory agent's average updates per table entry close to the 10 000-episode baseline. Actual visits are highly uneven, so this is only an average. The observation-only agent also receives 30 000 episodes.

The historical long runs use `eps_decay_fraction=0.5`, so epsilon reaches its floor at 15 000 episodes rather than 5 000. This also changes exploration in their first 10 000 episodes: they are separate runs, not continuations. Comparisons across budgets therefore confound training duration and exploration schedule. Within each budget both state representations share the same schedule. Last selected action is only a proxy for the hidden executed action; it does not restore a Markov state.

In [76]:

```
lab_stamp("extra - memory-augmented agents")
# Historical long-budget comparison: epsilon decay stretches to 15 000 episodes.
# This is not a continuation or a controlled data-only comparison with the 10k runs.
CFG_LONG = replace(CFG, n_episodes=3 * CFG.n_episodes)
MEM = {}  # (algorithm, state mode, episodes) -> RunResult
for alg in ("q_learning", "sarsa"):  # both algorithms
    # baseline: already trained in section 3
    MEM[(alg, "obs", CFG.n_episodes)] = RUNS[(alg, "modified")]
    # the three new runs
    for mode, cfg_m in (("memory", CFG), ("obs", CFG_LONG), ("memory", CFG_LONG)):
        MEM[(alg, mode, cfg_m.n_episodes)] = train_agent(
            alg,
            "modified",
            cfg=cfg_m,
            seed=SEED,
            state_mode=mode,
            # 4 progress lines per run
            log_every=cfg_m.n_episodes // 4,
        )
mem_rows = []  # evaluation rows
for (alg, mode, n_ep), r in MEM.items():  # every memory-experiment run
    s = summarise(evaluate_greedy(r, "modified"))  # same 100 test seeds as section 4.6
    mem_rows.append(
        {
            "algorithm": ALGO_LABEL[alg],
            "state": (
                "obs + last selected action" if mode == "memory" else "observation only"
            ),
            "episodes": n_ep,
            "Q entries": r.agent.Q.size,
            # average data per table entry; actual visitation is uneven
            "updates per Q entry": round(r.log["length"].sum() / r.agent.Q.size),
            "train success (last 1000)": r.log["success"].iloc[-1000:].mean(),
            **{
                m: s[m]
                for m in (
                    "success rate",
                    "gentle-arrival rate",
                    "mean reward",
                    "mean episode length",
                )
            },
        }
    )
mem_table = pd.DataFrame(mem_rows)  # results table
# save to CSV
mem_table.to_csv(os.path.join(OUT_DIR, "memory_agent_eval.csv"), index=False)
```

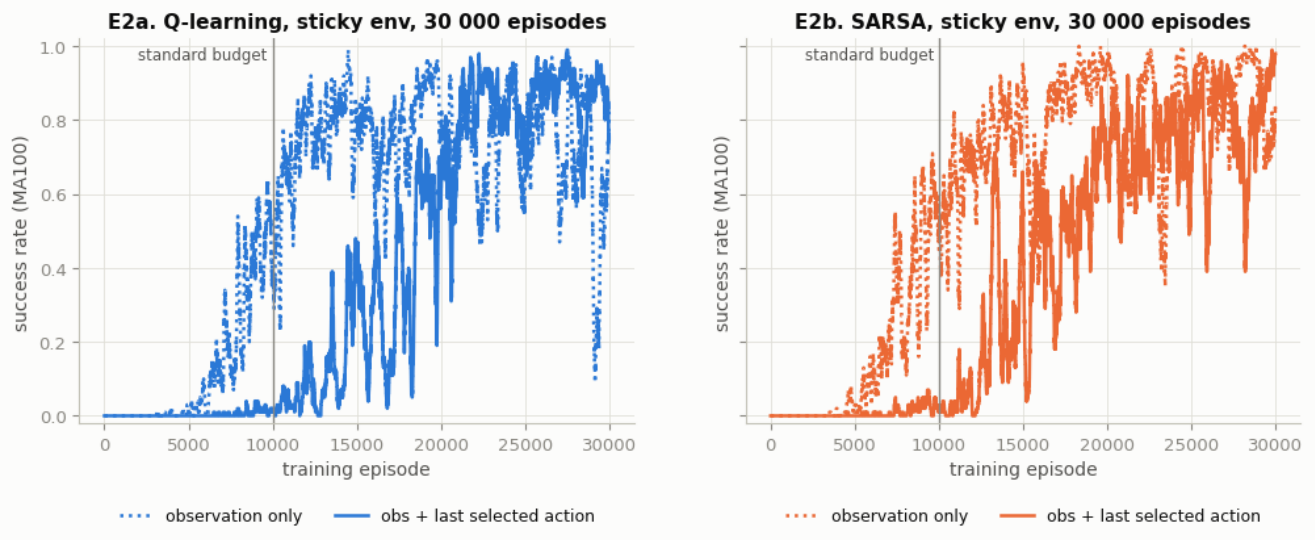
```
display(mem_table.round(3))  # show the table

fig, axes = plt.subplots(1, 2, figsize=(12, 3.8), sharey=True)  # two panels
# one panel per algorithm, 30 000-episode runs
for ax, alg in zip(axes, ("q_learning", "sarsa")):
    # both representations
    for mode, style, name in (
        ("obs", ":", "observation only"),
        ("memory", "-", "obs + last selected action"),
    ):
        # success rate, trailing mean
        y = trailing_mean(
            MEM[(alg, mode, CFG_LONG.n_episodes)].log["success"].astype(float)
        )
        ax.plot(y, color=ALGO_COLOR[alg], ls=style, label=name)  # curve
    # the standard 10 000-episode budget
    ax.axvline(CFG.n_episodes, color=INK_MUTED, lw=1)
    # label the budget line
    ax.text(
        CFG.n_episodes,
        1.0,
        "standard budget ",
        ha="right",
        va="top",
        fontsize=8.5,
        color=INK_2,
    )
    # labels and title
    ax.set(
        xlabel="training episode",
        ylabel="success rate (MA100)",
        ylim=(-0.02, 1.02),
        title=f"E2{'a' if alg == 'q_learning' else 'b'}. {ALGO_LABEL[alg]}, sticky "
        "env, 30 000 episodes",
    )
    # legend below the panel, clear of the curves
    ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.18), ncol=2)
save_fig(fig, "E2_memory_agent")  # save and show
```

```
[lab-stamp] 2026-11-09 17:41:06 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | extra -
memory-augmented agents
[lab-stamp] 2026-11-09 17:41:06 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | train
Q-learning | sticky | memory | seed 109 | 10000 ep
  R100 / succ / gentle / push = means over the previous 100 episodes
  maxQ = average of max_a Q(s,a) over the fixed validation states
17:41:42  ep  2500/10000  eps 0.224  R100  -240.0  succ 0.00  gentle 0.00  push 133.5  maxQ  -24.7
17:42:21  ep  5000/10000  eps 0.050  R100  -240.0  succ 0.01  gentle 0.01  push 135.1  maxQ  -35.0
17:43:01  ep  7500/10000  eps 0.050  R100  -240.9  succ 0.00  gentle 0.00  push 136.4  maxQ  -41.6
17:43:39  ep 10000/10000  eps 0.050  R100  -239.4  succ 0.01  gentle 0.01  push 132.5  maxQ  -47.2
17:43:39  done in 152.6 s -> outputs/log_q_learning_modified_memory_seed109_ep10000.csv
[lab-stamp] 2026-11-09 17:43:39 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | train
Q-learning | sticky | seed 109 | 30000 ep
  R100 / succ / gentle / push = means over the previous 100 episodes
  maxQ = average of max_a Q(s,a) over the fixed validation states
17:45:23  ep  7500/30000  eps 0.224  R100  -232.0  succ 0.12  gentle 0.11  push 125.5  maxQ  -52.7
17:47:01  ep 15000/30000  eps 0.050  R100  -188.7  succ 0.83  gentle 0.80  push 106.7  maxQ  -61.6
17:48:39  ep 22500/30000  eps 0.050  R100  -202.1  succ 0.55  gentle 0.53  push  97.6  maxQ  -64.5
17:50:20  ep 30000/30000  eps 0.050  R100  -190.0  succ 0.80  gentle 0.79  push  97.9  maxQ  -65.8
17:50:20  done in 400.6 s -> outputs/log_q_learning_modified_obs_seed109_ep30000.csv
[lab-stamp] 2026-11-09 17:50:20 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | train
Q-learning | sticky | memory | seed 109 | 30000 ep
  R100 / succ / gentle / push = means over the previous 100 episodes
  maxQ = average of max_a Q(s,a) over the fixed validation states
17:52:06  ep  7500/30000  eps 0.224  R100  -240.4  succ 0.00  gentle 0.00  push 134.6  maxQ  -38.5
17:53:57  ep 15000/30000  eps 0.050  R100  -220.9  succ 0.40  gentle 0.37  push 131.4  maxQ  -52.3
17:55:39  ep 22500/30000  eps 0.050  R100  -180.8  succ 0.89  gentle 0.80  push 116.0  maxQ  -59.4
17:57:12  ep 30000/30000  eps 0.050  R100  -186.6  succ 0.75  gentle 0.75  push 108.2  maxQ  -61.5
17:57:12  done in 411.9 s -> outputs/log_q_learning_modified_memory_seed109_ep30000.csv
[lab-stamp] 2026-11-09 17:57:12 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | train
SARSA | sticky | memory | seed 109 | 10000 ep
  R100 / succ / gentle / push = means over the previous 100 episodes
  maxQ = average of max_a Q(s,a) over the fixed validation states
17:57:44  ep  2500/10000  eps 0.224  R100  -240.1  succ 0.00  gentle 0.00  push 133.6  maxQ  -27.4
17:58:20  ep  5000/10000  eps 0.050  R100  -240.3  succ 0.00  gentle 0.00  push 134.4  maxQ  -36.2
17:58:55  ep  7500/10000  eps 0.050  R100  -240.9  succ 0.00  gentle 0.00  push 136.4  maxQ  -42.8
17:59:29  ep 10000/10000  eps 0.050  R100  -238.2  succ 0.02  gentle 0.02  push 131.0  maxQ  -48.4
17:59:30  done in 137.5 s -> outputs/log_sarsa_modified_memory_seed109_ep10000.csv
```

```
[lab-stamp] 2026-11-09 17:59:30 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | train
SARSA | sticky | seed 109 | 30000 ep
  R100 / succ / gentle / push = means over the previous 100 episodes
  maxQ = average of max_a Q(s,a) over the fixed validation states
18:01:06  ep  7500/30000  eps 0.224  R100  -219.4  succ 0.35  gentle 0.33  push 124.9  maxQ  -60.5
18:02:39  ep 15000/30000  eps 0.050  R100  -176.0  succ 0.94  gentle 0.94  push  99.0  maxQ  -67.7
18:04:06  ep 22500/30000  eps 0.050  R100  -154.2  succ 0.96  gentle 0.96  push  96.2  maxQ  -67.5
18:05:31  ep 30000/30000  eps 0.050  R100  -184.2  succ 0.84  gentle 0.83  push  97.9  maxQ  -67.9
18:05:31  done in 361.0 s -> outputs/log_sarsa_modified_obs_seed109_ep30000.csv
[lab-stamp] 2026-11-09 18:05:31 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 | train
SARSA | sticky | memory | seed 109 | 30000 ep
  R100 / succ / gentle / push = means over the previous 100 episodes
  maxQ = average of max_a Q(s,a) over the fixed validation states
18:07:11  ep  7500/30000  eps 0.224  R100  -240.6  succ 0.02  gentle 0.02  push 138.0  maxQ  -43.7
18:08:53  ep 15000/30000  eps 0.050  R100  -206.3  succ 0.57  gentle 0.57  push 127.4  maxQ  -55.0
18:10:27  ep 22500/30000  eps 0.050  R100  -202.1  succ 0.62  gentle 0.61  push 117.2  maxQ  -61.6
18:11:57  ep 30000/30000  eps 0.050  R100  -165.5  succ 0.98  gentle 0.91  push 102.1  maxQ  -64.6
18:11:58  done in 386.4 s -> outputs/log_sarsa_modified_memory_seed109_ep30000.csv
```

	algorithm	state	episodes	Q entries	updates per Q entry	train success (last 1000)	success rate	gentle-arrival rate	mean reward	mean episode length
0	Q-learning	observation only	10000	3024	624	0.826	0.98	0.98	-158.996	154.99
1	Q-learning	obs + last selected action	10000	9072	220	0.024	0.01	0.01	-238.941	199.72
2	Q-learning	observation only	30000	3024	1802	0.454	0.75	0.72	-190.349	179.48
3	Q-learning	obs + last selected action	30000	9072	618	0.873	0.94	0.92	-176.566	168.03
4	SARSA	observation only	10000	3024	624	0.761	0.93	0.87	-180.032	168.29
5	SARSA	obs + last selected action	10000	9072	220	0.057	0.02	0.02	-241.011	199.40
6	SARSA	observation only	30000	3024	1765	0.781	0.59	0.55	-195.202	179.60
7	SARSA	obs + last selected action	30000	9072	620	0.906	0.99	0.97	-155.383	151.31



Interpretation of the memory experiment. With 10 000 episodes, memory-agent greedy success is only 1-2%; its larger table has roughly 220 average updates per entry versus 624 for the baseline. At 30 000 episodes, memory-agent success is 94% and 99%, compared with 75% and 59% for observation-only agents at the same budget and epsilon schedule. This supports the usefulness of a last-selected-action proxy on this seed.

The 30 000-episode baseline is a fresh run with a slower epsilon decay. Its lower final greedy success cannot be attributed to additional data alone. These experiments also do not prove that memory needs exactly three times the data or makes the observation Markovian. Multiple seeds and a fixed 5 000-episode decay horizon across budgets would provide a better controlled comparison.

6. Part (e) — Discussion

(e)-1 Does the sticky actuator increase the difference between the value estimates of Q-learning and SARSA?

Yes — in our runs the gap is clearly larger in the modified (sticky) environment, but this comparison cannot attribute it to stickiness alone. In D2a and D2b Q-learning's average max_a Q(s,a) on the fixed validation set lies above SARSA's in both environments. Over the last 500 episodes the Q-learning − SARSA gap is 2.451 in the modified environment against 1.883 in the original one for seed 109 (3.9 % vs 3.6 % of SARSA's value), and it is larger in the modified environment on **all three seeds** (≈ 3.0 vs ≈ 1.4 on average, section 5.2).

Why the gap grows:

1. **SARSA prices in its own future exploration; Q-learning does not.** SARSA's target Q(s′, a′) uses the action the ε-greedy policy actually chose next. With a sticky actuator an exploratory command, once executed, becomes `a_prev`, and every later attempt to switch back is ignored 30 % of the time, so one random action can push the car the wrong way for several steps. Q-learning's target max_a′ Q(s′, a′) assumes greedy future actions. It still sees actuator noise in the transitions it samples, but never charges itself for its own exploration.
2. **Plausible further contributors** are maximisation bias (the max over noisy estimates is biased upwards, Session 9) and observation aliasing. This experiment does not measure them separately.

Caveat. The modified environment changes three things together: stickiness, the push penalty and the gentle-arrival bonus. The larger gap therefore belongs to the complete modification. An ablation that varies only the stick probability while keeping the original reward would isolate the actuator's effect.

(e)-2 Why does the hidden `a_prev` make the environment non-Markovian with respect to the observation, and how does it affect credit assignment?

The physical Markov state is (position, velocity, last executed action); the agent observes only (position, velocity), so it effectively faces a POMDP. **Example:** at the same position and velocity, selecting *push right* executes right with probability 1 if `a_prev` = right, but executes **left** with probability 0.30 if `a_prev` = left. The next-observation distribution therefore depends on history, so P(o′ | o, a) is not a function of (o, a) alone. Discretisation adds further aliasing. (The 200-step time limit is truncation, not a terminal state, and does not remove the bootstrap.)

Effects on credit assignment:

- **Updating the selected command is still correct.** It is the only decision the agent makes; the actuator is part of the environment.
- **Values are averaged over hidden histories.** Q(o, a) mixes the outcomes of different hidden actuator states, weighted by a history distribution that depends on the agent's own (changing) policy. The observation-only table cannot give different values to the same observation with different actuator histories.
- **Blocked commands blur the timing of credit.** A blocked command is discarded, not queued. The car keeps executing `a_prev` until a later selection gets through, so the consequence of a decision appears only after a random delay, and one-step TD spreads the credit over neighbouring steps.
- **No convergence guarantee.** A non-Markov state, constant learning rates and finite training mean the standard tabular convergence results cannot be invoked. This matches the slower, noisier learning in the modified environment (D1b, D3, wider bands in E1b).

(e)-3 Do the energy penalty and the gentle-arrival bonus encourage fewer pushes or a slower, more controlled arrival?

Both behaviours are observed, consistent with the intended incentives.

- **Fewer pushes (D4):** the share of steps with a selected push ends near 0.62 in the modified environment and 0.84 in the original. In the fixed-seed greedy tests the modified Q-learning and SARSA agents select 95.02 and 111.64 pushes per episode, against 119.66 and 151.47 for their original counterparts.

- **Policy maps (D5):** more 'no push' cells (≈ 36 % vs ≈ 20 % of visited cells) and push-left selections just before the flag at high speed, consistent with braking.
- **Controlled arrival (D6, table 4.7):** the modified agents arrive at a median v ≈ 0.024. Among arrivals, 100 % (Q-learning) and 94.62 % (SARSA) are at or below 0.03, and the full gentle criterion holds for 98/98 = 100 % and 87/93 = 93.55 % (one SARSA arrival happens on step 200, so it earns no bonus). Over all 100 tests the gentle rates are 98 % and 87 %.

Caveat. These patterns do not separate the effects of the penalty, the bonus and stickiness: the original Q-learning policy already arrives gently in 63 of 100 tests without any bonus. Penalty-only, bonus-only and stickiness-only ablations, with the same seeds and settings, would be needed for causal attribution. A static action map also cannot prove that a cell causes braking.

(e)-4 Which algorithm performs better under sticky actuators? Is this consistent with on-policy vs off-policy theory?

By the main objective (expected return), Q-learning performs better; SARSA is slightly more consistent in reaching the flag.

- **Seed 109:** Q-learning has the higher greedy mean return (−158.996 vs −180.032), higher success (98 % vs 93 %) and shorter episodes (154.99 vs 168.29 steps).
- **Three seeds:** Q-learning's mean return is −167.410 vs −178.520 and its mean length 161.337 vs 168.733 steps. SARSA has slightly higher mean success (96 % vs 93 %) with a smaller spread. Three seeds are descriptive and do not establish statistical significance.

This is consistent with theory, in a mild form. Q-learning (off-policy) learns the value of the greedy policy, so its greedy test policy is the more efficient one. SARSA (on-policy) learns the value of the ε-greedy behaviour it actually follows, including the extra cost that sticky actuators add to exploratory mistakes, so it ends more conservative: longer episodes, more pushes, steadier success. This is the Cliff-Walking pattern (Sutton & Barto, Example 6.6). It is weak here because a mistake in MountainCar costs a few extra steps, not −100, and because both algorithms experience the actuator noise in their samples. The on/off-policy difference concerns only the agent's own ε-exploration. Under partial observability and finite training, theory does not guarantee that SARSA is safer or that Q-learning reaches the optimum.

(e)-5 One limitation of the experimental setup and one possible improvement

Limitation. The modified environment changes stickiness, the push penalty and the bonus *at the same time*, and the main results use one training seed (three in section 5.2). So we can describe how the complete modified task differs, but we cannot attribute any difference (value gap, fewer pushes, gentler arrivals) to one modification. The observation also omits the actuator state, so tabular learning runs on an aliased, non-Markovian state.

Improvement. Run a matched ablation: stickiness-only, penalty-only and bonus-only variants, each trained with the same seeds, grid, α, γ and ε schedule, and more training seeds. A second improvement, tried in section 5.3, is to give the agent its own last selected action as a proxy for the hidden `a_prev`. That helped on this seed, but the state is still partially observed, and its 30 000-episode runs also stretch the ε decay. A follow-up should fix the absolute decay horizon (5 000 episodes) across budgets and compare state representations at equal episode budgets, or use tile coding or a small Q-network (Lectures 11–12) so the extra input generalises instead of tripling the table.

Appendix — Virtual-lab evidence

Screenshots of this notebook running in the BITS virtual lab (with visible date/time) are attached on the following pages. The `[lab-stamp]` lines in the outputs above record the host, user and time of each stage.

In [80]:

```
lab_stamp("notebook complete")
# total wall-clock time
print(
    f"total run time: {(time.perf_counter() - NOTEBOOK_T0) / 60:.1f} min | outputs "
    f"saved in ./{OUT_DIR}/"
)
```

```
[lab-stamp] 2026-11-09 18:12:09 | host=ip-10-3-17-64.ap-south-1.compute.internal | user=s2025ag05066 |
notebook complete
total run time: 56.0 min | outputs saved in ./outputs/
```

In [81]:

```
# ---- Run-environment report: records where, when and with what this notebook ran ----
import os  # environment variables, CPU count, folder sizes
import sys  # Python version, executable and platform name
import time  # high-resolution timer (matches NOTEBOOK_T0 from the setup cell)
import socket  # host name and IP address of the machine
import getpass  # login name of the user
import platform  # operating-system and processor details
import shutil  # disk usage and locating optional tools
import subprocess  # optional GPU query (nvidia-smi)
import datetime as dt  # local and UTC timestamps
from importlib import metadata  # versions of the installed packages


def safe(fn, default="n/a"):
    """Call fn() and return its result, or `default` if the machine does not allow
    it."""
    try:  # may be blocked on a restricted VM
        return fn()  # the requested value
    except Exception:  # restricted VM / missing feature: never crash the report
        return default  # placeholder instead of an error


def read_key_values(path, sep):
    """Read a 'KEY<sep>VALUE' text file (e.g. /proc/meminfo) into a dict."""
    pairs = {}  # parsed entries
    with open(path) as fh:  # small system text file
        for line in fh:  # one entry per line
            if sep in line:  # skip blank or malformed lines
                key, value = line.split(sep, 1)  # split at the first separator
                pairs[key.strip()] = value.strip().strip('"')  # drop quotes/spaces
    return pairs  # parsed key/value pairs


def ram_gb():
    """Return (total, available) physical memory in GB on Linux or Windows."""
    if os.name == "nt":  # Windows: ask the Win32 API (no extra package needed)
        import ctypes  # foreign-function interface of the standard library

        class MemStatus(ctypes.Structure):
            """Win32 MEMORYSTATUSEX structure filled by GlobalMemoryStatusEx."""

            # first two fields of MEMORYSTATUSEX
            _fields_ = [("dwLength", ctypes.c_ulong), ("dwMemoryLoad", ctypes.c_ulong)]
            # the seven 64-bit counters that follow, in API order
            _fields_ += [
                (name, ctypes.c_ulonglong)
                for name in (
                    "ullTotalPhys",
                    "ullAvailPhys",
                    "ullTotalPageFile",
                    "ullAvailPageFile",
                    "ullTotalVirtual",
                    "ullAvailVirtual",
                    "ullAvailExtendedVirtual",
                )
            ]
```

```
        stat = MemStatus()  # empty structure
        stat.dwLength = ctypes.sizeof(MemStatus)  # the API requires its own size
        ctypes.windll.kernel32.GlobalMemoryStatusEx(ctypes.byref(stat))  # fill it
        return stat.ullTotalPhys / 1e9, stat.ullAvailPhys / 1e9  # bytes -> GB
    mem = read_key_values("/proc/meminfo", ":")  # Linux: "MemTotal: 16384 kB"
    to_gb = lambda key: int(mem[key].split()[0]) / 1e6  # kB -> GB
    return to_gb("MemTotal"), to_gb("MemAvailable")  # total, available


def ip_address():
    """Primary IPv4 address of the machine (a UDP 'connect' sends no packet)."""
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)  # UDP socket
    try:  # connect, then always close
        sock.connect(("10.255.255.255", 1))  # lets the OS pick the outgoing interface
        return sock.getsockname()[0]  # address of that interface
    finally:
        sock.close()  # release the socket


def os_rows():
    """OS-specific identity: distribution + desktop session on Linux, domain on
    Windows."""
    if os.name == "nt":  # Windows machine
        return [
            (
                "Windows domain / computer",
                f"{os.environ.get('USERDOMAIN', 'n/a')} / "
                f"{os.environ.get('COMPUTERNAME', 'n/a')}",
            ),
            ("OS edition", safe(platform.win32_edition)),  # e.g. Professional
            ("desktop session", os.environ.get("SESSIONNAME", "n/a")),  # RDP-Tcp#N
        ]
    release = safe(lambda: read_key_values("/etc/os-release", "="), {})  # distro info
    session = (  # remote-desktop evidence: X display, xrdp flag, desktop name
        f"DISPLAY={os.environ.get('DISPLAY', 'n/a')} | "
        f"xrdp={'yes' if 'XRDP_SESSION' in os.environ else 'no'} | "
        f"desktop={os.environ.get('XDG_CURRENT_DESKTOP', 'n/a')}"
    )
    return [
        ("Linux distribution", release.get("PRETTY_NAME", platform.platform())),
        ("kernel", platform.release()),  # Linux kernel version
        ("desktop session", session),
    ]


def folder_size_mb(path):
    """Number of files and total size (MB) inside `path`, searched recursively."""
    files = [os.path.join(r, f) for r, _, fs in os.walk(path) for f in fs]  # all files
    return len(files), sum(os.path.getsize(f) for f in files) / 1e6  # count, MB


def gpu_name():
    """Name of an NVIDIA GPU if nvidia-smi exists, else a note (none is needed)."""
    if not shutil.which("nvidia-smi"):  # no NVIDIA driver tools on this machine
        # no GPU tools installed
        return "none detected (not needed: tabular methods run on the CPU)"
    out = subprocess.run(  # ask the driver for the GPU model
        ["nvidia-smi", "--query-gpu=name", "--format=csv,noheader"],
        capture_output=True,
        text=True,
        timeout=10,
    )
    return out.stdout.strip() or "nvidia-smi found, no GPU reported"  # model name(s)


def version(pkg):
    """Installed version of a Python package, or 'not installed'."""
    return safe(lambda: metadata.version(pkg), "not installed")  # metadata lookup


now_local = dt.datetime.now().astimezone()  # local time with the VM's time zone
now_utc = dt.datetime.now(dt.timezone.utc)  # same instant in UTC
host = socket.gethostname()  # machine name
total_gb, free_gb = safe(ram_gb, (float("nan"), float("nan")))  # memory in GB
```

```
disk = safe(lambda: shutil.disk_usage(os.getcwd()), None)  # disk of this folder
in_venv = sys.prefix != getattr(sys, "base_prefix", sys.prefix)  # venv active?
env_name = (  # which Python environment the kernel runs in
    os.environ.get("CONDA_DEFAULT_ENV")  # conda environment name, if any
    or (os.path.basename(sys.prefix) + " (venv)" if in_venv else "system Python")
)
n_out, mb_out = safe(lambda: folder_size_mb("outputs"), (0, 0.0))  # outputs/ folder
cfg = globals().get("CFG")  # experiment configuration, if this notebook defined it
t0 = globals().get("NOTEBOOK_T0")  # start time recorded in the setup cell, if any
fallback_ip = safe(lambda: socket.gethostbyname(host))  # used if the UDP trick fails

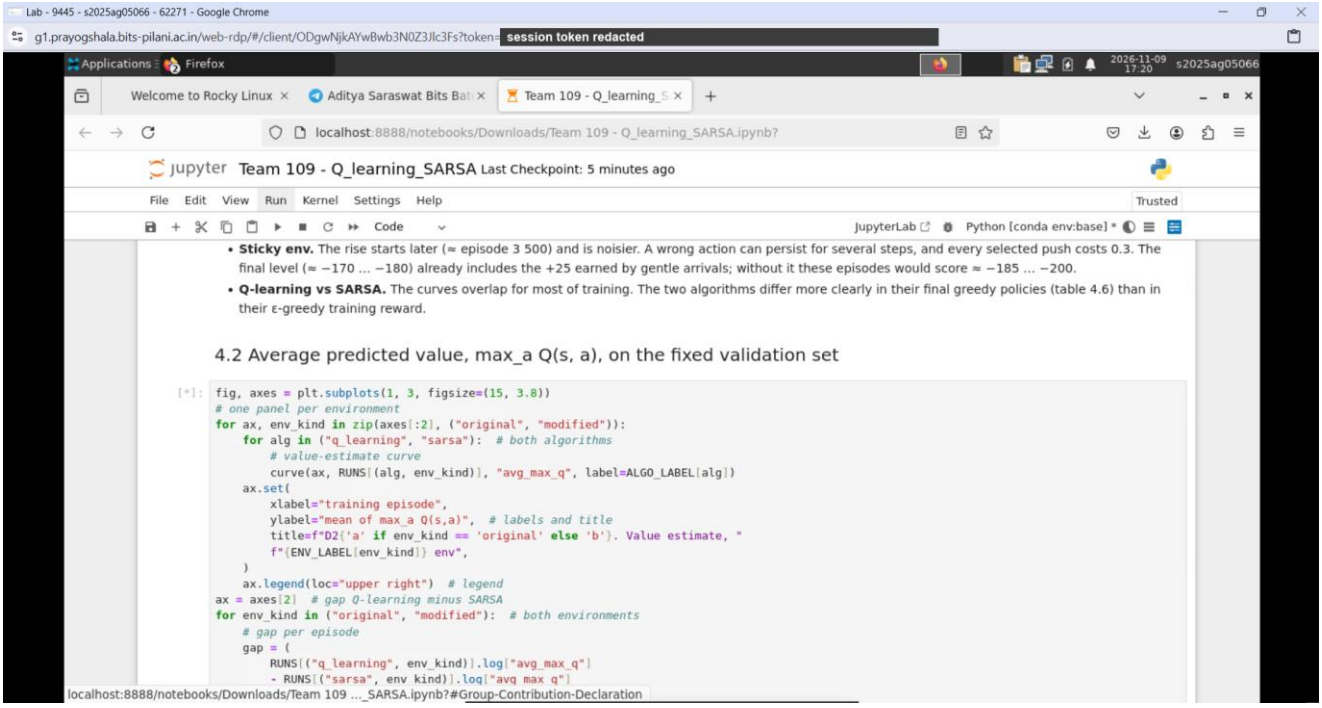
rows = [  # (label, value) pairs of the report
    ("report time (local)", now_local.strftime("%Y-%m-%d %H:%M:%S %Z (UTC%z)")),
    ("report time (UTC)", now_utc.strftime("%Y-%m-%d %H:%M:%S")),
    ("host name", host),
    ("IP address", safe(ip_address, fallback_ip)),
    ("user", safe(getpass.getuser)),
    ("operating system", platform.platform()),
    *os_rows(),  # Linux distribution / Windows domain details
    ("machine / processor", f"{platform.machine()} / {platform.processor() or 'n/a'}"),
    ("logical CPUs", os.cpu_count()),
    ("RAM total / available", f"{total_gb:.1f} GB / {free_gb:.1f} GB"),
    (
        "disk (this folder)",
        (
            f"{disk.total / 1e9:.1f} GB total, " f"{disk.free / 1e9:.1f} GB free"
            if disk
            else "n/a"
        ),
    ),
    ("GPU", safe(gpu_name)),
    ("Python", f"{platform.python_version()} ({platform.python_implementation()})"),
    ("Python executable", sys.executable),
    ("Python environment", env_name),
    ("working folder", os.getcwd()),
    ("outputs/ folder", f"{n_out} files, {mb_out:.1f} MB"),
    ("gymnasium / numpy", f"{version('gymnasium')} / {version('numpy')}"),
    ("pandas / matplotlib", f"{version('pandas')} / {version('matplotlib')}"),
    ("ipykernel / jupyter_core", f"{version('ipykernel')} / {version('jupyter_core')}"),
]
if cfg is not None:  # add the experiment settings when run inside our notebook
    rows.append(  # experiment settings when run inside this notebook
        (
            "experiment",
            f"seed {globals().get('SEED', 'n/a')} | grid "
            f"{cfg.n_pos_bins}x{cfg.n_vel_bins} | alpha {cfg.alpha} | "
            f"gamma {cfg.gamma} | {cfg.n_episodes} episodes",
        )
    )
if t0 is not None:  # total wall-clock time since the setup cell
    # time since the setup cell
    rows.append(("notebook run time", f"{(time.perf_counter() - t0) / 60:.1f} min"))

width = max(len(label) for label, _ in rows)  # align the colon column
print("=" * 25, "RUN ENVIRONMENT (virtual lab evidence)", "=" * 25)  # header line
for label, value in rows:  # one aligned line per item
    print(f"{label:<{width}} : {value}")  # one aligned report line
print("=" * 90)  # footer line
```

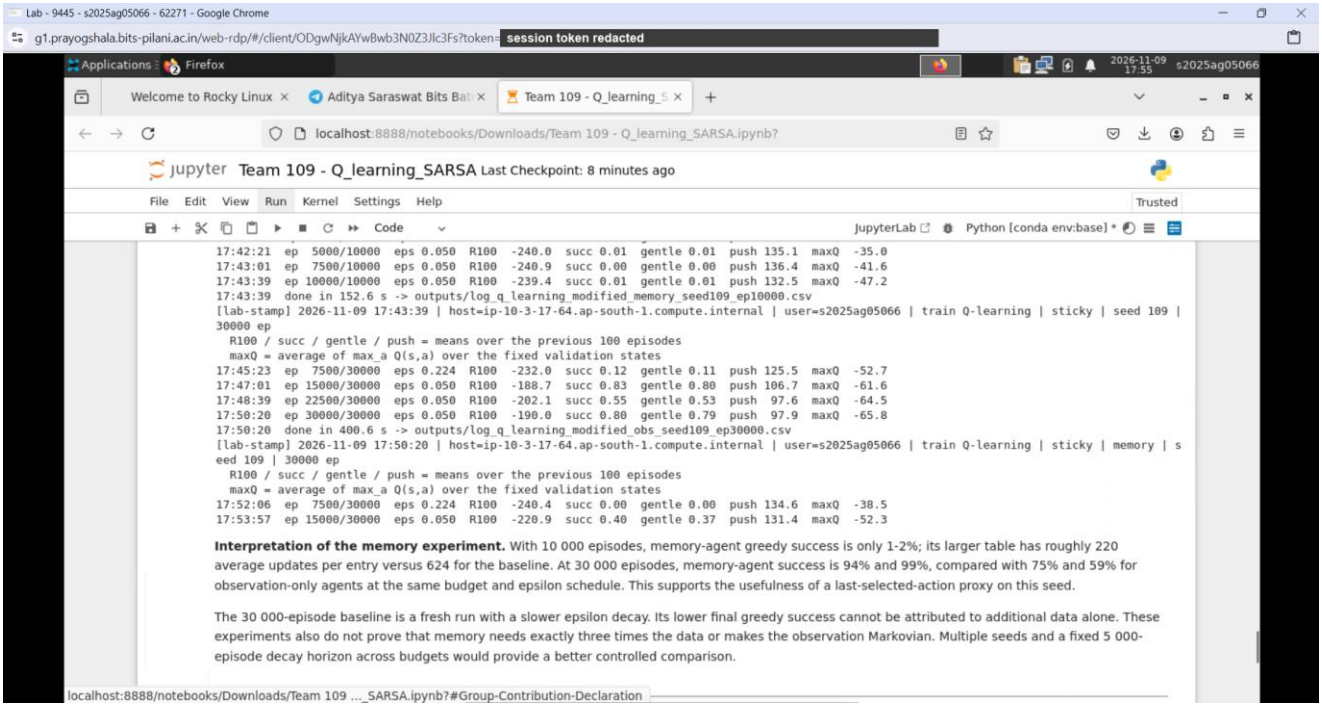
```
======================== RUN ENVIRONMENT (virtual lab evidence) ========================
report time (local)      : 2026-11-09 18:12:09 IST (UTC+0530)
report time (UTC)        : 2026-11-09 12:42:09
host name                : ip-10-3-17-64.ap-south-1.compute.internal
IP address               : 10.3.17.64
user                     : s2025ag05066
operating system         : Linux-5.14.0-503.14.1.el9_5.x86_64-x86_64-with-glibc2.34
Linux distribution       : Rocky Linux 9.5 (Blue Onyx)
kernel                   : 5.14.0-503.14.1.el9_5.x86_64
desktop session          : DISPLAY=:10.0 | xrdp=yes | desktop=XFCE
machine / processor      : x86_64 / x86_64
logical CPUs             : 4
RAM total / available    : 16.0 GB / 13.1 GB
disk (this folder)       : 39.7 GB total, 20.5 GB free
GPU                      : none detected (not needed: tabular methods run on the CPU)
Python                   : 3.12.7 (CPython)
Python executable        : /home/cloud/anaconda3/bin/python
Python environment       : base
```

```
working folder           : /home/s2025ag05066/Downloads
outputs/ folder          : 73 files, 29.9 MB
gymnasium / numpy        : 1.4.0 / 1.26.4
pandas / matplotlib      : 2.2.3 / 3.10.1
ipykernel / jupyter_core : 6.29.5 / 5.7.2
experiment               : seed 109 | grid 36x28 | alpha 0.1 | gamma 0.99 | 10000 episodes
notebook run time        : 56.0 min
================================================================================
```

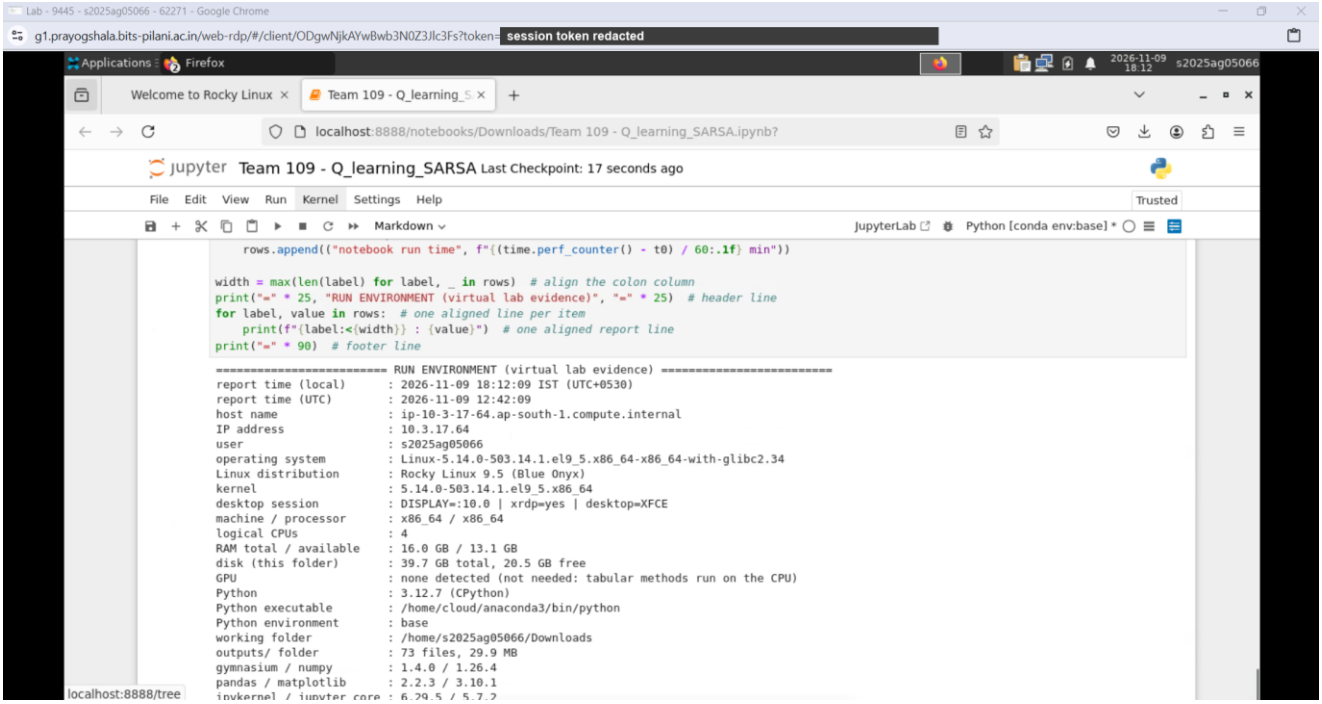
Lab screenshot 1 — notebook running in the virtual lab (cell In [52]): VM clock 2026-11-09 17:20, the section 4.2 value-estimate cell executing ([*]) in Jupyter on the lab VM as user s2025ag05066 (session token in the address bar redacted).



Lab screenshot 2 — lab-stamped training progress (cell In [76]): VM clock 17:55, progress and [lab-stamp] lines of the section 5.3 runs (30 000 episodes) on host ip-10-3-17-64.



Lab screenshot 3 — run-environment report (cell In [81]): VM clock 18:12, lab host, user, Rocky Linux over xrdp and library versions, the same report as printed above.



Lab screenshot 4 — BITS virtual-lab session: Prayogshala portal showing the Deep Reinforcement Learning (AIMLZG512) lab session RUNNING with 3 h 32 m used, user s2025ag05066 (password redacted).

